RWTH Aachen University



Master's Thesis

Comparative Analysis of Static, Heuristic, and ML-Driven Task Scheduling in Medallion Lakehouse ETL Pipelines

Gurusaran Sivakumar

Matriculation Number: 469304

M.Sc. Data Analytics & Decision Science

RWTH Business School

Submission Date: October 27, 2026

Supervisor and Examiners

First Examiner and Supervisor

Prof. Dr. Sven Müller

Chair of Data and Business Analytics

Second Examiner

Prof. Dr. Thomas Lontzek

Chair of Computational Economics

Abstract

This thesis compares Static, HEFT-based and offline XGBoost-derived task priorities in Apache Airflow across the Bronze, Silver and Gold layers of a Medallion Lakehouse. It examines realised task admission and layer processing time rather than full-pipeline makespan. The XGBoost model approximates an engineered priority formula built from task features; it does not learn from observed scheduling outcomes or adapt online.

The evaluation comprises 90 local runs, 45 Azure Databricks benchmark runs and 40 separate controlled cloud runs: 175 runs and 735 task observations. Five cloud repetitions per strategy and layer use a fixed single-node cluster. Controlled experiments vary prerequisite release, Gold opening pairs and permitted task concurrency.

In local tests, Static has the lowest Bronze mean and HEFT the lowest Gold mean, while Silver means are similar. In the main cloud benchmark, XGBoost has the lowest observed Bronze (187.639 s) and Gold (346.976 s) means; HEFT has the lowest Silver mean (152.387 s). Identical effective task selections in Silver and between Static and HEFT in Gold prevent attributing those timing differences to different priority decisions. In a separate Bronze study, delaying a prerequisite was associated with a 24% longer mean processing window. Gold concurrency reduces elapsed time through overlap and fewer sequential hand-offs, although the individual contributions of these mechanisms are not identified.

Results are sensitive to task dependencies, execution conditions and small cloud samples. The evidence supports conditional, layer-specific priority decisions, not a universally superior or optimally learned scheduler.

Contents

Abstract **I**

List of Tables **VI**

List of Figures **VII**

List of Abbreviations **VIII**

1 Introduction **1**

1.1 Motivation . . . 1
1.2 Scheduling Decisions and Performance Objectives . . . 2
1.3 Problem Statement . . . 3
1.4 Research Aim and Questions . . . 5
1.5 Analytical Expectations . . . 5
1.6 Scope and Delimitations . . . 5
1.7 Contributions . . . 6
1.8 Thesis Structure . . . 7

2 Related Work and Conceptual Background **8**

2.1 ETL Workflows and Scheduling Decisions . . . 8
2.2 Medallion Lakehouse and Execution Levels . . . 9
2.3 HEFT-Based Priority Heuristics . . . 10
2.4 Offline Machine Learning and Performance Prediction . . . 10
2.5 Resource Sharing, Interference, and Scheduling Overhead . . . 12
2.5.1 Shared-Resource Interference . . . 12
2.5.2 Scheduling and Control-Plane Overhead . . . 13
2.6 Scheduling Objectives, Policy Evaluation, and Observability . . . 13
2.6.1 Realised Decisions and Interpretation . . . 14
2.6.2 Observability and Interpretation . . . 14
2.7 Benchmarking, Statistical Evidence, and Workload Scope . . . 14
2.8 Research Synthesis and Positioning . . . 15

3 Methodology **18**

3.1 Research Design . . . 18
3.2 Experimental Architecture . . . 20
3.3 Pipeline Definition and Task Catalogue . . . 21
3.3.1 Bronze Layer . . . 22
3.3.2 Silver Layer . . . 22
3.3.3 Gold Layer . . . 23
3.4 Scheduling Strategy Implementations . . . 24

3.4.1 Static Priority Scheduling . . . 24
3.4.2 HEFT-Based Upward-Rank Prioritisation . . . 24
3.4.3 XGBoost Task-Priority Scoring . . . 25
3.4.4 Distinguishing Prediction Accuracy from a Scheduling Treatment . . . 27
3.5 Airflow Priority and Concurrency Semantics . . . 28
3.6 Implementation Structure and Treatment Fidelity . . . 28
3.6.1 DAG Factories and Strategy Wrappers . . . 28
3.6.2 HEFT and XGBoost Priority Generation . . . 29
3.6.3 Implementation Verification . . . 30
3.6.4 Traceability from Intended Priority to Executed Work . . . 30
3.7 Notebook Execution and Measurement Boundaries . . . 31
3.8 Local Benchmark Environment . . . 31
3.8.1 Local Run Structure and the Limits of Pairing . . . 32
3.9 Azure Databricks Environment . . . 33
3.9.1 Workspace and Compute . . . 33
3.9.2 Data and Transformations . . . 34
3.9.3 Metric Storage and Semantics . . . 34
3.10 Azure Databricks Benchmark Protocol . . . 36
3.11 Exploratory Mechanism Experiments . . . 37
3.11.1 Bronze Dependency-Unlocking Study . . . 37
3.11.2 Gold Opening-Pair Study . . . 37
3.11.3 Gold Concurrency-Sensitivity Study . . . 38
3.12 Experiment Automation, Filtering, and Data Validation . . . 38
3.13 Statistical and Robustness Analysis . . . 39
3.14 Validity and Reproducibility Controls . . . 40
3.15 Experimental Records and Provenance . . . 41
4 Results **42**
4.1 Experimental Overview . . . 42
4.2 Offline XGBoost Model Diagnostics . . . 42
4.3 Local Benchmark Results . . . 43
4.3.1 Descriptive Runtime Results . . . 43
4.3.2 Pairwise Local Comparisons . . . 43
4.3.3 Local Scheduling Delay . . . 45
4.4 Azure Databricks Benchmark Results . . . 46
4.4.1 Measurement and Comparison Scope . . . 46
4.4.2 Observed Processing Windows . . . 46
4.4.3 Comparison with Static and Sensitivity to Small Samples . . . 48
4.4.4 Model Predictions and Observed Execution Order . . . 49
4.4.5 Task-Level Execution and Timelines . . . 51
4.4.6 Local-to-Cloud Transfer . . . 53
4.4.7 Independent Output and Metrics Validation . . . 55

4.5 Bronze Dependency-Unlocking Experiment . . . 56
4.5.1 Reading the Bronze Intervention as a Timing Mechanism . . . 57
4.6 Gold Opening-Pair Experiment . . . 58
4.6.1 Interpreting Pair-Level Effects Across Sessions . . . 60
4.6.2 Run-Level Variation in the Gold Pairing Study . . . 60
4.7 Gold Concurrency-Sensitivity Experiment . . . 61
4.7.1 Why Task-Seconds and Elapsed Time Diverge . . . 63
4.7.2 Connecting Concurrency Limits to the Measured Timeline . . . 64
4.8 Run-Order and Session Diagnostics . . . 64
4.9 Output Consistency Across Experiments . . . 65
4.10 Research-Question Results Summary . . . 65
5 Discussion **66**
5.1 Interpreting the Azure Benchmark . . . 66
5.2 RQ1: HEFT Compared with Static Scheduling . . . 67
5.2.1 Bronze Dependency Release . . . 67
5.2.2 The Limits of Upward Rank in a Single-Cluster Admission Policy . . . 67
5.3 RQ2: XGBoost-Based Scheduling . . . 67
5.3.1 Observed Layer Differences . . . 68
5.3.2 What the XGBoost Model Has and Has Not Learned . . . 68
5.4 RQ3: Task Dependencies and Concurrent Execution . . . 69
5.4.1 Bronze Unlocking . . . 69
5.4.2 Gold Pairing and Concurrency . . . 69
5.5 RQ4: Local and Azure Results . . . 70
5.6 Identification, Uncertainty, and Reproducibility . . . 70
5.6.1 Internal and Construct Validity . . . 72
5.6.2 Output Validity and Lineage . . . 72
5.6.3 External and Conclusion Validity . . . 72
5.7 What the Observed Contrasts Can Establish . . . 73
5.7.1 Policy Labels, Realised Decisions, and Counterfactuals . . . 73
5.7.2 Uncertainty and Robustness Across Small Cloud Samples . . . 73
5.7.3 A Counterfactual Standard for Attributing Scheduling Improvements . . . 74
5.8 Priority, Concurrency, and the Deployment Decision . . . 74
5.8.1 Practical Decision Framework . . . 75
5.9 Decision-Science Interpretation and Transfer . . . 75
5.9.1 Choosing a Policy under Measurement Uncertainty . . . 76
5.9.2 Generalisation Scenarios and Testable Propositions . . . 77
5.10 Analytical Expectations and Observed Findings . . . 77
5.11 Engineering and Scientific Implications . . . 78
5.11.1 Theoretical Contributions . . . 78
5.11.2 Evidence Needed for Stronger Follow-Up Studies . . . 79
5.11.3 Relation to Prior Work . . . 79

6 Conclusion **80**
6.1 Study Overview . . . 80
6.2 Answers to the Research Questions . . . 80
6.3 Contributions and Boundaries . . . 81
6.4 Further Research . . . 82
6.5 Final Perspective . . . 82
A Detailed Experimental Configuration **83**
A.1 Observation Counts . . . 83
A.2 Local Simulated Task-Duration Inputs . . . 83
A.3 Azure Priority Weights . . . 83
A.3.1 Bronze . . . 84
A.3.2 Silver . . . 84
A.3.3 Gold . . . 85
A.4 Mechanism Configurations . . . 85
A.5 Experiment Identification . . . 85
A.6 Silver Sensor Treatment . . . 85
A.7 Software and Infrastructure . . . 86
A.8 Saved-Model Priority Verification . . . 86
A.9 Observed Opening Pairs in Every Benchmark Run . . . 87
B Robustness and Integrity Details **88**
B.1 Earlier and Repeated Benchmark Comparison . . . 88
B.2 Output-Integrity Scope . . . 88
B.3 Output Verification Method . . . 89
B.4 Cloud Robustness . . . 89
B.5 Run-Order Diagnostics . . . 89
B.6 Gold Orchestration Gaps . . . 90
Bibliography **91**

List of Tables

1.1 Scheduling decision levels and their treatment in this thesis. 3

2.1 Positioning of scheduling literature. 15

3.1 Experiment inventory. 19
3.2 Evidence used to answer each research question and the principal interpretation limit. 20
3.3 Computational task catalogue for the evaluated pipeline. 21
3.4 Feature groups used by the offline XGBoost priority scorer. 25
3.5 Offline XGBoost priority-model evaluation. 26
3.6 Local and Azure execution environments. 33
3.7 Task-level metrics retained for Azure analysis. 35
3.8 Timing sources and interpretation boundaries. 35
3.9 Principal validity risks and experimental controls. 40

4.1 Local benchmark runtime by layer and strategy. 43
4.2 Local pairwise runtime comparisons against Static. 44
4.3 Azure processing windows (five repetitions per strategy–layer cell). 46
4.4 Individual remote processing windows for the Azure experiment (seconds). 47
4.5 Azure comparisons against Static and leave-one-round-out sensitivity. 48
4.6 Observed first two remote task starts in Azure Round 2, under the configured two-slot limit. . . 50
4.7 Admission choices reconstructed from all 45 Azure benchmark runs. 51
4.8 Within-environment mean processing-window improvement over Static. 55
4.9 Bronze dependency-unlocking mechanism results. 56
4.10 Gold processing window by first-wave task pair. 58
4.11 Gold task-duration contexts. 59
4.12 Individual Gold opening-pair processing windows (seconds). 61
4.13 Gold concurrency-sensitivity results. 62
4.14 Summary of controlled mechanism effects. 63
4.15 Summary of the research-question answers. 65

5.1 Layer-aware deployment framework derived from the experiments. 75

A.1 Detailed repetition and task-observation counts. 83
A.2 Configured local simulation base durations (seconds). 84
A.3 Bronze task priorities for all five repetitions. 84
A.4 Silver task priorities for all five repetitions. 84
A.5 Gold task priorities for all five repetitions. 85
A.6 Controlled mechanism-study conditions. 85
A.7 Key software and infrastructure versions. 86
A.8 XGBoost model predictions and the corresponding Airflow task priorities. 86
A.9 All 45 observed benchmark opening pairs, by remote job-start order. 87

B.1 Earlier and repeated cloud benchmark means (seconds; five runs per cell). 88

List of Figures

3.1 End-to-end experimental architecture. 20
3.2 Bronze dependency graph. 22
3.3 Silver convergence graph. 23
3.4 Gold independent-task graph. 23
3.5 Twelve highest feature importances in the offline XGBoost priority scorer. 27

4.1 Local runtime by layer and strategy. 44
4.2 Local runtime distributions for ten repetitions per layer-by-strategy cell. 44
4.3 Local scheduling delay by layer and strategy. 45
4.4 Azure processing-window means. 48
4.5 Azure leave-one-round-out sensitivity. 49
4.6 Bronze remote task durations in the Azure Databricks benchmark. Each bar is the mean of five job-level durations; jobs can overlap within one layer. 51
4.7 Silver remote task durations by scheduling strategy, averaged over five benchmark repetitions per cell. The three policies admitted the same first-wave task pair. 52
4.8 Gold remote task durations by scheduling strategy, averaged over five benchmark repetitions per cell. Each mean describes task executions under the observed co-execution conditions. 52
4.9 Bronze repetition 2 remote task windows, each trial aligned to its own first job start. . . 53
4.10 Silver repetition 2 remote task windows, each trial aligned to its own first job start. . . . 54
4.11 Gold repetition 2 remote task windows, each trial aligned to its own first job start. 54
4.12 Within-environment mean processing-window differences relative to Static. Local and Azure absolute durations and concrete priority implementations are not directly comparable. 55
4.13 Bronze unlocking mechanism. 57
4.14 Gold opening-pair and task execution contexts. 59
4.15 Gold concurrency sensitivity under a fixed priority order. 62

List of Abbreviations

Abbreviation	Meaning
ACID	Atomicity, Consistency, Isolation, Durability
ADLS	Azure Data Lake Storage
API	Application Programming Interface
CI	Confidence Interval
CPU	Central Processing Unit
CSV	Comma-Separated Values
CV	Coefficient of Variation
DAG	Directed Acyclic Graph
DBR	Databricks Runtime
DDS	Data Analytics and Decision Science
ETL	Extract, Transform, and Load
HEFT	Heterogeneous Earliest Finish Time
LOO	Leave-One-Out
LTV	Lifetime Value
MAE	Mean Absolute Error
ML	Machine Learning
NAT	Network Address Translation
PEFT	Predict Earliest Finish Time
RMSE	Root Mean Squared Error
RQ	Research Question
SD	Standard Deviation
SF	Scale Factor
SQL	Structured Query Language
TPC-DS	TPC Decision Support benchmark
UC	Unity Catalog
VM	Virtual Machine
XGB	XGBoost

Chapter 1

Introduction

1.1 Motivation

Recurring ETL workflows supply dashboards, forecasting models, regulatory reports, and other analytical applications. Their value depends in part on when the processed data becomes available. A delay in a prerequisite transformation can postpone several downstream products even when that transformation consumes little computation itself. Consequently, pipeline completion is not determined by the runtime of the largest task alone. The order in which eligible tasks receive the available execution capacity also matters. In Apache Airflow, this decision arises when more tasks are ready than can run concurrently. The scheduler can use `priority_weight` to choose among them, subject to dependency and capacity constraints (Apache Software Foundation, 2026*a*,*b*). Three plausible ways of assigning those priorities are an informed manual order, a graph-based rank such as HEFT (Topcuoglu et al., 2002), and an offline model-derived score fitted to a designed priority formula. The latter reproduces an engineered ranking target, not scheduling choices learnt from measured makespan. Each represents different assumptions about why starting one task before another might shorten completion time. Their relative value therefore has to be examined in the workflow where they are used.

The comparison concerns a narrower decision than general-purpose cluster scheduling. Airflow determines the start order of ready notebooks; it does not control Spark task placement or the resources assigned within the Databricks cluster. This separation lets the comparison address a practical question: when two Airflow slots are available, do different priority rules select different tasks, and does that decision change the time needed to finish the layer? The recorded timestamps are examined together with the configured weights to answer that question.

The evaluated Medallion Lakehouse provides an informative setting because its Bronze, Silver, and Gold layers have different task dependencies and transformation workloads. Bronze preserves source data and lineage, Silver validates and integrates data, and Gold constructs consumer-oriented aggregates (Databricks, 2026; Armbrust et al., 2021). The layer names do not prescribe a fixed graph, but practical implementations often expose different scheduling structures. A Bronze ingestion graph can contain short dimension tasks whose completion

releases multiple fact-table loads. A Silver graph can converge into a final integration task that cannot begin until all required upstream transformations complete. A Gold graph can contain independent aggregations that become ready simultaneously and compete for shared compute, memory, storage, and shuffle resources on the same Spark cluster. An average over the complete pipeline can conceal these differences. A layer-specific evaluation makes it possible to assess whether a priority strategy suits the observed dependency structure.

The topic is also relevant to the distinction between orchestration and distributed execution. Airflow decides which ready notebook submission starts first, but Azure Databricks determines how the submitted Spark jobs behave on the cluster. Two schedules that appear equivalent in a local simulation may behave differently when the tasks share real CPU, memory, storage, and control-plane resources. Conversely, two numerically different priority tables may produce the same effective start order and offer no meaningful scheduling difference. For this reason, the analysis considers observed task starts alongside the configured priority values.

A concrete example is provided by the Bronze ingestion graph, whose initially eligible tasks include `date_dim`, `item`, and `customer`. Under the study limit of two simultaneously active tasks, not all three can begin together. Starting `date_dim` and `item` first makes the two shared prerequisites for the sales-ingestion tasks available as soon as both finish. Starting `customer` ahead of `item` instead postpones the release of these downstream tasks until `item` has also completed; `store_sales` additionally requires `customer`. Thus, a decision involving only the first two execution slots can affect the subsequent ready set and the number of execution waves. The comparison concerns the timing of dependency release, not simply which individual task has the shortest estimated duration. The graph and experimental protocol are presented together in Chapter 3.

1.2 Scheduling Decisions and Performance Objectives

Scheduling involves several distinct decisions. Workflow design determines the tasks and dependency edges. Ready-task prioritisation chooses among currently eligible tasks. Concurrency control limits how many tasks may execute simultaneously. Resource placement selects the worker or processor on which a task executes. Runtime adaptation changes decisions after observing current utilisation or completed work. These levels are related, but they require different evidence and algorithms. The present thesis focuses on ready-task priority and supplements the strategy comparison with a controlled Gold concurrency study. Workflow structure, notebook implementation, cluster configuration, and task-concurrency limits are held fixed in the strategy comparisons to reduce confounding. Effective task-start orders are examined separately to establish whether different priority configurations produce distinct scheduling decisions.

The decision levels and the scope of this study are summarised in Table 1.1.

The primary performance objective is layer makespan, measured as the processing window: the interval from the start of the first computational task to the end of the last task in the layer.

Table 1.1: Scheduling decision levels and their treatment in this thesis.

Decision level	Main question	Typical variables	Study setting
Workflow design	Which tasks and links exist?	Task groups, DAG edges, outputs	Fixed in benchmark runs
Ready-task priority	Which ready task starts next?	Priority weights, graph rank, task duration	Three policies
Concurrency	How many tasks run at once?	Task slots, pools, admission limits	2 in benchmark; Gold study 1–3
Resource placement	Where do tasks execute?	Cluster, node type, CPU and memory	Fixed Databricks cluster
Runtime adaptation	Do priorities change at runtime?	Queue length, utilisation, online estimates	Not implemented

It excludes queueing before the first task starts and does not measure end-to-end completion across all three layers. This objective differs from several alternatives. The sum of individual task durations measures aggregate task-seconds but ignores overlap. The full Airflow DAG runtime includes notebook submission, queue transitions, polling, and post-processing. A schedule can reduce makespan while increasing aggregate task-seconds. It can also reduce task-level contention yet worsen wall-clock completion when sequential hand-offs are slow. The Gold concurrency study later demonstrates this trade-off directly.

The study does not assume that the most sophisticated scheduler should win. Algorithmic complexity has an operational cost: additional features must be maintained, duration estimates can become stale, and model outputs require monitoring and explanation. The comparison should consider more than the lowest mean runtime. It should also assess whether the benefit is stable, whether the strategy changes the effective schedule, and whether a simpler rule already captures the governing mechanism.

1.3 Problem Statement

The central problem is the mismatch between task-level priority models and layer-level completion objectives. Schedulers assign scores to individual tasks, whereas the operational objective is usually to minimise the completion time of an entire layer or pipeline. The task with the longest estimated duration is not necessarily the task that should start first. A short task can be critical when it releases broad downstream fan-out. Conversely, launching several individually important tasks together can slow all of them through resource interference. A task-by-task ranking may therefore produce an unfavourable execution order even when the individual duration estimates are accurate.

Existing ETL optimisation research has examined logical workflow transformation, physical implementation, and precedence-constrained scheduling (Simitsis et al., 2005; Karagiannis et al., 2013). Cluster-scheduling research has examined multi-resource packing, heterogeneity,

fairness, interference, and learned scheduling (Ghodsi et al., 2011; Grandl et al., 2014; Delimitrou and Kozyrakis, 2013; Mao et al., 2019). Although these streams establish relevant scheduling concepts, the comparative behaviour of Static, HEFT-based, and XGBoost-based priority policies within the particular Airflow–Medallion configuration evaluated here remains an empirical question. The study examines how these policies behave across the three layers under local and cloud execution.

The choice of a Static baseline is important to the interpretation of the Azure comparison. Manually selected priorities can encode knowledge about prerequisite tasks and downstream dependencies; they need not represent a naive first-in-first-out schedule. In the Azure Bronze graph, the Static weights deliberately favour the roots that release sales ingestions. The local Static implementation, by contrast, used Airflow's default task weights rather than this manual table. Comparing an automated rank with the informed Azure order asks a stricter question than comparing it with an arbitrary queue: does the extra scheduling logic improve on a transparent policy that already reflects the workflow? The same logic applies to negative results. Where a dependency barrier or the capacity limit produces the same effective schedule across strategies, a more elaborate priority calculation does not necessarily create a new scheduling opportunity. The comparison therefore tests the practical consequences of assigning different priorities, not the sophistication of the algorithms in isolation.

A second problem is experimental transferability. A local benchmark permits repeated and tightly controlled execution, but cannot reproduce all Spark effects. Cloud execution provides greater realism, but exhibits variation even when the nominal configuration is unchanged (Maricq et al., 2018; Uta et al., 2020; Leitner and Cito, 2016). A credible comparison requires both environments and should avoid treating one cloud mean as a definitive ranking. The analysis therefore reports variability and sensitivity alongside mean processing times and observed execution order.

Interpreting the comparison requires distinguishing three types of evidence. A configured priority table shows the intended treatment, but priority values by themselves do not establish which tasks actually ran first. Task-start timestamps reveal the realised execution waves and whether a policy changed an observable scheduling decision. Finally, the processing window measures the resulting layer-level completion time, which can also vary because individual task durations and remote execution conditions differ. In particular, if two strategies induce the same effective order, a difference between their observed mean processing windows cannot be attributed to a different priority-selected task sequence. Where the realised orders do differ, the observed timing difference must still be interpreted within the limits of the experimental design. This distinction links the research questions to the recorded evidence and motivates reporting execution timelines alongside aggregate performance statistics.

1.4 Research Aim and Questions

This study examines how task-priority strategies relate to layer-level completion time in a Medallion Lakehouse ETL pipeline and which scheduling decisions help explain the observed differences. The study evaluates four research questions:

RQ1: How does HEFT-based scheduling compare with static scheduling in task execution order and processing time across the Bronze, Silver, and Gold layers?

RQ2: How does XGBoost-based scheduling compare with static and HEFT-based scheduling in task execution order and processing time across these layers?

RQ3: How do task dependencies and the choice of concurrently executed tasks affect layer completion time?

RQ4: To what extent do the scheduling results observed locally also occur on Azure Databricks, and how does task concurrency affect processing time?

The first two questions compare scheduling strategies; the third examines task-dependency and concurrency mechanisms explored in separate controlled studies; and the fourth considers cross-environment patterns and Gold concurrency sensitivity. The objective is not to identify one universal winner.

1.5 Analytical Expectations

The research questions motivate four expectations. First, HEFT-based ranking may be effective when the longest remaining dependency path determines completion time. However, its upward-rank calculation does not explicitly reward a short prerequisite for releasing several tasks at once. Second, the XGBoost model may reproduce the task-ordering preferences encoded in its engineered training target. Because the target and input features do not describe which tasks execute together, the model was not trained to predict pair-specific slowdowns. Third, Silver should show little change in scheduling decisions when all three strategies select the same ready tasks. Fourth, Gold processing time may depend on both the opening task pair and the number of available execution slots.

These expectations guide interpretation rather than function as pre-registered directional hypotheses. The controlled studies were designed to investigate patterns observed in the strategy comparisons. They are therefore reported as exploratory experiments rather than pooled with either benchmark.

1.6 Scope and Delimitations

The scope is limited to task-level priority scheduling in a three-layer ETL workflow orchestrated with Apache Airflow. The study compares three strategy families:

- fixed Static baselines, implemented with Airflow defaults locally and manual weights in Azure;
- a HEFT-based upward-rank heuristic adapted to Airflow priorities; and
- an XGBoost regression model whose predictions were calculated before the cloud runs and used as fixed task priorities. The model was trained on a priority score defined from workflow characteristics rather than observed optimal schedules.

The implementation is evaluated in two environments. The local benchmark uses Python tasks with controlled stochastic durations and PostgreSQL metric logging. The cloud benchmark uses Azure Databricks notebooks, Spark, Delta tables, and a fixed single-node cluster. The selected cloud dataset comprises six tables from the built-in TPC-DS SF1 sample, totalling approximately 5.23 million records and 273.93 MB of reported compressed source data. The study does not claim compliance with the complete TPC-DS benchmark and does not claim that the subset represents a large enterprise-scale workload.

The Azure strategy comparison holds the nominal cluster, selected input data, layer-specific notebook transformations, DAG dependencies, metric definitions, and concurrency fixed; individual output tables are isolated by run. The cloud experiments were not fully randomised or carried out in matched blocks, and variation in cloud state remains a limitation. The separate Bronze and Gold experiments each vary one defined scheduling factor, while retaining the same logical workload within that study. The cloud findings are bounded by one region, one runtime, one VM type, one selected dataset, and five cloud repetitions per scheduling strategy and layer. The resulting Delta tables were checked for availability and consistency, although this does not establish independent row-by-row correctness. The local results model orchestration and graph behaviour rather than Spark internals; the local and Azure benchmarks also use different concrete priority configurations. The XGBoost model approximates a deterministic formula based on quantities included among its features and does not adapt online. The HEFT adaptation does not implement the full processor-assignment algorithm.

The scope also excludes several adjacent questions. It does not redesign the ETL transformations, optimise Spark physical plans, tune autoscaling, compare cloud providers, or measure end-user query latency. These delimitations keep the empirical analysis focused on task priority, dependency release, co-execution context, and permitted concurrency.

1.7 Contributions

The study makes five contributions.

1. It implements and evaluates an Airflow framework for comparing Static, HEFT-based and XGBoost-based task priorities across the three Medallion layers, with repeated runs, logged execution metrics and output checks.

2. It combines a local Airflow benchmark with Azure Databricks execution under Spark, Delta Lake, and Unity Catalog. Comparing realised task-start orders reveals when different priority values lead to the same effective schedule and permits a mechanism-focused comparison across environments.

3. It experimentally examines dependency-release timing in Bronze. Under the evaluated conditions, delaying the `item` task increases the observed layer processing window despite similar aggregate task durations.

4. It characterises task-duration differences across controlled Gold opening pairs and between overlapping and non-overlapping execution waves, providing evidence consistent with context-dependent resource interference.

5. It documents the Gold concurrency trade-off: increasing permitted concurrency raises aggregate task-seconds but reduces the observed layer processing window. This evidence informs a layer-aware decision framework rather than a claim of universal scheduler optimality.

1.8 Thesis Structure

Chapter 2 reviews ETL, DAG scheduling, machine-learning approaches, and resource contention. Chapter 3 presents the system, scheduling implementations, and experimental protocols. Chapter 4 reports the local and Azure benchmarks and exploratory mechanism studies. Chapter 5 interprets these findings, examines limitations, and develops a layer-aware decision framework. Chapter 6 answers the research questions and outlines future work.

Chapter 2

Related Work and Conceptual Background

2.1 ETL Workflows and Scheduling Decisions

ETL research considers more than the speed of individual transformations. In the logical-optimisation setting of Simitsis et al. (2005), an ETL flow can be represented as alternative, semantically equivalent configurations and searched for a lower-cost arrangement. That is a different intervention from the one examined here: the task graph is retained, but competing ready tasks receive different execution priorities. The distinction matters because improving the transformation plan and changing start order can affect completion time through different mechanisms.

In the context of ETL scheduling, Karagiannis et al. (2013) compare execution policies in terms of execution time and allocated memory. Their work establishes the relevance of choosing when ETL operations execute, but the policies and execution setting differ from the present Airflow-to-Databricks experiment. Here the choices are between already eligible notebook tasks, with the Spark cluster and transformation logic fixed within the strategy comparisons. The question is whether an alternative priority rule changes the observed task waves enough to affect the processing window.

A workflow DAG contains nodes representing extraction, transformation, or loading tasks and directed edges representing precedence constraints. At time t, the ready set comprises unfinished tasks whose predecessors have completed. When ready tasks exceed available capacity, the scheduler selects which may start. With overlap and capacity constraints, makespan differs from the sum of task durations. It depends jointly on precedence, ready-task order, slot availability, orchestration overhead, and context-dependent duration.

The performance measure should reflect the operational decision. A schedule with extensive overlap may finish the layer earlier even if individual tasks become slower while sharing a cluster. Conversely, a mostly sequential order may shorten individual task executions yet leave a longer wall-clock window. The experiments therefore use the elapsed interval between the first task start and last task completion as the primary outcome. The sum of task durations and the variation between repetitions are reported separately; neither the task-duration sum nor the processing

window is a direct measurement of CPU consumption.

Scheduling approaches can be classified along several dimensions. A static priority policy fixes task rankings before execution, while the realised task-start order still depends on precedence and available slots. Such policies are easy to inspect and reproduce, but may become less suitable as workloads change. Adaptive heuristics revise decisions using current execution state or updated cost estimates. Learned policies derive priority functions or actions from data; a trained policy can remain fixed during execution, as in this study, or be updated online. A separate dimension concerns whether the scheduler controls only order, also controls concurrency, or performs resource placement across heterogeneous workers.

List scheduling is a common abstraction for precedence-constrained tasks. Each task receives a priority, and the scheduler repeatedly assigns the highest-priority ready task to available capacity. HEFT is a prominent list-scheduling method that combines an upward rank with a processor-selection phase (Topcuoglu et al., 2002). In the evaluated DAGs, Airflow's integer `priority_weight` with `weight_rule="absolute"` ranks eligible tasks (Apache Software Foundation, 2026*a*), but does not perform HEFT's processor-assignment phase.

Resource placement operates at a lower level. Mesos offers fine-grained resource sharing across distributed frameworks (Hindman et al., 2011), and YARN separates cluster resource management from application-specific execution (Vavilapalli et al., 2013). Borg integrates priorities, resource requests, admission control, and rescheduling in a production cluster manager (Verma et al., 2015). Unlike these placement systems, the evaluated Airflow policies order notebook submissions to a fixed cluster.

Consider the two available task slots in the evaluated Bronze DAG. Initially, `date_dim`, `item`, and `customer` compete for admission. Choosing `item` before `customer` can release additional sales ingestions once `date_dim` has completed; the reverse choice may postpone those tasks even if `customer` itself finishes quickly. Thus, a priority decision changes not only the identity of the next task but also the number and timing of later eligible tasks. Its value cannot be judged reliably from that task's estimated runtime alone.

2.2 Medallion Lakehouse and Execution Levels

Lakehouse systems seek to retain the open and low-cost storage characteristics of data lakes while adding transactional controls and analytical capabilities associated with warehouse platforms (Armbrust et al., 2021). Delta Lake adds a transaction-log layer over object storage to support ACID-style table updates, schema controls, versioned access, and scalable metadata operations (Armbrust et al., 2020). The Medallion pattern organises data into Bronze, Silver, and Gold layers with progressively higher quality and business meaning (Databricks, 2026).

The layer labels are architectural conventions rather than fixed graph templates. Nevertheless, they imply recurring classes of transformation. Bronze often performs append or overwrite ingestion with lineage metadata. Silver commonly standardises dimensions, validates facts,

removes duplicates, and joins datasets. Gold creates grouped, ranked, or segmented analytical products. The chosen experimental graphs exhibit different scheduling opportunities: Bronze has dependency-release effects, Silver contains a convergence barrier, and Gold begins with independent aggregation tasks. These properties arise from the selected DAGs rather than from universal rules of the Medallion architecture.

Distributed execution further complicates the relation between logical tasks and observed duration. Spark's resilient distributed datasets and DAG execution model provide fault-tolerant parallel processing (Zaharia et al., 2012). Spark SQL combines relational optimisation with distributed execution and can generate scans, joins, aggregates, shuffles, and materialisation stages whose costs depend on the data and available resources (Armbrust et al., 2015). An Airflow task that submits one notebook is a coarse-grained orchestration unit containing many Spark stages. When several notebooks share a cluster, their durations can change because their internal stages compete.

2.3 HEFT-Based Priority Heuristics

Topcuoglu et al. (2002) propose HEFT for scheduling a precedence-constrained application across heterogeneous processors. The algorithm ranks tasks by estimated computation and successor-path costs, then selects a processor using an earliest-finish-time calculation. These two stages should be distinguished.

An alternative heterogeneous list scheduler is Predict Earliest Finish Time (PEFT), which uses an optimistic cost table and lookahead in ranking and processor selection (Arabnejad and Barbosa, 2014). Neither the full HEFT algorithm nor PEFT is evaluated as a heterogeneous placement algorithm here; the intervention is an Airflow-ready-task rank. This thesis borrows the upward-rank calculation to order eligible Airflow tasks, but does not implement heterogeneous processor assignment. The use of the term *HEFT-based* is deliberate: the experiment evaluates a rank-derived priority rule, not the full algorithm or its performance guarantees.

Critical-path information is valuable when a long chain determines the completion time. By itself, it does not fully describe the potential parallelism released at a fan-out point. A short prerequisite can release several independent successors, but upward rank reflects the longest successor path rather than the number of tasks made ready. Upward rank also usually treats task cost as a property of the task-processor combination. It does not automatically represent pair-specific slowdown when two independent tasks share one cluster. The Bronze unlocking and Gold pairing experiments examine these potential limitations in the selected DAGs.

2.4 Offline Machine Learning and Performance Prediction

XGBoost provides a practical way to approximate a nonlinear scoring rule from structured features (Chen and Guestrin, 2016). In this study, the offline regressor receives task identity,

layer, estimated duration, graph descriptors, and historical summaries, and is fitted to a priority target constructed from task metadata. The training labels are not observed optimal decisions or measured makespan rewards. During the benchmark, Airflow reads fixed priorities instead of invoking the saved model. The deployed priorities are rounded from the saved model's predictions and preserve its within-layer ordering for the evaluated tasks.

Learned scheduling has broader precedents. Decima applies reinforcement learning and graph-neural representations to learn scheduling policies for data-processing clusters, optimising job-completion objectives from system state and experience (Mao et al., 2019). Its setting differs substantially from the present XGBoost scorer: Decima uses a state-dependent learned scheduling policy with a completion-time objective, whereas the model used here predicts an engineered task-level priority target. The comparison is conceptually useful because it identifies what is absent from the simpler design: direct feedback from realised makespan, set-level task interaction, and online state.

Performance prediction and configuration selection also provide related evidence. Ernest fits analytical performance models to predict the runtime of analytics jobs under different resource allocations (Venkataraman et al., 2016). CherryPick uses Bayesian optimisation to search cloud configurations while balancing performance and cost (Alipourfard et al., 2017). Both show that empirical models can reduce the search burden of systems tuning. They also show that prediction quality depends on the domain represented by the training observations. A model trained on isolated or simulated tasks cannot be assumed to predict co-execution slowdown on a real Spark cluster.

The analysis separates model-fit evaluation from scheduler evaluation. A low error against the engineered priority target only demonstrates that XGBoost reproduces that target. The scheduling outcome of interest is the processing window produced by the resulting task order. The feature and target definitions do not explicitly encode the identities of tasks running together or the resulting shared-cluster slowdowns. This is a plausible limitation of the model representation, not an independently identified cause of every observed runtime difference. Repeated observations of the same task identities also constrain what the original train/test evaluation establishes about unseen tasks.

Evaluation design matters for the same reason. If training and test rows share task identities, a row-wise holdout assesses prediction for familiar patterns more directly than transfer to unseen tasks or workflow graphs. A stronger generalisation study would hold out complete task identities, DAGs, or workload regimes, and would evaluate whether model-induced changes in ordering improve measured layer outcomes. The current study does not claim to perform such a generalisation experiment; it distinguishes the offline model fit from the scheduling evidence gathered on the evaluated tasks.

Recent related work considers different decisions and objectives. Gao et al. (2025) investigate adaptive deep Q-learning for ETL task scheduling under heterogeneous conditions; their feedback-based policy is not equivalent to the fixed XGBoost priorities evaluated here. Ma et al. (2025)

describe a Databricks orchestration framework combining runtime prediction with optimisation, which differs from the layer-processing-window measure used here.

2.5 Resource Sharing, Interference, and Scheduling Overhead

Cluster schedulers allocate finite CPU, memory, storage, and network capacity among jobs. Dominant Resource Fairness allocates several resource types by equalising users' dominant shares rather than balancing each resource independently (Ghodsi et al., 2011). Tetris instead focuses on packing tasks with complementary resource profiles to reduce fragmentation and improve job completion time (Grandl et al., 2014). These approaches highlight two different objectives: equitable sharing and efficient resource composition.

Sparrow uses decentralised sampling and late binding to achieve low scheduling latency without relying on one global placement bottleneck (Ousterhout et al., 2013). Firmament shows that a centralised min-cost-flow formulation can support rapid placement decisions at cluster scale (Gog et al., 2016). Their emphasis on scheduling latency is relevant to the present study even though Airflow operates at a coarser granularity. Submission and state-transition gaps can contribute to layer completion time, especially for short notebook tasks, but the recorded timestamps do not separately identify every control-plane component.

2.5.1 Shared-Resource Interference

Co-executing workloads can slow one another when they share resources. Paragon characterises applications by their sensitivity to hardware heterogeneity and interference, then selects placements intended to preserve performance (Delimitrou and Kozyrakis, 2013). Quasar extends this direction by jointly classifying workloads and allocating resources to satisfy performance constraints efficiently (Delimitrou and Kozyrakis, 2014). Their core insight is that performance is conditional on the execution context.

Interference can arise through several mechanisms. Tasks may compete for CPU cycles, cache, memory capacity, memory bandwidth, disk throughput, network bandwidth, or shared service queues. Spark aggregations can additionally compete for shuffle and spill resources. The relevant bottleneck can change over time as each notebook progresses through stages. A single isolated duration is an incomplete descriptor of the cost of starting a task.

The Gold pairing experiment varies the opening pair when two task slots are available. It compares tasks that overlap in the first wave with the remaining task executing without overlap in the second wave of the same run. Second-wave duration can also depend on execution position, caching, and cluster state; it is not a separately isolated baseline. A second experiment varies permitted concurrency while holding the configured priority rule fixed. Together they examine the timing consequences of task pairing and overlap.

There is also an identification problem when interpreting timings from shared infrastructure. If two strategies generate the same realised opening pair, a difference between their mean

completion times does not by itself demonstrate an ordering advantage; the treatment has not changed that particular task-selection decision. By contrast, deliberately forcing different opening pairs changes the observed scheduling condition and provides more direct evidence about that choice, although the remaining variability and task execution context still matter. Similarly, comparing a task's first-wave and second-wave duration within a run cannot be interpreted as a controlled comparison between concurrent and independently isolated execution. Execution position, caching, and preceding work also differ. The strongest conclusions are therefore about the observable contrast actually induced, not about a hardware contention component that was never measured independently.

2.5.2 Scheduling and Control-Plane Overhead

The scheduler itself contributes latency. A task can wait after a slot becomes free because state changes must be persisted, the next task must be queued, a remote run must be submitted, and the operator must poll for completion. Fine-grained distributed schedulers such as Sparrow and Firmament are partly motivated by the need to keep such decision and placement latency small (Ousterhout et al., 2013; Gog et al., 2016).

In a local sleep-based benchmark, the hand-off can be much smaller than the task duration and is comparatively stable. In Airflow-to-Databricks execution, every notebook is a remote job submission. When concurrency is one, two transitions occur between the three Gold tasks. When concurrency is two, one second-wave submission remains. When concurrency is three, all jobs are admitted initially and no later wave is required. The observed difference combines changes in overlap, context-dependent notebook durations, and between-wave transition intervals. It does not separately estimate the contribution of Airflow submission, polling, and remote admission overhead.

2.6 Scheduling Objectives, Policy Evaluation, and Observability

Makespan is a common scheduling objective, but production systems rarely optimise latency in isolation. A policy may also seek high throughput, stable completion time, resource efficiency and fairness. Dominant Resource Fairness formalises equitable allocation across multiple resource dimensions (Ghodsi et al., 2011), while Tetris emphasises efficient packing and job completion (Grandl et al., 2014). These objectives can favour different schedules even for the same workload.

A scheduling study should state the objective and supporting metrics explicitly. The present thesis treats the task processing window as the primary outcome because the research questions concern layer completion. Standard deviation and coefficient of variation describe run-to-run variability. Aggregate task-seconds sum observed task durations and may indicate contention-related slowdowns, but do not directly measure resource consumption. Reporting these measures reveals trade-offs without claiming to identify a formal Pareto frontier.

Predicting the completion time of concurrently executed tasks requires information about their joint execution conditions, not only estimates of their individual durations.

2.6.1 Realised Decisions and Interpretation

Changing configured priorities can affect processing time through scheduling only if the change alters which eligible tasks are admitted. Two algorithms can assign different numerical scores and still produce the same top-k ready set. If two policies yield the same effective ready-task selection and task-wave structure, their ordering decisions are equivalent for that DAG and capacity setting. Any observed runtime difference cannot then be attributed to a different opening pair alone and may reflect variation in execution conditions.

Controlled scheduling experiments provide more direct evidence about a specific mechanism when they vary one scheduling factor while keeping notebook code and inputs fixed. The Bronze experiment changes one root-order relationship. The Gold pair experiment changes the companion set. The concurrency experiment changes admission capacity. These studies do not eliminate all cloud variation, but they provide a closer link between intervention and mechanism than a post-hoc comparison of strategy means.

Local and cloud measurements must also be compared with their different measurement conditions in mind. The local benchmark compares priorities under simulated task durations, whereas Azure compares observed processing windows under shared-cluster execution. Changes in task order and execution context may both affect outcomes, so the two environments provide related but non-identical evidence. Cross-environment comparisons are therefore interpreted primarily at the level of scheduling mechanisms.

2.6.2 Observability and Interpretation

Priority methods also differ in interpretability. Static rankings can be explained through domain assumptions, and HEFT ranks can be traced to task costs and successor paths. XGBoost feature importance describes model-level associations with predicted priority, but does not demonstrate that a feature caused shorter execution. Versioning the trained model, feature schema and applied priority table is important for interpreting an offline scoring policy consistently across executions.

2.7 Benchmarking, Statistical Evidence, and Workload Scope

Performance evaluation in computer systems requires careful metric selection, repetition, and transparent treatment of variability. Jain (1991) emphasises that a benchmark should isolate factors, use appropriate experimental designs, and report conclusions that match the evidence. Public and virtualised infrastructure introduces additional uncertainty. Maricq et al. (2018) document hardware performance variation across nominally similar servers and repeated runs. Uta et al. (2020) show that cloud variability can undermine the reproducibility of big-data experiments.

Leitner and Cito (2016) characterise patterns of performance variation and predictability in infrastructure clouds.

The local benchmark contains ten repetitions per layer-strategy combination. Corresponding repetition indices do not constitute matched observations, so comparisons between strategies use independent-sample Mann–Whitney U tests and probability-of-superiority effects. The six primary comparisons against Static are assessed with Holm adjustment for multiple testing. The cloud benchmark contains five repetitions per layer-strategy cell and is evaluated descriptively. Percentile bootstrap intervals expose uncertainty without being interpreted as a substitute for more independent runs (Efron, 1979). Leave-one-round-out analysis reveals whether the direction of a comparison depends on one round. Run-order correlations are used only as diagnostics because small groups do not support strong trend inference.

TPC-DS is an industry decision-support benchmark containing a schema, data generator, and query templates across scale factors (Transaction Processing Performance Council, 2026). It is designed to evaluate complete systems under a defined benchmark procedure. This study uses six tables from Databricks' built-in TPC-DS SF1 sample to instantiate a reproducible decision-support-style schema. It does not run the full TPC-DS benchmark procedure, and the selected subset does not establish production-scale representativeness.

2.8 Research Synthesis and Positioning

Table 2.1 positions the study against representative literature. ETL research establishes the importance of workflow structure, HEFT provides a graph-criticality heuristic, learned scheduling demonstrates richer policy representations, and cluster systems show that compatibility and scheduling latency matter. The present study examines these ideas at the Airflow task-admission level of a Medallion pipeline.

Table 2.1: Positioning of representative scheduling and data-platform literature.

Work	Setting and decision	Objective and scope	Relevance to this thesis
Simitsis et al. (2005)	ETL workflow optimisation through logical workflow transformation	Reduces ETL execution cost; no explicit co-execution model	Establishes that ETL graph structure is an optimisation variable.
Karagiannis et al. (2013)	ETL workflows with precedence-constrained scheduling	Investigates execution time and allocated memory for ETL schedules	Provides the closest ETL-specific scheduling foundation.
Topcuoglu et al. (2002)	Heterogeneous DAG computing using rank and processor assignment	Heuristically targets short makespan using task and communication costs; no explicit co-execution model	Supplies the upward-rank heuristic adapted to Airflow.

Continued on next page

Table 2.1 continued

Work	Setting and decision	Objective and scope	Relevance to this thesis
Grandl et al. (2014)	Cluster resource packing through multi-resource task placement	Improves completion time and utilisation while representing resource demand	Motivates set-level resource compatibility.
Delimitrou and Kozyrakis (2013)	Datacentre placement through application-to-machine assignment	Optimises performance and utilisation with explicit interference awareness	Motivates examination of shared-resource effects in Gold.
Mao et al. (2019)	Data-processing clusters using a learned online scheduling policy	Reduces job completion time by learning from system state	Demonstrates the potential and data requirements of learned scheduling.
Alipourfard et al. (2017)	Cloud configuration search through empirical configuration selection	Uses Bayesian optimisation to search cloud configurations; co-scheduling interference is not the focus	Motivates empirical rather than purely analytical cloud tuning.
Venkataraman et al. (2016)	Analytics performance prediction through resource and runtime modelling	Predicts scale-out performance; co-scheduling is not the primary focus	Illustrates model-based execution-time prediction.
Armbrust et al. (2021)	Lakehouse data-platform architecture	Provides a unified analytics platform rather than a scheduling model	Defines the architectural environment for the evaluated layers.
This thesis	Airflow and Databricks Medallion ETL using ready-task priorities and controlled concurrency	Compares observed processing windows and Gold task-duration changes across pairing and concurrency conditions	Relates ready-task priorities, DAG structure, co-execution timing, and local/cloud outcomes.

A comparison of the decision variables further clarifies this scope. Simitsis et al. (2005) consider transformations of the ETL workflow, while Topcuoglu et al. (2002) combine priority ranking with heterogeneous processor assignment. Tetris and Paragon address resource placement and task compatibility (Grandl et al., 2014; Delimitrou and Kozyrakis, 2013); CherryPick tunes cloud configurations rather than selecting ready tasks (Alipourfard et al., 2017). In contrast, the strategy comparisons in this thesis keep notebook logic, cluster configuration, and Airflow's capacity limit fixed, varying the priority policy. The separate Gold concurrency study changes the capacity limit explicitly. This narrower decision space enables a focused interpretation of Airflow-level scheduling, but excludes resource-aware placement, autoscaling, dynamic model updates, and topology-changing workflow optimisation. Accordingly, speedup figures from the cited systems are not treated as directly comparable numerical baselines, because their workloads and controllable decisions differ.

Chapter 3

Methodology

3.1 Research Design

The study comprises two scheduling-strategy benchmarks and three controlled mechanism studies. In each benchmark, Static, HEFT-based, and XGBoost-derived priority configurations are evaluated separately for Bronze, Silver, and Gold. The task graph, transformation code, concurrency limit, and metric definitions are fixed within that comparison. The local benchmark uses simulated Python task durations, whereas the Azure benchmark runs the actual Spark and Delta notebooks on one shared cluster. The two environments share analogous layer graphs and scheduling strategy families, but the local Static baseline and some HEFT priorities differ from their Azure counterparts. Their results therefore describe related, not identical, task-selection treatments and are not interchangeable measurements of task cost.

The additional studies were designed after examining patterns in the strategy comparisons. They deliberately change the early Bronze root order, the first Gold pair, or Gold's maximum active-task limit. Their purpose is to test specific explanations suggested by the strategy comparisons. They are reported separately so that exploratory mechanism evidence is not mistaken for new repetitions of the original treatment.

The design combines complementary forms of evidence rather than treating them as interchangeable replications. The local phase examines interactions between precedence structure, task priority, and a constrained ready-task queue. The Azure phase asks whether comparable scheduling patterns occur during Spark and Delta transformations, without assuming that local and cloud runtimes have the same physical meaning. The Bronze manipulation varies a specific prerequisite-release decision; the Gold experiments vary first-wave pairing and the maximum number of admitted tasks. Gold task-duration changes provide evidence consistent with co-execution effects but do not independently measure CPU or memory contention.

The configured scheduling policy must be distinguished from the schedule actually executed. The *configured policy* comprises task priorities and the applicable admission limit. The *effective schedule* is reconstructed from observed task starts and overlaps. The *outcome* is the recorded layer processing window. A different admission choice can help explain a runtime contrast

only when the configured priorities change which eligible tasks run or overlap. Even then, the non-randomised cloud design limits causal attribution. When policies produce the same effective schedule, observed timing differences cannot be explained by a change in task selection. This distinction guides the strategy comparisons and the targeted follow-up studies.

Table 3.1: Experiment inventory.

Experimental phase	Runs	Task observations
Local strategy benchmark	90	390
Azure strategy benchmark	45	195
Bronze dependency-unlocking study	10	60
Gold opening-pair study	15	45
Gold concurrency-sensitivity study	15	45
Total	**175**	**735**

Note: The strategy benchmark and the exploratory mechanism studies are analysed separately.

Linking the research questions to the evidence

The four research questions require different kinds of comparison. For RQ1 and RQ2, the unit of observation is a completed layer run, and the principal outcome is its processing window. Priority tables show the intended ordering rule; task-start records show whether the rule changed the first available tasks or later execution waves. Both are needed to interpret the runtime summaries. The comparison is not an estimate of the performance of HEFT with heterogeneous processor assignment or of an online learning scheduler, since neither is implemented.

RQ3 uses the separate controlled experiments to investigate particular scheduling decisions. The Bronze study changes which prerequisite is admitted first; the Gold pairing study changes the initial pair under two slots; and the Gold concurrency study changes how many tasks can be active. These comparisons are more specific than a three-strategy benchmark. They test the consequences of an admissible choice under the measured conditions without treating its effect as a universal property of the corresponding Medallion layer. RQ4 compares patterns across execution environments and interprets the Gold concurrency evidence; it does not add local seconds to cloud seconds or infer an unmeasured end-to-end pipeline runtime.

The experiments also differ in how much evidence they provide about a proposed cause. Matching priority values to observed starts establishes that the intended configuration was used, but does not guarantee that those priorities changed admission. A contrast with a changed admission decision is more informative about ordering, although it remains sensitive to other differences between remote runs. The controlled mechanism studies narrow the decision being changed, but still contain session variation. The evidence map is therefore used to keep the conclusions aligned with the question each experiment can answer, rather than to assign all numerical differences to a scheduling algorithm.

Table 3.2: Evidence used to answer each research question and the principal interpretation limit.

	Main evidence	What the comparison cannot establish
RQ1	Static–HEFT layer windows and observed task starts	Performance of the complete heterogeneous HEFT algorithm or causal effects where task selection is unchanged
RQ2	XGBoost–Static–HEFT layer windows, observed starts and saved-model priorities	Online adaptation, unseen-workflow generalisation or learned scheduling optimality
RQ3	Controlled Bronze root, Gold opening-pair and Gold slot-limit contrasts	A separately measured hardware-interference cause of duration differences
RQ4	Within-environment comparisons and Gold concurrency sensitivity	Equality of local and cloud task durations; a measured full-pipeline makespan; transfer of an identical priority treatment; or a causal effect under differing cloud conditions

3.2 Experimental Architecture

Figure 3.1 shows the end-to-end architecture. Airflow runs locally in Docker and stores orchestration metadata in PostgreSQL. Strategy-specific DAGs define task execution policies; the local Static baseline uses Airflow defaults, while the other evaluated configurations specify strategy-dependent weights. Local DAGs execute Python functions with controlled stochastic durations. Azure DAGs use the Databricks Airflow provider to submit parameterised notebooks to a fixed cluster. Task-level metrics are retained in a Unity Catalog Delta table, with supplementary entries for the first nine Bronze runs reconstructed from archived records. The recorded results are analysed with Python to generate statistics, tables, and figures.

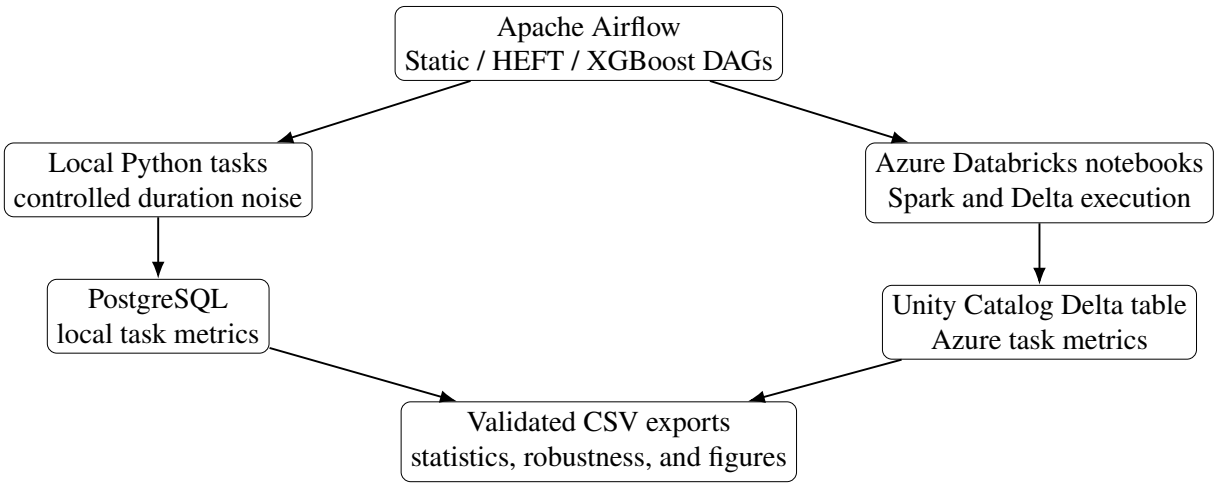


Figure 3.1: End-to-end experimental architecture.

The separation between orchestration and computation determines how processing time is measured. Airflow admits ready tasks according to dependencies and priority, while each admitted Azure task submits a Databricks notebook that runs Spark transformations. The

provider must also monitor the remote execution and record its terminal state. For the main Azure benchmark, the layer processing window is therefore computed from the first recorded remote Databricks job start to the last remote job end, rather than from the full Airflow DAG duration. The latter includes additional orchestration periods that the primary metric was not designed to measure.

The benchmark treatment was checked against logged Airflow priorities and Databricks job identifiers and timestamps. A few XGBoost constants in historical implementation files differ slightly from the applied weights without changing the within-layer ranking. The limits of authenticating the executed source revision are specified in Section 3.15.

3.3 Pipeline Definition and Task Catalogue

The computational workload is a three-layer Medallion pipeline built from six selected TPC-DS SF1 source tables. Table 3.3 summarises every benchmark task, its transformation role, and its scheduling relevance. The catalogue distinguishes logical data work from orchestration-only nodes such as the local Silver sensor.

Table 3.3: Computational task catalogue for the evaluated pipeline.

Layer	Task	Transformation role	Scheduling relevance
Bronze	`ingest_date_dim`	Copies the date dimension to a Delta Bronze table with lineage columns	Initially ready; unlocks all three sales ingestions
Bronze	`ingest_item`	Copies the item dimension to Bronze	Initially ready; unlocks all three sales ingestions and is the controlled high-unlock task
Bronze	`ingest_customer`	Copies the customer dimension to Bronze	Initially ready; additionally required by store sales
Bronze	`ingest_web_sales`	Copies the web-sales fact table	Released after date and item complete
Bronze	`ingest_store_sales`	Copies the store-sales fact table	Released after date, item, and customer complete
Bronze	`ingest_catalog_sales`	Copies the catalog-sales fact table	Released after date and item complete
Silver	`clean_sales`	Unions sales channels, casts measures, filters invalid keys, and derives margins	Initially ready and usually the longest cleaning task
Silver	`clean_customer`	Standardises names, countries, flags, and customer keys	Initially ready; converges into the fact join
Silver	`clean_item`	Standardises item attributes and removes duplicate keys	Initially ready; converges into the fact join
Silver	`join_fact_sales`	Joins cleaned sales, customer, item, and date attributes	Final convergence task; begins only after all cleaning tasks

Table 3.3 continued

Layer	Task	Transformation role	Scheduling relevance
Gold	`agg_customer_ltv`	Computes customer-level revenue, margin, activity, and value segments	Independent; expensive and sensitive to co-execution
Gold	`agg_sales_by_region`	Aggregates channel and monthly measures by customer region	Independent; strongest measured concurrency slowdown
Gold	`agg_top_items`	Aggregates item performance and ranks the top 1,000 items	Independent; durations vary by observed execution context

3.3.1 Bronze Layer

The Bronze layer ingests three dimensions (`date_dim`, `item`, and `customer`) and three sales tables (`web_sales`, `store_sales`, and `catalog_sales`) as Delta outputs. Each copy adds the run ID, strategy, priority weight, and ingestion timestamp as lineage fields. The notebook's narrower transformation timer includes the Delta write but excludes post-write row-count validation. The main Azure benchmark instead uses remote job intervals, which may also include validation and other notebook work.

All three dimension tasks are initially eligible. The sales tasks are released after their required dimensions complete. `date_dim` and `item` unlock all three sales ingestions, while `customer` is additionally required by `store_sales`. The two shared prerequisites have several downstream dependants.

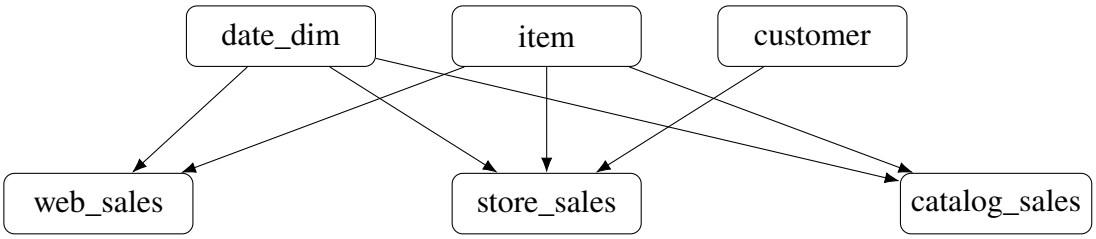


Figure 3.2: Bronze dependency graph.

3.3.2 Silver Layer

In Silver, three cleaning tasks precede the final integration task. `clean_sales` unions the store, web, and catalog channels into a common fact schema, validates required keys and positive quantities, removes duplicates, and derives gross margin and unit revenue. `clean_customer` standardises names, country, preferred-customer flags, and email addresses. `clean_item` standardises item attributes and removes duplicate keys. The final `join_fact_sales` task enriches the cleaned sales records with customer, item, and date attributes and derives gross-margin ratio and customer age at sale.

The local Airflow implementation also contains `wait_for_bronze`, a cross-layer sensor that verifies the availability of upstream data. It is not a Silver computation and is excluded from task

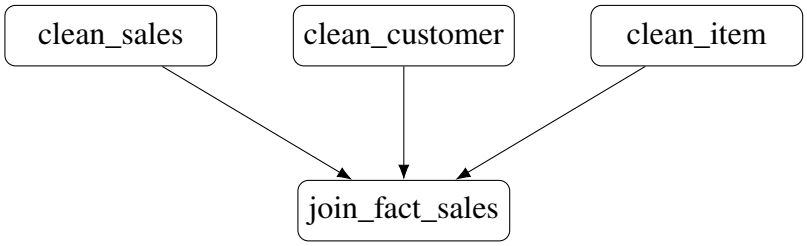


Figure 3.3: Silver convergence graph.

metrics. The Azure benchmark starts from prepared Bronze tables and executes the same four computational transformations without the sensor.

In the Azure benchmark, all three strategies admit the same pair of initially eligible cleaning tasks, as illustrated by the dependency structure in Figure 3.3. The Silver DAG therefore illustrates a case in which different numerical weights can produce the same task-admission pattern. Where admission is unchanged, timing differences cannot be explained by a different priority-selected sequence. It is not a randomised negative-control experiment, because the three strategy executions can still experience different run conditions.

3.3.3 Gold Layer

The Gold layer reads the integrated Silver fact table and creates three independent outputs. `agg_customer_ltv` groups sales by customer and calculates lifetime revenue, margin, profit, active days, unique items, and customer-value segments. `agg_sales_by_region` groups measures by customer region, channel, year, and month. `agg_top_items` aggregates item-level performance and applies a dense ranking to retain the top 1,000 items by revenue.

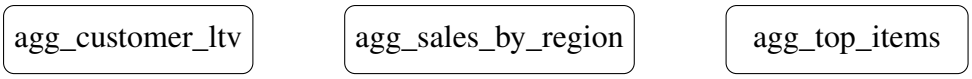


Figure 3.4: Gold independent-task graph. The absence of precedence edges makes concurrency and task compatibility the dominant scheduling factors.

As Figure 3.4 shows, no precedence edge exists, so all three tasks enter the ready set together. Under two active slots, priority selects the opening pair. Under three slots, all tasks may start. Under one slot, priority selects a purely sequential order. The graph removes precedence-driven release effects from the initial choice and makes differences in concurrent task execution observable through timestamps and durations. These measurements do not, by themselves, identify the underlying CPU, memory, cache, or control-plane contribution to a slowdown.

3.4 Scheduling Strategy Implementations

3.4.1 Static Priority Scheduling

In the Azure benchmark, Static scheduling assigns a fixed, manually selected table of integer priorities with the absolute weight rule. Higher weights take precedence when eligible tasks compete for a slot. This is an informed baseline rather than an arbitrary order. The Azure Bronze table gives the highest weights to `date_dim` and `item`, intentionally favouring shared sales-ingestion prerequisites. In Silver, the configured order begins with `clean_sales` and `clean_customer`; in Gold, Static selects Top Items and Sales by region as its opening pair.

The local Static benchmark used a different implementation. Its retained DAG definitions do not explicitly assign task priorities, and the local Static records used a base `priority_weight` of 1. Airflow's default downstream weight rule can aggregate successor weights, so a base weight of 1 does not imply an equally weighted effective ready-task queue. These local Static observations must not be described as executions of the manually weighted Azure Static policy.

3.4.2 HEFT-Based Upward-Rank Prioritisation

For a task i, an upward-rank score is calculated as

$$\text{rank}_u(i) = \bar{w}_i + \max_{j \in \text{succ}(i)} \left(\bar{c}_{ij} + \text{rank}_u(j) \right), \tag{3.1}$$

where $\bar{w}_i$ is the estimated computation cost and $\bar{c}_{ij}$ is the communication cost from task i to successor j (Topcuoglu et al., 2002). For terminal tasks, the maximum over the empty successor set is defined as zero. Communication-edge costs are set to zero in the implemented rank calculation; no separately estimated inter-task transfer cost is included. For terminal tasks the rank equals the estimated computation cost, and for non-terminal tasks the longest estimated successor path contributes to the rank. Ranks are scaled to integer `priority_weight` values and used with Airflow's absolute weight rule.

The local HEFT plugin computes upward ranks from its DAG and simulated base-duration estimates. Azure benchmark HEFT wrappers instead provide fixed integer tables from prior cloud task-duration estimates; for Gold, the retained historical factory identifies a preliminary Static execution as their source. These inputs produce different rankings: the retained local Gold HEFT ranks favour Customer LTV and Sales by region, while the recorded Azure HEFT opening pair is Sales by region and Top Items. Local Bronze HEFT also ties Item and Customer in its rank calculation, unlike the Azure Bronze HEFT priorities. A local computed ranking alone does not establish the exact timestamp order under tied priorities. The rank calculation is not rerun during each Azure DAG invocation. This ensures the strategy comparison evaluates a fixed policy rather than an adaptive learner. If computation-cost estimates or precedence information fail to represent the cloud workload, the resulting priority misalignment persists across repetitions. Neither version implements HEFT's classical processor-assignment phase: all Azure notebooks

are submitted to one existing cluster.

The retained historical Gold factory documents computation-cost estimates of 47.668 s (Sales by region), 47.663 s (Top Items) and 32.974 s (Customer LTV). These estimates came from a preliminary Static Gold cloud execution and may reflect co-running work. These values determine the corresponding terminal-task ranks; they are not isolated Spark runtime estimates. Consequently, Customer LTV can receive the smallest HEFT priority even though it takes longer under other measured execution contexts. The recorded Airflow weights, rather than constants in later historical code snapshots, define the priority treatment analysed here. The separate *local simulated* base durations are listed in Table A.2.

3.4.3 XGBoost Task-Priority Scoring

The XGBoost component is an offline regression model that predicts an engineered priority score. Its inputs combine duration, scale, graph structure, dependency release, task identity, and layer context. Table 3.4 summarises the feature groups.

Table 3.4: Feature groups used by the offline XGBoost priority scorer.

Feature group	Examples	Intended scheduling signal
Duration and scale	Base duration, normalised duration, row count, source size	Approximate computational demand
Graph position	Upstream count, downstream count, normalised counts, HEFT rank	Remaining path and precedence criticality
Dependency release	Direct-successor count and computed short-unlock score	Value of completing a task early
Task identity	One-hot task ID and task type	Repeated task-specific behaviour not explained by generic features
Layer context	Bronze, Silver, and Gold indicators	Structural context of the task
Historical performance	Observed or baseline duration summaries	Stability and empirical runtime information available offline

The target represents a deliberately constructed ranking objective, not labels derived from optimal schedules or measured makespan rewards.

Because the training target is a deterministic function of quantities also represented among the features, very high prediction accuracy on familiar tasks is expected. This reconstructs a prescribed scoring rule rather than discovering a schedule that minimises observed layer completion time. The implemented target is computed independently within each layer using normalised HEFT rank h_i, base-duration estimate d_i, direct-successor count f_i, and a short-unlock term $u_i = f_i(1 - d_i)$:

$$q_i = 1000\,(0.35h_i + 0.25d_i + 0.25u_i + 0.15f_i)\,. \tag{3.2}$$

The normalised quantities are based on the prescribed task metadata rather than online Spark resource measurements. The designed formula assigns weight to remaining-path estimates,

task duration and the number of directly released successors. Preliminary observations of the HEFT-based order informed the short-unlock feature and target design. The learned target is therefore an engineering choice, not an independently established scheduling optimum.

When a feature is constant within a layer, the implemented min–max normalisation assigns it a value of one. Consequently, the Gold layer, where all direct-successor counts are zero, receives constant normalised fan-out values. Its short-unlock term should not be interpreted as evidence of actual downstream unlocking. The feature groups and engineered target are specified in Table 3.4 and equation (3.2).

The training table contains 130 observations representing 13 distinct computational task identities, each repeated ten times in the retained local Static and HEFT history. The row-wise 75:25 split contains 97 training and 33 test rows. Categorical task, task-type, and layer attributes are one-hot encoded; numeric features include task estimates, graph descriptors, and historical duration and delay summaries. The latter summaries are aggregated by task over the collected local observations before the row split. The saved pipeline combines preprocessing with `XGBRegressor`. The training script records test mean absolute error (MAE), root mean squared error (RMSE), R^2, fivefold row-wise cross-validation MAE and feature importance.

Interpretation of model validation. The engineered target is constant for a given task identity, while repeated observations of that identity can occur in both partitions. The preprocessing and prior aggregation do not provide an unseen-task holdout. Hence the reported error assesses reconstruction of a specified priority function for familiar task patterns; it does not estimate performance on new transformations, unseen DAGs, or scheduling decisions under different resource conditions. Repetition of identical or nearly identical feature rows also limits the information available for a genuinely independent generalisation test. A grouped-by-task split would answer a different question and is not presented as part of the original training evaluation.

The offline fit statistics are reported in Table 3.5.

Table 3.5: Offline XGBoost priority-model evaluation.

Metric	Value
Training rows	97
Test rows	33
Mean absolute error	0.7646
Root mean squared error	0.9142
Test R^2	1.0000
Cross-validated MAE	0.8186 ± 0.1945
Target range	146.739–750.000

Upstream count has the largest reported importance, followed by normalised HEFT rank and downstream-count features. The model combines precedence structure, task identity, duration estimates, and historical aggregates. The feature-importances describe the fitted model under the constructed training target, not the causal importance of features for actual makespan. Pairwise

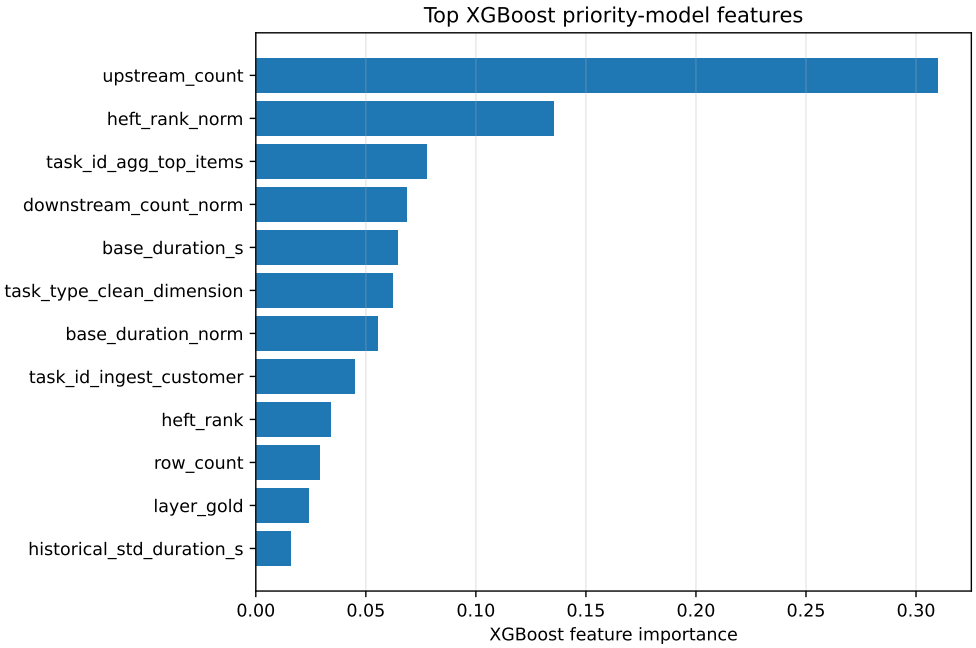


Figure 3.5: Twelve highest feature importances in the offline XGBoost priority scorer.

companion identity, simultaneous memory demand, and measured concurrent slowdown are absent; consequently the scoring representation cannot distinguish two different co-running companion tasks solely by their interaction with the focal task.

3.4.4 Distinguishing Prediction Accuracy from a Scheduling Treatment

A ranking also has a discontinuous relationship to prediction error. Suppose two eligible tasks have well-separated priorities. Small changes in their predicted values may not alter which task receives the next slot; the realised schedule is then unchanged. Conversely, an error close to a ranking boundary can reverse admission even if its contribution to an average regression metric is small. With three Gold aggregations and two slots, the practically relevant question is which task is excluded from the opening pair. A useful deployment check therefore compares the identity of the top two eligible tasks, not only the absolute discrepancy between predicted and target scores. For the evaluated tasks, the saved model produces the same within-layer ordering as the engineered priority targets. This correspondence concerns ranking, not the quality of the resulting schedule.

The fixed-priority Airflow implementation makes the evaluated priority assignments explicit and allows them to be checked against logged weights and task starts. The comparison tests the recorded XGBoost-derived priority *ordering* against the Static and HEFT-based orderings. It does not estimate the incremental effect of invoking a fitted regressor at runtime, and it does not distinguish that ordering from an otherwise identical manually configured lookup. Offline scoring could reduce manual work when task catalogues change, but this possible benefit was not evaluated on unseen tasks. The current results are valid measurements of the configured policy

without requiring either claim.

3.5 Airflow Priority and Concurrency Semantics

Airflow schedules a task only after its dependencies are satisfied and an execution slot is available. Priority weights do not override dependencies. They affect competition among ready tasks within the relevant executor, pool, and DAG capacity constraints (Apache Software Foundation, 2026*a*,*b*). The Azure benchmark DAGs and the explicitly weighted local HEFT and XGBoost DAGs use the absolute weight rule so that their configured weights are not recursively inflated by descendants. The local Static DAGs retain the Airflow default downstream weight rule.

The benchmark sets `max_active_tasks=2`. This value creates meaningful competition in all three layers. Bronze begins with three ready roots and two slots. Silver begins with three ready cleaning tasks and two slots. Gold begins with three independent aggregations and two slots. A value of three would eliminate the initial ranking choice in Gold, while a value of one would convert the experiment into purely sequential ordering.

A task may still start later than expected after a slot becomes free. Airflow must update states and queue the next operator, and the Databricks provider must submit a remote job. The Gold concurrency study records the intervals between a completed task and the next task start, where such transitions occur. The observed gaps reflect the combined orchestration and remote-execution path; they do not identify separate Airflow scheduling, notebook submission, and cluster-admission latencies.

3.6 Implementation Structure and Treatment Fidelity

3.6.1 DAG Factories and Strategy Wrappers

The project defines reusable task lists and scheduling configurations for each layer. For the Azure benchmark, each strategy–repetition combination was executed separately, with the same task dependencies and run-specific output tables. The Silver and Gold notebooks retain the original transformation logic; their input and output table names were made configurable to prevent experiments from overwriting one another. A comparison of notebook code supports the consistency of the implemented transformations; output checks provide additional evidence that comparable runs produced consistent results.

Each cloud execution has a unique Airflow run identifier and Databricks job identifier. Unsuccessful preliminary attempts were resolved and excluded from the completed benchmark sample. The mechanism experiments use additional DAG IDs and strategy labels. The Gold pairing wrappers change only the weights required to induce the controlled opening pair. The Gold concurrency wrappers hold weights fixed and change only `max_active_tasks`. The Bronze unlocking wrappers hold downstream weights fixed and exchange the order of the three initially eligible roots.

Azure benchmark DAG factories set `max_active_runs=1` to prevent overlapping repetitions of the same cloud DAG. The local strategy DAGs do not explicitly set this field; instead, the local runner triggers each DAG sequentially and waits for terminal completion before starting the next experiment. Azure task retries are disabled so that a successful benchmark run cannot contain a hidden repeated attempt. Execution timeouts provide a terminal failure condition rather than allowing a stalled remote run to remain active indefinitely.

3.6.2 HEFT and XGBoost Priority Generation

In the local benchmark, the HEFT component calculates upward ranks using the DAG and estimated task durations with zero communication costs. In Azure, the HEFT policy uses a fixed table of integer ranks; the full HEFT processor-assignment stage is not applied, because all Databricks notebooks share a single cluster. The Azure Static policy likewise uses a predefined integer table; this does not describe the local Static DAGs, which rely on Airflow defaults. The respective Azure weights are held fixed across repetitions.

The XGBoost model is stored together with its preprocessing steps. Its supervision is an engineered priority target, not labels from optimal cloud schedules. Before the Azure benchmark, the saved model was applied to the feature data for all 13 distinct tasks. The resulting predictions were rounded to integer Airflow priority values, and the model and input files were checked against their stored checksums. The experiment DAGs consumed those values with `weight_rule=absolute`; the model was not called by Airflow during task scheduling. The model generated scores before execution, and all 65 XGBoost task observations carried the corresponding integer priorities.

The cloud benchmark therefore used model-derived priorities, but *not* online adaptive ML scheduling. Predictions were computed in advance from fixed historical feature values. The model was not retrained between repetitions, and its features did not include current Spark cluster state or an executing companion task. The earlier benchmark used priorities based directly on the engineered training target and is not pooled with the repeated benchmark (see Appendix B.1). The reported comparison instead uses the saved model's predictions. For these tasks, both approaches lead to the same within-layer ranking; consequently, the results do not establish that model-based scoring creates a distinct or optimal schedule. Table A.8 reports the model predictions and the priorities applied in Airflow.

Airflow uses priority weights to choose among eligible tasks subject to dependencies and available slots. The analysis therefore examines actual start order as well as numerical weights; a runtime difference under the same effective order is not interpreted as an improvement caused by task prioritisation.

3.6.3 Implementation Verification

The applied benchmark priorities were checked against original Airflow task records containing integer weights and execution times. Databricks job records link each executed notebook to its run and task identifiers. All benchmark metrics matched the corresponding task and job identifiers, with no discrepancies in recorded priority values. The scheduling analysis uses these execution records to establish the applied weights and job-start times.

Effective schedule fidelity is checked from timestamps. For example, the Gold pairing condition is valid only if the intended two tasks begin in the first wave. The concurrency condition is valid only if the observed maximum overlap equals one, two, or three as planned. This distinction prevents the analysis from assuming that a configured priority produced the expected runtime order.

3.6.4 Traceability from Intended Priority to Executed Work

Confirming the applied scheduling policy requires linking task priorities to the execution records. At the Airflow-record level, the archived run metadata identify the DAG and applied task weights within the dependency graph, subject to the DAG's capacity limit, dependency constraints and scheduler state. At the execution level, the notebook receives a strategy label, run identifier and configured weight as parameters. Finally, the exported metrics associate the task identity with its recorded start and end times. Linking these artefacts makes it possible to check whether a difference in output statistics is accompanied by a difference in the relevant task-selection decision.

This trace is especially important because Airflow priorities are conditional preferences, not an absolute guarantee of a unique timestamp order. A task with the highest numerical weight cannot start before its predecessors have succeeded, nor can it displace already running work merely because a higher score has been assigned. Under two available slots, only priorities among *simultaneously eligible* tasks can change the opening set. The dependency graph therefore supplies part of the treatment definition. In Bronze, choosing between `ingest_item` and `ingest_customer` matters when both are eligible but only one slot remains. The former releases downstream sales tasks once the other shared prerequisite finishes. In Silver, three eligible cleaning tasks lead to the same initial pair under all tested tables. Thus a table with different numbers can represent the same admission treatment.

The Databricks job records establish the timing and successful completion of all 195 benchmark tasks. Their identifiers were matched to the corresponding metrics, excluding four preliminary test records. Processing windows are calculated from Databricks job timestamps rather than the supplementary metrics alone. Output consistency is examined in Section 4.4.7.

These linked records support the reported comparisons by relating configured weights to actual task execution. They also distinguish verification of the historical measurements from rerunning the experiment: a new cloud execution would be a separate sample that could experience

different infrastructure or cache conditions.

3.7 Notebook Execution and Measurement Boundaries

The notebooks are parameterised so that one Airflow task executes exactly one logical transformation. Parameters identify the task, strategy, run ID, priority, source or target tables, catalog, and schema. Identifiers are validated before being interpolated into SQL table names. This reduces accidental writes to an unintended namespace and makes the notebook reusable across strategies.

Within a notebook, the transformation/write timing begins immediately before the Spark action that materialises the output and ends when the write returns. The notebook's record-count validation and subsequent metrics append occur outside this narrower transformation timer. The primary repeated-cloud processing window, however, is independently reconstructed from *Databricks remote job start and end times*. These broader job intervals can include notebook setup, validation, metrics logging and other work occurring outside the transformation timer. Therefore, excluding a step from the notebook timer does not establish its exclusion from the main benchmark's remote-job duration.

Overwrite mode ensures that each run performs the complete materialisation rather than appending to an ever-growing target. It also means that concurrent tasks must write different tables, which is true for all evaluated layers. Shared input reads and shared cluster resources remain possible and are part of the intended Gold interference mechanism.

3.8 Local Benchmark Environment

The local environment uses Apache Airflow 2.9.0 in Docker, PostgreSQL for Airflow metadata and experimental metrics, and Python operators for task simulation. The custom image includes the Databricks provider and NetworkX so that local and Azure DAG definitions share the same orchestration stack. PostgreSQL is used because it supports repeated metric queries and avoids the limitations of a local SQLite metadata store.

Each local Bronze computational task sleeps for its declared base duration plus a uniform perturbation between -0.3 and $+0.3$ s. The corresponding Silver and Gold tasks use a perturbation between -0.5 and $+0.5$ s. All use a minimum simulated duration of 0.1 s. No common seed is used to match task-level noise across Static, HEFT, and XGBoost repetitions. The perturbation preserves broad task-duration differences while producing repeated observations for exploratory statistical analysis. The local task duration is not intended to imitate Spark stage behaviour. It represents an exogenous task cost that allows the effects of precedence, priority, and limited Airflow capacity to be observed in isolation.

For each layer and strategy, ten benchmark runs are retained. A PowerShell runner triggers the DAGs sequentially, waits for terminal state, and records the outcome of each run. Task metrics include scheduled time, start time, end time, duration, scheduling delay, layer, strategy,

DAG ID, and run ID. The local processing window is aggregated from the earliest task start to the latest task end. Scheduling delay is computed from Airflow's recorded `queued_dttm` to the task's `start_date`, with negative differences clipped to zero. It measures queue-to-start waiting, not the full period since the task first became eligible.

The first five retained Static repetitions came from an earlier local session than the corresponding HEFT and XGBoost observations. Later runs used a repeated Static–HEFT–XGBoost order rather than a randomised sequence. This chronology is identifiable from run IDs and timestamps. It precludes a justified paired comparison by common repetition index and leaves possible date, background-load, or run-order confounding. The statistical analysis therefore treats the groups as unpaired while acknowledging that independence may still be imperfect if conditions within a session are correlated.

The local benchmark offers controlled task costs and graph structure but limited realism. It cannot represent JVM warm-up, Spark SQL optimisation, shuffle, storage caching, executor memory pressure, Delta write behaviour, remote submission, or cloud control-plane variation. Its role is to test graph and priority behaviour under simulated costs and to provide qualitative context for the distinct Azure policy implementations, not an otherwise identical cloud-transfer treatment.

3.8.1 Local Run Structure and the Limits of Pairing

Independent-sample testing removes that unjustified index matching, but does not automatically solve every dependence concern. Runs collected in one session may share background CPU load, operating-system state, scheduler timing or other conditions. In addition, the later local collection followed a repeated strategy sequence rather than a fully randomised order. These factors limit a causal interpretation of every observed distribution difference. Sample means, medians, standard deviations and collection chronology help identify these limitations. The separate Azure and mechanism studies examine whether the dependency-related pattern also appears outside the original local batch.

The ten observations per cell are repetitions of the complete layer, not repetitions of each individual task considered in isolation. This unit matters: task durations from one run are linked by precedence and common orchestration conditions. Treating the six Bronze task rows of one run as six independent replicates would substantially overstate the effective sample size. The relevant runtime outcome is the difference between the latest task completion and earliest task start in that run. This window already incorporates the consequences of changed eligibility and overlap; summing the individual task durations instead would answer another question.

3.9 Azure Databricks Environment

3.9.1 Workspace and Compute

The cloud environment uses an Azure Databricks Premium trial workspace in UAE North. All benchmark and mechanism runs use the same single-node cluster:

- Databricks Runtime 17.3 LTS with Spark 4.0 and Scala 2.13;
- `Standard_D4ds_v4`, four cores and 16 GB memory;
- single-node mode;
- Photon disabled; and
- ten-minute auto-termination.

The local and Azure execution environments are compared in Table 3.6.

Table 3.6: Local and Azure execution environments.

Dimension	Local benchmark	Azure Databricks benchmark
Orchestration	Apache Airflow 2.9.0 in Docker	Same local Airflow instance with Databricks provider
Task execution	Python operators with bounded random sleep	Parameterised Spark notebooks on Azure Databricks
Data	Task metadata modelled from selected TPC-DS tables	Six selected TPC-DS SF1 sample tables
Storage	PostgreSQL task metrics	Delta tables and Unity Catalog metrics
Concurrency	max_active_tasks = 2	max_active_tasks = 2 in the benchmark
Repetitions	10 per layer-by-strategy cell	5 per layer-by-strategy cell
Primary metric	Airflow task processing window	Databricks task processing window
Purpose	Controlled graph and priority comparison	Spark/Delta execution and shared-resource effects

Airflow connects through a Databricks personal access token stored in its local connection configuration. Cluster ID, notebook paths, target catalog, and target schema are stored as Airflow variables. The notebooks receive task ID, strategy, run ID, priority, catalog, and schema as parameters. Existing-cluster submission is used so that the experiment does not create a different job cluster for every task.

The single-node design is intentionally resource constrained. It removes network transfer between workers and simplifies cluster configuration, but it creates a shared environment in which notebook tasks compete directly for the same CPU, memory, and local storage. It permits direct observation of how task durations and processing windows change with the number and identity of overlapping jobs, without measuring a separate hardware-interference term. The single-node design also restricts external validity to similar compact clusters.

3.9.2 Data and Transformations

The cloud workload uses six tables from `samples.tpcds_sf1`: `date_dim`, `item`, `customer`, `web_sales`, `store_sales`, and `catalog_sales`. The selected subset contains 5,229,704 records and 273.93 MB of reported compressed source data. SF1 denotes an approximately 1 GB complete benchmark database, but only the listed tables are processed. The selected subset does not constitute an execution of the full TPC-DS benchmark (Transaction Processing Performance Council, 2026).

Bronze tasks materialise source tables as Delta outputs. Silver tasks standardise and integrate the selected data, producing `silver_fact_sales`. Gold tasks independently aggregate the Silver fact table. Each task uses overwrite mode for its output, meaning that every run performs the transformation and replaces the corresponding table. Each strategy is expected to produce the same logical output.

The source-size value is read from `DESCRIBE DETAIL` when available. It is metadata rather than a measured byte count of every physical read. Record counts are calculated after the notebook's timed transformation and stored for integrity checking. Matching record counts provide output-count consistency evidence. They do not demonstrate row-by-row equality, equality of every numerical value, or identical output content. The validation scan is outside the notebook's transformation timer but may fall within the broader Databricks remote-job interval used for the primary benchmark.

3.9.3 Metric Storage and Semantics

The shared Delta table `etl_task_run_metrics_azure` contains one supplementary metrics entry for each benchmark task. Records for the first nine Bronze runs (54 tasks) were reconstructed later from archived execution data; the remaining 141 entries were collected using a subsequent recording procedure. Because the entries were not collected in the same way, the original Airflow task weights and Databricks job timestamps are used for the primary scheduling and timing comparisons. Table 3.7 lists the retained fields.

Two runtime concepts are retained:

$$T_{\text{processing}} = \max_i(t_i^{\text{end}}) - \min_i(t_i^{\text{start}}), \tag{3.3}$$

and

$$T_{\text{Airflow}} = t^{\text{DAG end}} - t^{\text{DAG start}}. \tag{3.4}$$

The sum of recorded task durations,

$$T_{\text{task-sum}} = \sum_i T_i, \tag{3.5}$$

Table 3.7: Task-level metrics retained for Azure analysis.

Field	Type	Analytical purpose
`run_id`	String	Identifies the benchmark repetition or controlled experimental condition; distinguishes preliminary tests
`strategy`	String	Encodes Static, HEFT, XGBoost, or the controlled mechanism treatment
`layer`	String	Separates Bronze, Silver, and Gold analyses
`task_id`	String	Supports task-level duration and execution-wave analysis
`start_time`, `end_time`	Timestamp	Define task duration, processing window, start offsets, and overlap
`duration_s`	Double	Quantifies task execution time under its observed context
`records_in`, `records_out`	Long	Validate equivalent work and output counts
`source_size_bytes`	Long	Records available source-size metadata
`priority_weight`	Double	Verifies the treatment applied to each task
`extra_json`	String	Retains input/output tables and execution metadata for auditability

is reported in task-seconds rather than as wall-clock duration. It can exceed the processing window when tasks overlap, and it can increase when co-running tasks take longer. This sum is not an independent meter of consumed CPU-seconds, memory or energy: it adds task elapsed times even while multiple tasks share the same machine.

Measurement boundaries across experimental phases

Table 3.8: Timing sources and interpretation boundaries.

Study	Reported interval	Interpretation
Local layer benchmark	Earliest to latest Airflow computational-task execution timestamps	Simulated Python task and scheduler interval; no Spark jobs
Earlier cloud benchmark	Earliest to latest recorded task intervals in earlier metrics exports	Historical notebook-level task measurements and recording path; not harmonised to final provider-level intervals
Repeated cloud benchmark	Earliest Databricks remote job start to latest job end	Provider-level execution window from 195 archived remote jobs
Controlled cloud studies	Earliest to latest task times in retained study-specific metrics	Within-study processing windows; do not compare absolute seconds across sessions

The earlier cloud export and the controlled studies use processing windows derived from their retained per-task timing records, including notebook-level recorded intervals in historical analyses; the repeated benchmark uses Databricks remote job start/end timestamps for all tasks.

These windows cover different operational measurement boundaries. *Absolute times cannot be compared across experiments or sessions as matched task costs*, even when the same logical notebooks and nominal cluster are involved. The much longer Gold windows in the repeated benchmark remain unexplained; the original cloud workspace is no longer available to investigate their cause. Differences in caching, code revisions and orchestration are possible explanations, not identified causal factors.

3.10 Azure Databricks Benchmark Protocol

The Azure Databricks benchmark was conducted in a single benchmark session. The design includes five repetitions for each combination of three layers (Bronze, Silver, Gold) and three scheduling policies (Static, HEFT and saved-model XGBoost), yielding 45 completed DAG runs and 195 computational task jobs. The nominal Airflow concurrency limit is two tasks per DAG, with one active DAG run and zero automatic task retries. Airflow submits one task per notebook to the existing fixed single-node Databricks cluster.

The deployment uses run-specific DAG IDs, idempotency tokens and output-table suffixes, together with the same intended transformation logic across the three policies. Silver uses a common set of previously prepared Bronze tables, and Gold uses the same prepared Silver fact table in every treatment. Those input choices prevent one strategy's upstream runtime or output table from determining another's layer-level result. The benchmark therefore measures three separate layer-level scheduling comparisons, not a complete Bronze–Silver–Gold execution under each strategy.

An earlier benchmark used the same nine strategy-by-layer cells and five repetitions per cell, but its XGBoost treatment took priorities directly from the engineered formula instead of predicting them with the saved model. The benchmark was therefore repeated to evaluate the intended model-derived implementation. Only the repeated benchmark enters the principal strategy comparison and the 175-run inventory. The earlier benchmark provides separate descriptive sensitivity context. XGBoost has the lowest Bronze mean in both sets, while the lowest-mean strategy changes in Silver and Gold. Because timing boundaries and execution conditions also differ, these changes do not identify random noise or an effect of the change in XGBoost scoring alone (Table B.1).

Each benchmark run was launched separately, after checking task configuration and the absence of another active benchmark run. The runs were sequential, but strategy order rotated across rounds. Archived DAG start times show Static–HEFT–XGBoost in the first round of each layer and HEFT–XGBoost–Static in the second. This partial counterbalancing did not constitute randomisation or matching of cloud state. Unsuccessful attempts caused by a missing notebook and an orchestration-host restart were resolved before the affected benchmark runs were completed. Only successful repetitions were included in the analysis; interrupted or failed attempts were not counted as additional observations.

All 45 benchmark DAG runs and 195 associated Databricks tasks completed successfully, with distinct remote job identifiers. Subsequent checks found all 195 run-specific Delta tables readable, with all 105 independently specified row-count expectations met; the other 90 outputs were assessed for consistency with the corresponding task results. This supports the validity of the reported execution and output comparisons without establishing row-by-row equality against an independent reference dataset. The 40 controlled mechanism runs were analysed separately.

3.11 Exploratory Mechanism Experiments

3.11.1 Bronze Dependency-Unlocking Study

The Bronze study compares two root orders under two active task slots. In *unlock early*, `date_dim` and `item` receive the two highest root priorities. In *unlock delayed*, `customer` and `date_dim` start first while `item` waits. Priorities of downstream sales tasks remain fixed. Five repetitions are collected per condition, and the order alternates across rounds.

The study measures layer processing window, time from first task start to `item` completion, time to the first downstream sales-task start, time to `store_sales` start, and sum of task durations. If the delayed condition has a worse processing window without greater aggregate task duration, the evidence supports a dependency-release explanation rather than a heavier-computation explanation.

3.11.2 Gold Opening-Pair Study

With two active slots and three independent tasks, three first-wave pairs are possible: Customer LTV plus Top Items, Customer LTV plus Sales by region, and Top Items plus Sales by region. Five repetitions are executed for each pair while the third task remains in the second wave. Retained chronological records show that the three conditions rotated across rounds, rather than running in fixed consecutive batches. The observed order of all 15 runs was checked against the recorded timestamps. Priority values are selected only to induce the intended opening pair; notebook code and data remain unchanged.

To describe task durations across the observed execution contexts, a relative difference is calculated as

$$S_i = \frac{\bar{T}_{i,\text{overlap}} - \bar{T}_{i,\text{nonoverlap}}}{\bar{T}_{i,\text{nonoverlap}}} \times 100\%. \tag{3.6}$$

An overlapping observation comes from a task admitted in the opening pair; a non-overlapping observation comes from the third task starting after that first wave. For each task identity, the calculation compares the pooled first-wave mean with the corresponding non-overlapping second-wave mean across the three controlled pairing conditions. The second-wave measurement is not an independent cold-run or standalone baseline: it occurs later in the same experimental workflow and can depend on caching, warm state, execution position, and cluster history. The

relative differences therefore describe *execution contexts consistent with interference*, rather than separately identifying an isolated causal effect of hardware contention.

3.11.3 Gold Concurrency-Sensitivity Study

The concurrency experiment fixes the priority order as Customer LTV, Top Items, and Sales by region, then varies `max_active_tasks` between one, two, and three. Five repetitions are executed per condition in a near-counterbalanced order. Concurrency one admits a single task at a time but does not remove orchestration or remote-notebook start gaps. Concurrency two uses the opening pair with the lowest observed mean in the separate Gold pairing study; that study did not establish a reliable difference between pairs. Concurrency three permits all three tasks to be admitted concurrently. These are alternative Airflow admission limits; they do not directly change the Spark executor's internal parallelism or the cluster hardware.

The study distinguishes wall-clock makespan from aggregate task-seconds through

$$R = \frac{\sum_i T_i}{T_{\text{processing}}}. \tag{3.7}$$

A larger R indicates a greater sum of task elapsed times relative to the layer window. It may reflect stronger overlap, slower co-executing tasks, or both; it is not a calibrated CPU-utilisation or energy-consumption measure. Start offsets allow the observed overlap to be checked and reveal inter-task gaps, but do not independently decompose those gaps into scheduler, provider, and cluster delays.

The pairing and concurrency studies are not pooled even where a condition appears similar. They were executed in separate sessions, and session-level cluster state or caching may affect absolute duration. Inference is made within each controlled study.

3.12 Experiment Automation, Filtering, and Data Validation

Experiment scripts controlled run order, checked that Airflow and Databricks were ready, and saved execution evidence. Before the cloud runs, the XGBoost scores were independently reproduced from the saved model. Each subsequent run was checked against its Airflow and Databricks records. Completed runs were not repeated by the experiment script, and unsuccessful attempts were recorded separately. The export was reconciled offline against 45 expected run identities and 195 task identities, including verification that no remote job ID was reused or recorded concurrency exceeded two.

Before Silver and Gold execution, prerequisite source tables were checked for availability and expected row counts. The six Bronze inputs contained 73,049 dates, 18,000 items and 100,000 customers. They also contained 718,931 web-sales, 2,879,789 store-sales and 1,439,935 catalog-sales rows. The Silver fact table contained 4,799,274 rows. After the benchmark, the 195 run-specific Delta outputs were examined for readability, row counts, schemas, source

identifiers and consistency between comparable executions. All tables were readable, and all 105 independently specified row-count expectations were met. The only observed numerical differences involved Gold top-item floating-point aggregates, which agreed after rounding to eight decimal places (Section 4.4.7). These checks support output consistency but do not establish independent row-level ground truth.

3.13 Statistical and Robustness Analysis

For every cell, the analysis reports sample size, mean, median, sample standard deviation, coefficient of variation, minimum, and maximum. The Azure benchmark summaries use the remote processing window computed from Databricks job interval timestamps; full Airflow DAG wall time is retained separately. The coefficient of variation is

$$\mathrm{CV} = \frac{s}{\bar{x}} \times 100\%. \tag{3.8}$$

Relative improvement over Static is calculated as

$$I = \frac{\bar{T}_{\mathrm{Static}} - \bar{T}_{\mathrm{candidate}}}{\bar{T}_{\mathrm{Static}}} \times 100\%, \tag{3.9}$$

so positive values favour the candidate.

For the primary local inferential analysis, two-sided exact Mann–Whitney U tests compare independent, unpaired run-level runtime samples (Mann and Whitney, 1947). The observed data comprise ten repetitions in each layer-strategy cell. A common repetition index does not constitute a matched random-seed block; the independent-sample approach avoids imposing an artificial one-to-one pairing. The tests compare ranks or distributions, not mean runtimes directly. Descriptive mean differences and percentage improvements are consequently reported alongside the tests, rather than presented as quantities whose significance was directly tested.

The primary comparison family contains six contrasts: HEFT against Static and XGBoost against Static in each of the three layers. Holm's step-down procedure adjusts the corresponding two-sided p-values for multiplicity (Holm, 1979). Secondary analyses of task delay, individual task duration, and HEFT–XGBoost comparisons are exploratory and are not interpreted as confirmatory evidence. Probability of superiority is reported as the proportion of all cross-strategy candidate–Static run pairs in which the candidate finishes sooner, assigning half credit for ties (Vargha and Delaney, 2000). As a rank-based effect summary, it does not require matching repetition labels.

The Azure benchmark is not subjected to null-hypothesis significance testing because five sequential observations per cell do not support strong inferential claims. Instead, 20,000-resample independent-within-strategy percentile bootstrap intervals are reported for means and differences (Efron, 1979). A leave-one-round-out analysis recomputes each comparison after removing one common round. If the direction changes, the comparison is considered sensitive to the small

sample. Bootstrap intervals and leave-one-out ranges are uncertainty diagnostics rather than additional independent evidence.

Spearman rank correlations between repetition index and runtime are calculated as a diagnostic for monotonic warm-up or drift. Given the small groups, mixed local session history, and partially counterbalanced cloud run order, they are not interpreted as definitive trend tests. Repetition index should not be mistaken for a uniformly spaced wall-clock measure. The complete individual runs remain in the analysis regardless of whether a value is slow, unless a documented execution failure occurred.

Exploratory controlled-study comparisons

For the separate Bronze prerequisite and Gold opening-pair studies, unadjusted exact two-sided Mann–Whitney tests provide descriptive checks using five observations per condition. These were not pre-registered or part of the primary six-test local family, and the observed cloud chronology prevents strong causal or population-level inference. The exact comparisons were calculated from the recorded run-level observations.

3.14 Validity and Reproducibility Controls

The principal validity risks and corresponding controls are summarised in Table 3.9.

Table 3.9: Principal validity risks and experimental controls.

Validity dimension	Main risk	Control or limitation statement
Internal	Run-order, warm-cluster, cache, and cloud-control-plane effects	Fixed configuration, sequential batches with recorded execution order, repeated runs, medians, dispersion, run-order diagnostics
Construct	Confusing task duration, processing window, full DAG runtime, or aggregate task-seconds	Separate metric definitions and explicit use of processing window as the primary Azure outcome
Implementation verification	Priorities differ from the intended strategy or duplicate rounds enter the sample	Run experiment records, priority-weight fields, run-ID patterns, and validation SQL
Output counts	A faster treatment omits records or transformations	Prerequisite Bronze and Silver sources counted; 195 outputs verified for readability and lineage; 105 predefined counts matched
Conclusion	Small cloud samples and multiple comparisons	Descriptive cloud interpretation, bootstrap and leave-one-out sensitivity, no selective removal of slow runs
External	One cluster, one selected data subset, and one fixed concurrency setting in the strategy benchmark	Layer- and mechanism-bounded claims; additional cluster and scale tests retained for future work

Internal validity is partially supported by fixed intended settings, run-ID filters, model

fingerprints, retained remote-job evidence, and explicit one-factor manipulations within the mechanism studies. These measures do not randomise all session conditions or eliminate caching effects. Construct validity requires keeping task duration, task processing window, Airflow DAG runtime, and aggregate task-seconds separate. Treatment fidelity is evaluated using the source priority table, logged weight, and observed effective start order; agreement among these is necessary before interpreting a runtime contrast as a consequence of a scheduling decision.

3.15 Experimental Records and Provenance

The Azure benchmark and controlled mechanism studies used separate run identifiers and were analysed as distinct experimental groups. Task execution records, source configurations and the saved model predictions allow the reported priority and timing comparisons to be examined independently. Checks of the resulting Delta tables are described in Section 4.4.7.

Record provenance and reproducibility limits. The analysis uses archived task and run records, including logged priorities, remote job identifiers and task timestamps. The exact source revision executed in the repeated cloud benchmark cannot be authenticated from the retained historical records; later implementation files therefore do not establish the original deployed configuration. Rerunning the cloud experiments would require independently configured infrastructure and could produce different timings.

Chapter 4

Results

4.1 Experimental Overview

The evaluation comprises 90 local benchmark runs, 45 Azure Databricks benchmark runs and 40 separate controlled cloud runs, representing 175 experimental runs and 735 computational task observations. The earlier cloud benchmark is not pooled with these runs; see Appendix B.1 for its descriptive comparison. The accuracy and comparability of the reported outputs are discussed in Section 4.4.7.

The local benchmark contains ten repetitions per layer and strategy. The Azure benchmark contains five repetitions per cell. Given the smaller cloud sample and the variability of some treatments, the cloud analysis emphasises mean, median, dispersion, individual-run sensitivity, effective start order, and mechanism evidence. No valid slow observation is removed.

4.2 Offline XGBoost Model Diagnostics

The offline priority model is evaluated before considering its scheduling results. The 130-row feature dataset is divided into 97 training and 33 test observations. The holdout MAE is 0.7646 and RMSE is 0.9142 on a target that ranges from 146.739 to 750.000. Five-fold cross-validated MAE is 0.8186 ± 0.1945. The reported errors are small relative to the engineered target scale, with test R^2 rounded to 1.000.

For the row-wise holdout, these results indicate close reconstruction of the engineered score for task identities already represented in the training set. The 130 observations comprise ten records for each of only 13 distinct tasks, with task-specific target values repeated across observations. This evaluation therefore does not test prediction on previously unseen task identities. Moreover, the displayed $R^2 = 1.000$ is rounded and should not be interpreted as a mathematically perfect fit. The feature-importance profile in Figure 3.5 is dominated by upstream count, followed by normalised HEFT rank, task identity, and downstream count; importance here characterises the fitted model rather than proving any individual feature's causal scheduling benefit.

The Azure benchmark uses numerical predictions from the saved XGBoost model, rounded

to integer Airflow priorities and checked against the archived model and feature files (Table A.8). The earlier benchmark using engineered scores directly is not pooled with the repeated benchmark (Appendix B.1). The records therefore establish that the benchmark used priorities derived from the saved model. Because the model was trained to reproduce an engineered priority function for familiar tasks, it still does not establish optimal scheduling learning, generalisation to new DAGs, or online adaptation.

4.3 Local Benchmark Results

4.3.1 Descriptive Runtime Results

The complete local runtime summaries appear in Table 4.1. Here Static denotes the local baseline using Airflow default task weights, not the manually weighted Azure Static policy described in Chapter 3.

Table 4.1: Local benchmark runtime by layer and strategy.

Layer	Strategy	n	Mean (s)	Median (s)	SD (s)	CV (%)
Bronze	Static	10	11.097	10.826	0.661	5.95
Bronze	HEFT	10	12.265	12.299	0.370	3.01
Bronze	XGBoost	10	11.536	11.400	0.489	4.24
Silver	Static	10	16.455	16.528	0.447	2.72
Silver	HEFT	10	16.558	16.610	0.664	4.01
Silver	XGBoost	10	16.537	16.522	0.475	2.87
Gold	Static	10	15.062	15.169	0.792	5.26
Gold	HEFT	10	13.338	13.173	0.649	4.86
Gold	XGBoost	10	13.771	13.543	0.772	5.61

Note: Runtime is the observed Airflow layer window. Lower values are better.

Figure 4.1 summarises the local means and sample standard deviations. Static is fastest in Bronze with a mean runtime of 11.097 s. HEFT is slowest at 12.265 s, while XGBoost lies between them at 11.536 s. In Silver, all three means are within 0.103 s. In Gold, HEFT is fastest at 13.338 s, followed by XGBoost at 13.771 s and Static at 15.062 s.

The distributions in Figure 4.2 show the pattern across repeated runs rather than in a single observation. Bronze HEFT is shifted upward relative to Static, while Gold Static is shifted upward relative to both alternatives. Silver distributions overlap strongly. The coefficients of variation remain between 2.72% and 5.95%, which is consistent with the bounded-noise design.

4.3.2 Pairwise Local Comparisons

The six primary comparisons against Static are reported in Table 4.2.

In Bronze, HEFT has a 10.53% higher mean runtime than Static; the independent-sample Mann–Whitney comparison remains significant after Holm adjustment ($p_{\text{Holm}} = 0.0012$). XGBoost has a 3.96% higher observed mean than Static, but the adjusted comparison is not

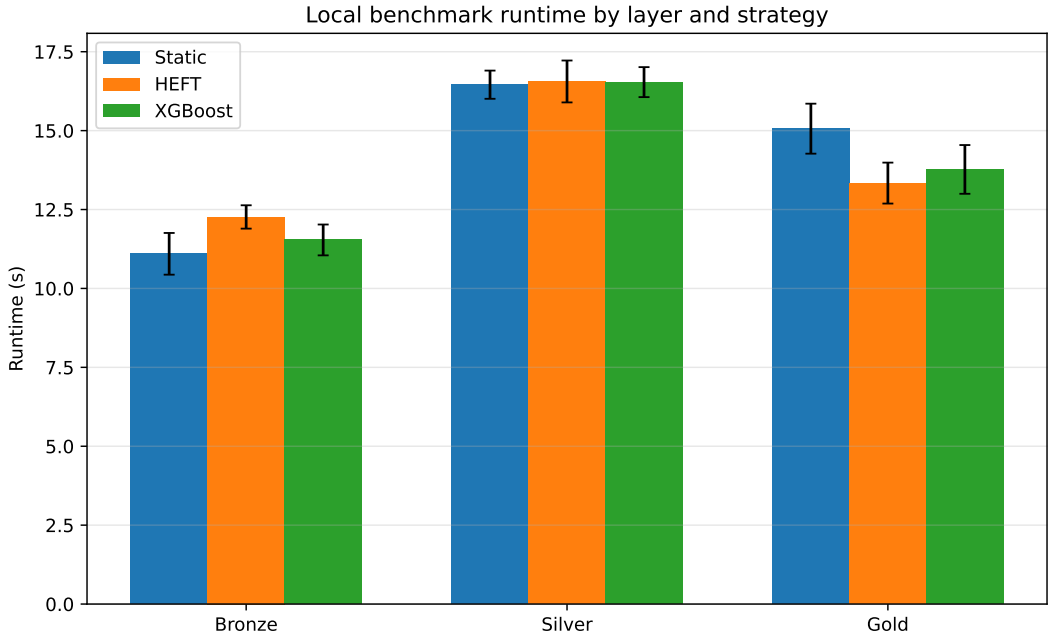


Figure 4.1: Local benchmark runtime by layer and strategy. Error bars show one sample standard deviation.

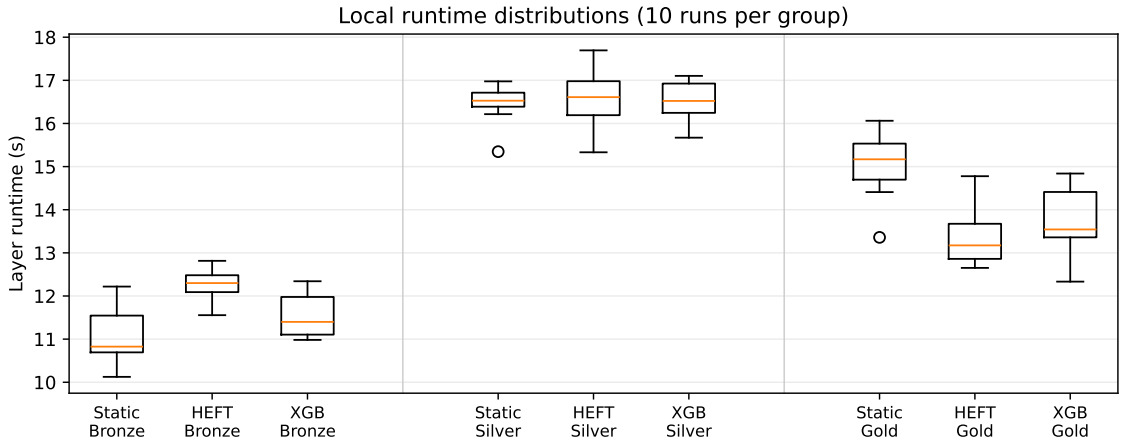


Figure 4.2: Local runtime distributions for ten repetitions per layer-by-strategy cell.

Table 4.2: Local pairwise runtime comparisons against Static.

Layer	Comparison	Improvement (%)	Δ mean (s)	Holm-adjusted p	P(faster)
Bronze	HEFT vs Static	-10.53	1.168	0.0012	0.05
Bronze	XGBoost vs Static	-3.96	0.440	0.2676	0.27
Silver	HEFT vs Static	-0.62	0.102	1.0000	0.45
Silver	XGBoost vs Static	-0.50	0.082	1.0000	0.47
Gold	HEFT vs Static	11.45	-1.724	0.0016	0.94
Gold	XGBoost vs Static	8.57	-1.291	0.0115	0.88

Note: Positive improvement indicates a lower mean runtime than Static. The p-values are from exact two-sided Mann–Whitney U tests with Holm correction for the six comparisons against Static. P(faster) is the cross-run probability that a randomly selected candidate run is faster than a randomly selected Static run (half credit for ties).

significant ($p_{\text{Holm}} = 0.2676$). The probability-of-superiority effect is 0.05 for HEFT and 0.27 for XGBoost, indicating different degrees of overlap in the observed runtime distributions.

In Silver, the observed mean differences relative to Static are approximately one tenth of a second, with no significant differences in the adjusted rank-based comparisons (both $p_{\text{Holm}} = 1.0000$). The probability-of-superiority estimates are close to 0.5. In Gold, HEFT has an 11.45% lower observed mean runtime than Static and XGBoost has an 8.57% lower mean. Both independent-sample comparisons remain significant after Holm adjustment ($p_{\text{Holm}} = 0.0016$ and 0.0115, respectively). The exploratory, unadjusted Mann–Whitney comparison between HEFT and XGBoost is not significant ($p = 0.2176$). These rank-based tests do not independently establish the magnitude of the mean differences.

4.3.3 Local Scheduling Delay

The runtime outcome depends on task durations and the times between successive execution waves. Figure 4.3 compares the mean local scheduling-delay metric, which does not by itself partition the entire layer window into execution and waiting components. Bronze changes root admission and hence downstream eligibility. Silver has the same first-wave pair and subsequent task sequence across strategies. Gold changes the initial pair and the identity of the task left for the second wave.

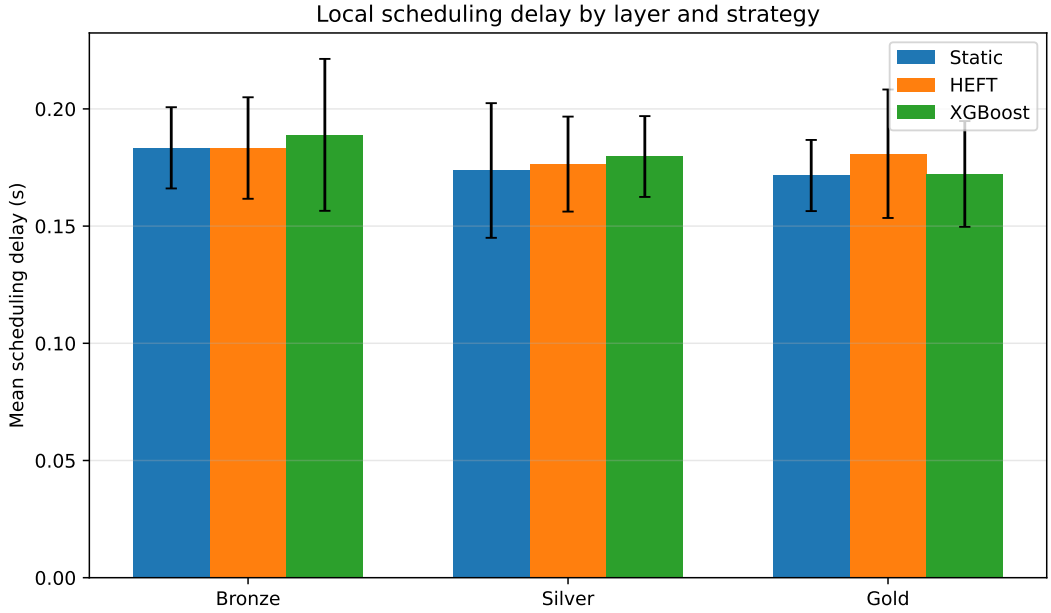


Figure 4.3: Mean local scheduling delay by layer and strategy. Delay is computed as Airflow task `start_date` minus `queued_dttm`, clipped at zero; it is not a measure of time since first eligibility.

4.4 Azure Databricks Benchmark Results

4.4.1 Measurement and Comparison Scope

The Azure Databricks benchmark includes five repetitions per strategy in each of the three layers, giving 45 completed layer runs and 195 successful task executions. All comparisons used a two-task concurrency limit. The processing times were calculated from the recorded Databricks task intervals, independently of the supplementary metrics. Output consistency is addressed separately in Section 4.4.7.

The primary Azure measure is the *remote processing window*, defined as the elapsed time between the first recorded Databricks task start and the final recorded Databricks task end in a layer-specific DAG. Airflow DAG duration is a secondary measure: it also reflects orchestration and job-submission overhead. The two measures must not be conflated. The strategy comparison is analysed separately from the controlled mechanism studies and from earlier trials that used engineered priority scores directly.

All three strategies were evaluated with the same layer DAG dependencies, notebook transformations within each layer, two-task Airflow capacity limit, and common layer-specific input snapshots. Bronze copies the same selected TPC-DS SF1 source tables. Every Silver trial reads the same prepared Bronze inputs, and every Gold trial reads the same prepared Silver fact table. The trials therefore compare *layer-level priorities with controlled upstream inputs*; they do not measure a strategy-specific end-to-end Bronze–Silver–Gold run. Because the cloud trials were sequential rather than randomised under identical cluster states, differences in elapsed time remain potentially confounded by warm-up, Spark execution, cache state, and platform activity.

4.4.2 Observed Processing Windows

Table 4.3 reports the descriptive distribution of the five processing windows for every treatment. Table 4.4 retains every individual observation, including unusually fast and slow Gold measurements.

Table 4.3: Azure processing windows (five repetitions per strategy–layer cell).

Layer	Strategy	n	Mean (s)	Median (s)	SD (s)	CV (%)	Range (s)
Bronze	Static	5	226.697	209.457	34.885	15.39	[194.837, 266.561]
Bronze	HEFT	5	246.057	233.206	41.881	17.02	[206.205, 310.002]
Bronze	XGBoost	5	187.639	185.905	14.817	7.90	[168.432, 202.392]
Silver	Static	5	165.323	164.518	9.289	5.62	[153.307, 178.832]
Silver	HEFT	5	152.387	153.489	5.478	3.59	[143.042, 157.349]
Silver	XGBoost	5	154.377	155.773	7.259	4.70	[143.709, 163.031]
Gold	Static	5	399.317	388.658	27.441	6.87	[375.446, 440.663]
Gold	HEFT	5	353.782	379.701	48.843	13.81	[270.043, 386.412]
Gold	XGBoost	5	346.976	360.513	113.574	32.73	[157.748, 456.820]

Note: The processing window spans the first recorded remote task start to the last recorded remote task end within a DAG. All results are descriptive; the separate verification of output correctness is reported in the results chapter.

Table 4.4: Individual remote processing windows for the Azure experiment (seconds).

Layer	Round	Static	HEFT	XGBoost
Bronze	1	266.561	233.206	168.432
Bronze	2	200.301	310.002	202.392
Bronze	3	209.457	264.003	179.126
Bronze	4	262.328	216.870	185.905
Bronze	5	194.837	206.205	202.340
Silver	1	178.832	157.349	163.031
Silver	2	162.029	143.042	143.709
Silver	3	153.307	153.162	157.843
Silver	4	167.928	154.892	151.527
Silver	5	164.518	153.489	155.773
Gold	1	440.663	350.939	157.748
Gold	2	413.168	386.412	353.703
Gold	3	388.658	379.701	406.095
Gold	4	378.648	381.813	360.513
Gold	5	375.446	270.043	456.820

Note: The round index is a within-strategy label, not a matched trial with identical cloud state. No observation was omitted on the basis of speed.

In Bronze, the model-derived XGBoost priority policy has the lowest observed mean, 187.639 s, against 226.697 s for Static and 246.057 s for HEFT. These correspond to a 17.23% lower mean than Static and 23.74% lower mean than HEFT. The result is consistent with the model-derived priority policy selecting `ingest_item` and `ingest_date_dim` for the first two slots, releasing sales tasks earlier than the HEFT order. Static selects those same two roots, but later ready-task ordering differs; the overall difference against Static cannot be attributed exclusively to the opening pair.

In Silver, HEFT has the lowest observed mean (152.387 s), followed closely by XGBoost (154.377 s), with Static at 165.323 s. The mean advantage of HEFT over Static is 7.82%, and XGBoost is 6.62% below Static. Importantly, the three strategies all admit `clean_sales` and `clean_customer` first, then `clean_item`, followed by the dependency-constrained `join_fact_sales`. Their task admission waves are equivalent at the tested capacity, although the exact start timestamps of the first two tasks differ between runs. Consequently, the timing differences cannot credibly be attributed to the priority rule causing a different task order.

In Gold, XGBoost has the lowest five-run mean (346.976 s), only 6.806 s below HEFT (353.782 s), whereas Static averages 399.317 s. This is a difference of approximately 1.92% between XGBoost and HEFT; the five-run means alone do not establish a reliable advantage. Gold XGBoost includes a very fast first round and a particularly slow fifth round, yielding a large sample standard deviation (113.574 s). The HEFT and Static policies admit the same first two tasks, whereas XGBoost admits Customer LTV and Sales by region. Their different recorded execution contexts are relevant, but uncontrolled session conditions remain alternative explanations for the numerical differences.

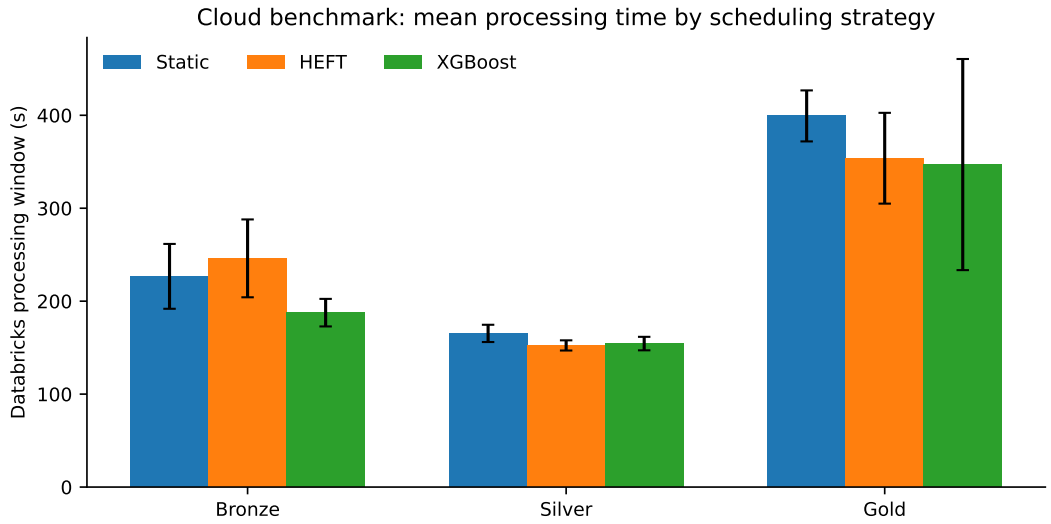


Figure 4.4: Azure Databricks processing windows by layer and strategy. Bars show five-run means; error bars denote sample standard deviation. This figure does not show full Airflow DAG duration.

4.4.3 Comparison with Static and Sensitivity to Small Samples

Table 4.5 reports the independent-group bootstrap differences and leave-one-round-out sensitivity calculated from the retained cloud records. The bootstrap resamples five measurements within each strategy independently, rather than treating identical round labels as matched cloud-state blocks. It is used as a descriptive uncertainty aid, not as a significance test. The leave-one-round-out analysis omits a common round label in each contrasted group to reveal sensitivity; it likewise does not transform a sequential observational batch into a randomised experiment.

Table 4.5: Azure comparisons against Static and leave-one-round-out sensitivity.

Layer	Comparison	Improvement (%)	Δ mean (s)	Bootstrap 95% CI for Δ (s)	LOO improvement range (%)
Bronze	HEFT vs Static	-8.54	+19.360	[-22.204, +63.045]	[-16.33, +1.38]
Bronze	XGBoost vs Static	17.23	-39.058	[-69.292, -10.589]	[+11.21, +21.60]
Silver	HEFT vs Static	7.82	-12.936	[-21.595, -4.779]	[+6.67, +9.58]
Silver	XGBoost vs Static	6.62	-10.946	[-20.320, -1.851]	[+5.48, +8.80]
Gold	HEFT vs Static	11.40	-45.535	[-94.274, -7.243]	[+7.54, +14.27]
Gold	XGBoost vs Static	13.11	-52.341	[-154.710, +25.664]	[-1.36, +21.16]

Note: $\Delta = \bar{T}_{\text{candidate}} - \bar{T}_{\text{Static}}$; negative values favour the candidate. Independent-within-group bootstrap resamples (20,000 draws; seed 10202026) and common-label leave-one-round-out checks are descriptive, not causal or confirmatory tests.

The differences in Gold demand particular caution. XGBoost's first remote processing window is much shorter than its other repetitions, and its fifth round is slower than both competing policies. Removing Round 1 from *both* XGBoost and HEFT for a sensitivity check changes the four-run mean ordering: XGBoost is then slower than HEFT. This calculation does not justify excluding Round 1. Rather, it demonstrates that the apparent five-run Gold lead is fragile and cannot support an unconditional claim of ML superiority. All five results remain in the main

summaries.

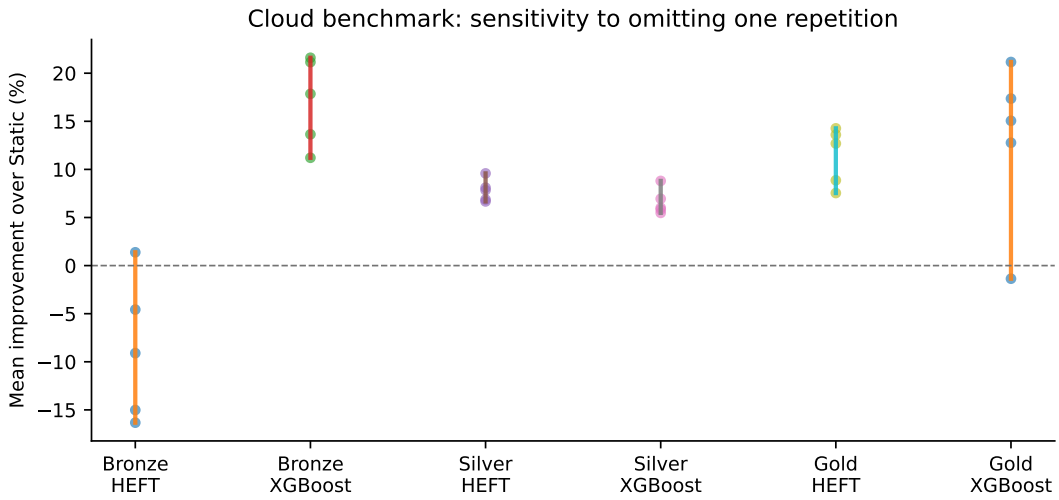


Figure 4.5: Descriptive improvement over Static after omitting one nominal repetition label at a time from both strategy groups. Positive values indicate a shorter candidate processing window.

4.4.4 Model Predictions and Observed Execution Order

The XGBoost task priorities were calculated before cloud execution, rather than by a model running inside the Airflow scheduler. The saved model provided scores for all 13 tasks. Each prediction was rounded to an integer and used as the corresponding Airflow priority, consistently across the five repetitions in each layer. This establishes which scores were used; it does not demonstrate that the model learned optimal scheduling decisions or adapted to new conditions during execution.

Table 4.6 illustrates representative measured task-start orders reconstructed from the remote start times. The Bronze strategy policies differ mainly in root release and later ready-task selection. Silver illustrates an unchanged task-admission pattern despite different numeric priorities. In Gold, Static and HEFT select the same opening pair despite substantially different numerical priority scales, whereas XGBoost selects a different pair. These facts constrain how performance differences may be interpreted: different weights only create a distinct scheduling treatment when they change which eligible tasks can begin.

What the recorded starts establish. Table 4.6 illustrates why the identity of the first two tasks is more informative than the size of their priority weights. Each layer begins with three computational tasks that are eligible under the benchmark dependencies, while the configured limit permits two to be active. The first choice therefore concerns the *set* admitted in the opening wave. Within that set, small differences in recorded start times do not necessarily represent a different choice by the priority rule. The times are remote job-start timestamps, which also reflect the order and duration of notebook submissions.

Table 4.6: Observed first two remote task starts in Azure Round 2, under the configured two-slot limit.

Layer	Strategy	First two task IDs by remote start timestamp
Bronze	Static	`ingest_date_dim` and `ingest_item`
Bronze	HEFT	`ingest_date_dim` and `ingest_customer`
Bronze	XGBoost	`ingest_item` and `ingest_date_dim`
Silver	Static	`clean_sales` and `clean_customer`
Silver	HEFT	`clean_customer` and `clean_sales`
Silver	XGBoost	`clean_sales` and `clean_customer`
Gold	Static	`agg_sales_by_region` and `agg_top_items`
Gold	HEFT	`agg_sales_by_region` and `agg_top_items`
Gold	XGBoost	`agg_customer_ltv` and `agg_sales_by_region`

Note: A representative round illustrates the treatment. Opening pairs from all 45 benchmark runs are reported in Table A.9; the full-run admission-pattern summary is in Table 4.7.

In Bronze, Static and XGBoost begin with the two shared prerequisites, `ingest_date_dim` and `ingest_item`, whereas HEFT starts `ingest_customer` instead of `ingest_item` alongside `ingest_date_dim`. This is an observable change in the admitted set, not merely a different numeric score. Its importance comes from the graph: the sales loads that require `item` cannot become eligible until that task completes, even if another execution slot has become available. However, sharing the opening set does not make their complete Bronze schedules identical. In all five Static runs, `ingest_web_sales` started before `ingest_catalog_sales`, while XGBoost reversed that order in all five runs. Both sales tasks depend on the same two shared roots. This confirms a repeatable later-admission difference, but does not isolate its contribution to the observed mean processing-window contrast.

In Silver, each recorded strategy admits `clean_sales` and `clean_customer` in the opening wave, with `clean_item` subsequently admitted and the final join awaiting all three cleaners. One of the first two tasks may have an earlier remote timestamp in one strategy and a later timestamp in another. That reversal does not change the opening *pair* or remove the dependency barrier before the join. The different mean windows in Table 4.3 therefore cannot be interpreted as improvements caused by selecting a different pair of cleaners. Their descriptive differences remain worth reporting, but the scheduling mechanism must be distinguished from variation in task execution conditions.

Gold provides a third case because its aggregations have no predecessor relationships with one another. Static and HEFT choose Sales by region and Top Items, leaving Customer LTV for the following wave. XGBoost instead chooses Customer LTV and Sales by region. Here the initial scheduling decision really does differ, and the identity of the overlapping tasks is relevant to the processing window. Nevertheless, the recorded start order alone does not identify the time that those same tasks would have taken under a different cloud state. The additional pairing experiment examines the alternative opening pairs under a separate protocol; its absolute window measurements are not substitutes for the benchmark runs.

These are interpretations of task admission, not new statistical tests. Table 4.7 summarises

the choices reconstructed across all 45 runs; Table A.9 lists their opening pairs. The recorded task starts for all 45 runs distinguish actual scheduling choices from numerical priority differences without assigning every runtime contrast to the policy.

Table 4.7: Admission choices reconstructed from all 45 Azure benchmark runs.

Layer	First two remote task starts	Subsequent observed order
Bronze	Static/XGBoost: Date + Item (5/5 each); HEFT: Date + Customer (5/5).	After both shared roots: Static first starts Customer + Web Sales (5/5); XGBoost Customer + Catalog Sales (5/5); HEFT Web Sales + Store Sales (5/5).
Silver	All policies: Sales + Customer (15/15).	Item next, then the prerequisite-constrained fact join (15/15).
Gold	Static/HEFT: Region + Top Items (10/10); XGBoost: Customer LTV + Region (5/5).	Customer LTV is last for Static/HEFT (10/10); Top Items is last for XGBoost (5/5).

Note: Names abbreviate the corresponding task IDs. Pair order within an opening set is ignored. In Bronze, the next pair consists of the first two remote starts after both Date and Item have completed. Counts describe consistency within the archived runs, not independent causal tests.

4.4.5 Task-Level Execution and Timelines

The recorded task start times, completion times and durations allow the schedules to be examined beyond layer-level means. Figures 4.6 to 4.8 summarise the remote job durations. These are provider-level elapsed job intervals, not measures of exclusive CPU work or the precise timed Spark transformation section within each notebook.

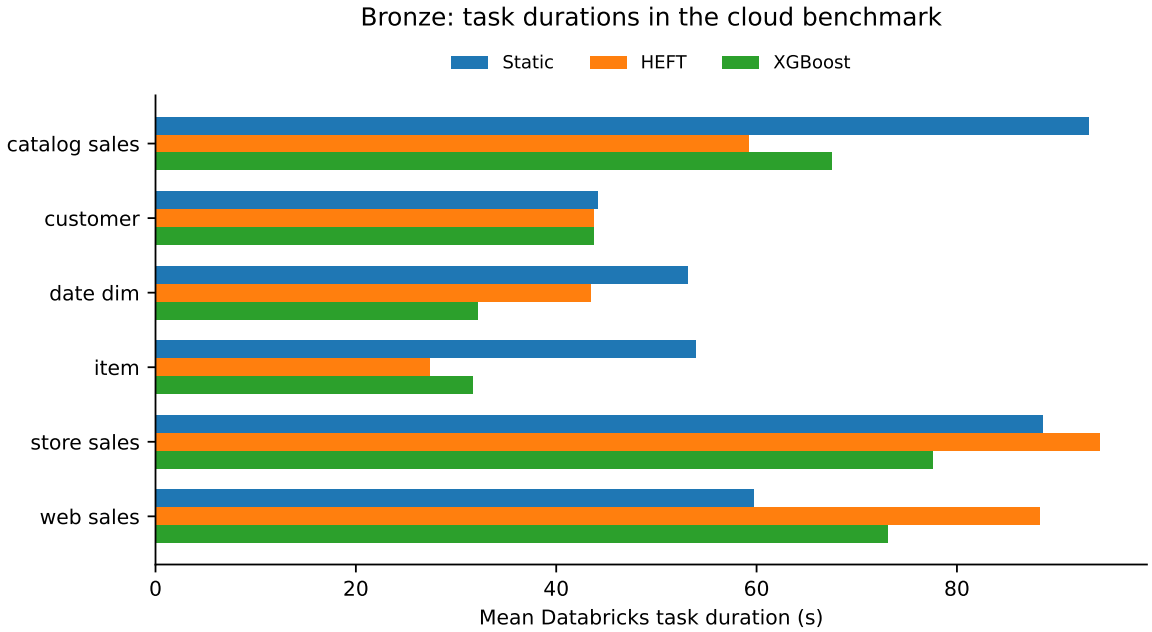


Figure 4.6: Bronze remote task durations in the Azure Databricks benchmark. Each bar is the mean of five job-level durations; jobs can overlap within one layer.

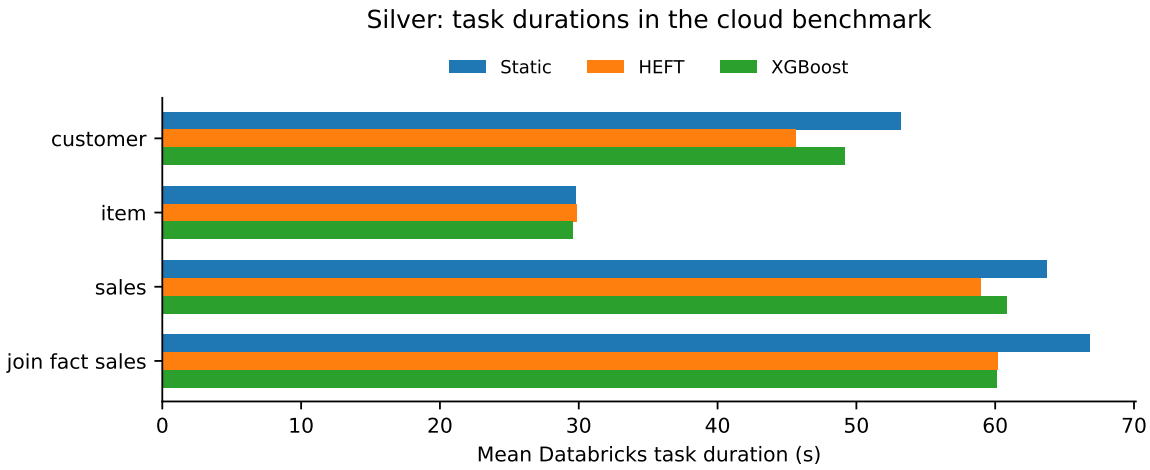


Figure 4.7: Silver remote task durations by scheduling strategy, averaged over five benchmark repetitions per cell. The three policies admitted the same first-wave task pair.

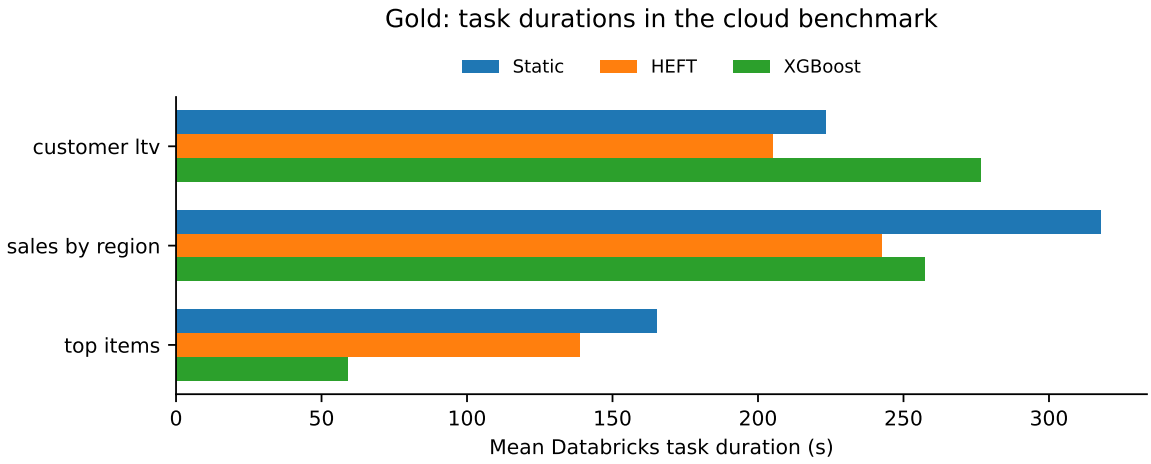


Figure 4.8: Gold remote task durations by scheduling strategy, averaged over five benchmark repetitions per cell. Each mean describes task executions under the observed co-execution conditions.

The timelines in Figures 4.9 to 4.11 display Round 2 for each strategy relative to that individual run's first remote start. They are separate trials, not simultaneous branches of one cloud execution. Their principal purpose is to show the realised waves rather than infer cloud resource utilisation from overlapping rectangles.

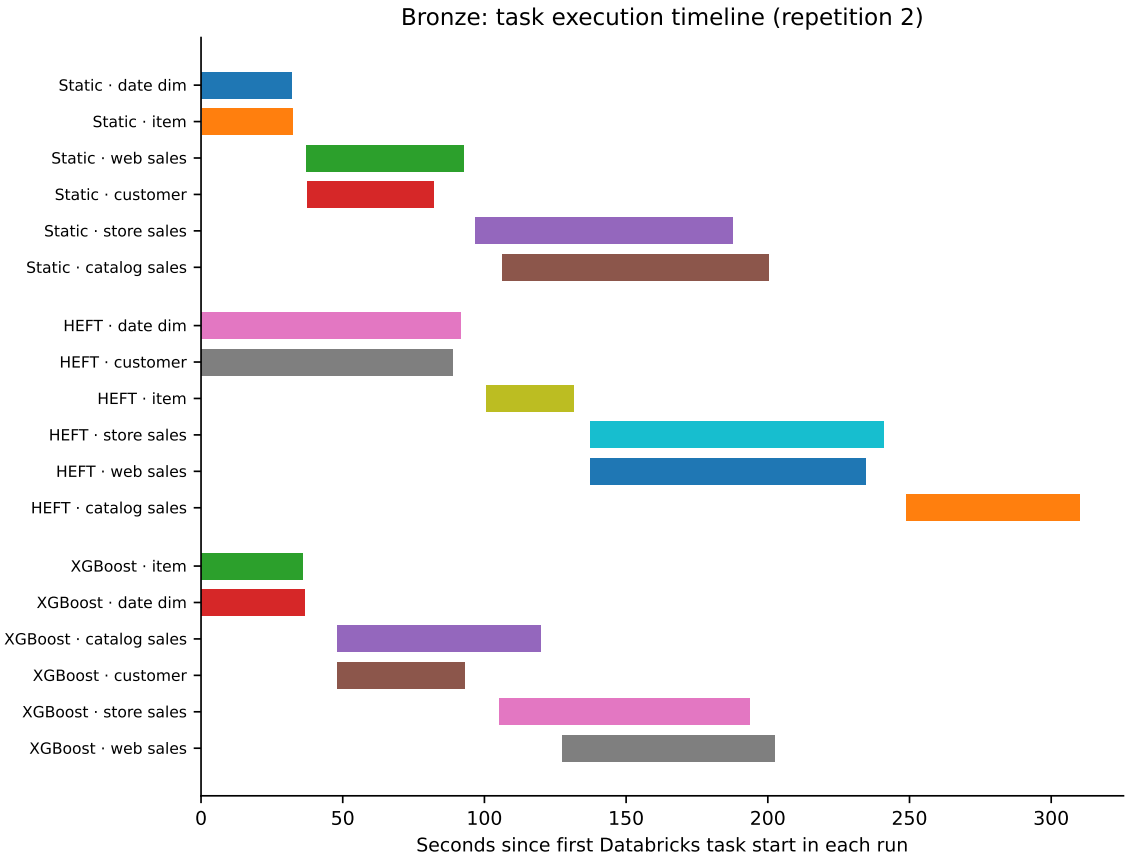


Figure 4.9: Bronze repetition 2 remote task windows, each trial aligned to its own first job start.

4.4.6 Local-to-Cloud Transfer

The local benchmark tests priority and precedence effects with bounded Python task durations, whereas the Azure benchmark includes Databricks remote job intervals, Spark execution, Delta writes and varying cluster state. The concrete policies also differ: local Static uses Airflow default weights instead of the manually weighted Azure Static table, and local HEFT ranks can differ from Azure HEFT. Table 4.8 therefore contrasts *within-environment improvements relative to each environment's own Static baseline*, not the transfer of identical policies or raw timing values.

The local and Azure samples both place HEFT behind Static in Bronze mean processing time, but their computed HEFT priorities and Static baselines are not identical. Silver provides limited admission freedom in the Azure benchmark, while the local task-cost experiment has a different priority context. Bronze XGBoost is slower than its local Static baseline but faster than

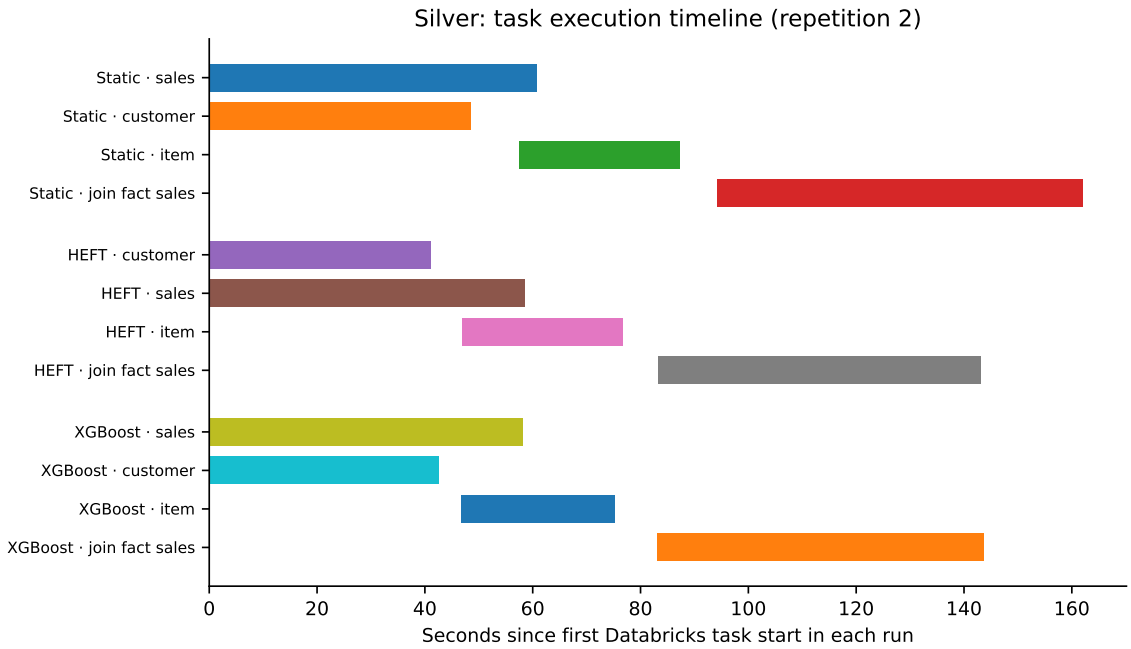


Figure 4.10: Silver repetition 2 remote task windows, each trial aligned to its own first job start.

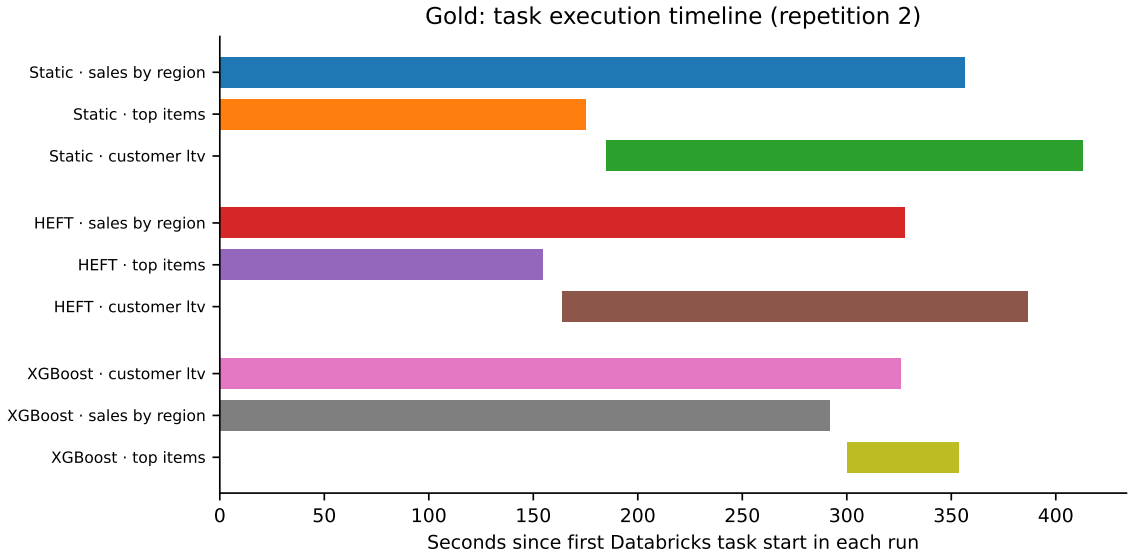


Figure 4.11: Gold repetition 2 remote task windows, each trial aligned to its own first job start.

Table 4.8: Within-environment mean processing-window improvement over Static.

Layer	Strategy	Local (%)	Azure (%)
Bronze	HEFT	-10.53	-8.54
Bronze	XGBoost	-3.96	+17.23
Silver	HEFT	-0.62	+7.82
Silver	XGBoost	-0.50	+6.62
Gold	HEFT	+11.45	+11.40
Gold	XGBoost	+8.57	+13.11

Note: Positive percentages indicate lower mean processing-window duration than Static within that session. Results use each environment's own Static baseline and priority implementation; the controlled mechanism studies are not pooled with the Azure benchmark.

Azure Static; the small Gold XGBoost–HEFT cloud contrast is sensitive to individual rounds. These differences cannot be assigned solely to Spark effects or cloud state because the concrete scheduling configurations, as well as task durations and execution environments, changed.

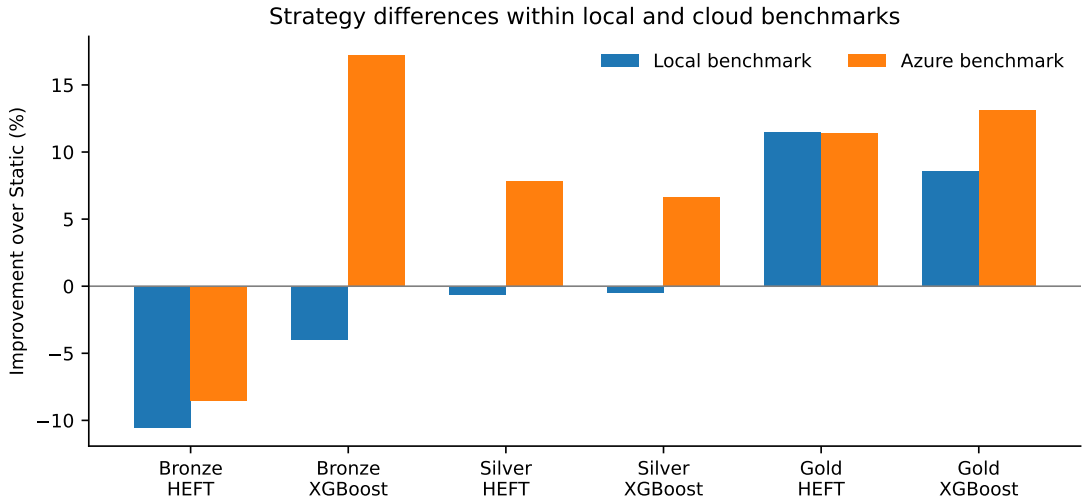


Figure 4.12: Within-environment mean processing-window differences relative to Static. Local and Azure absolute durations and concrete priority implementations are not directly comparable.

4.4.7 Independent Output and Metrics Validation

All 195 run-specific Delta output tables were present and readable. Their schemas and source identifiers were examined, and all 105 outputs with predefined row-count expectations matched those values. The remaining 90 outputs were assessed for consistency with comparable runs rather than against independently specified expected counts. Across 13 logical task groups, the recorded comparisons were consistent except for small numerical differences in the Gold top-item aggregation.

A separate Gold-specific check examined all 15 top-item result tables and compared representative tables from the two fingerprint classes. Both contained exactly 1,000 distinct item keys, with zero duplicate keys or key-set differences. The integer rank, sales-line-count, quantity and

unique-customer columns matched. Maximum absolute differences were 3.027×10^{-9} for total revenue, 8.149×10^{-10} for net profit and 4.657×10^{-10} for gross margin. There were 1,000 raw row differences in each direction, but none remained after rounding numeric aggregates to eight decimal places. Thus the top-item outputs are numerically equivalent at the stated tolerance, *not* bitwise identical. These comparisons establish agreement at the stated numerical precision, not exact equality with an independent reference result for every row.

The cloud metrics table contained 195 records for the completed benchmark tasks and four records from preliminary tests, which were excluded. All 195 benchmark records could be matched to their Airflow runs and Databricks jobs, with no missing task identifiers or numerical priority mismatches. For the first nine Bronze runs (54 tasks), supplementary measurements were assembled after execution from archived records. The remaining 36 runs (141 tasks) used the later recording procedure. This distinction concerns how supplementary metrics were collected, not a change to the scheduling treatments. The priority and timing analysis relies on the original Airflow and Databricks execution records.

4.5 Bronze Dependency-Unlocking Experiment

Table 4.9: Bronze dependency-unlocking mechanism results.

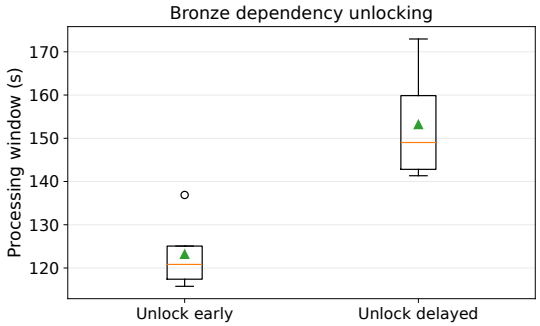
Condition	n	Mean window (s)	Median (s)	SD (s)
Unlock early	5	123.190	120.823	8.447
Unlock delayed	5	153.199	149.021	13.238

Condition	Item finished (s)	First sales start (s)	Sum task duration (task-s)
Unlock early	4.305	39.973	83.412
Unlock delayed	41.224	73.943	79.679

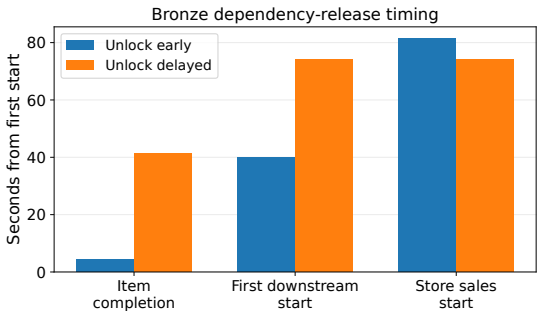
Note: Item completion and first sales start are offsets from the first task start. Task-seconds are summed elapsed task durations, not measured CPU usage.

Table 4.9 shows that the unlock-early condition achieves a mean processing window of 123.190 s, compared with 153.199 s for unlock delayed. The delayed-release condition had a mean processing window 30.009 s (24.36%) longer than early release, a median over 28 s higher, and greater dispersion, indicating that the mean difference is not attributable to a single summary statistic.

As illustrated in Figure 4.13, the delayed condition postpones mean `item` completion by 36.919 s and the first downstream start by 33.970 s. At the same time, summed observed task duration is slightly lower: 79.679 task-seconds, compared with 83.412 task-seconds for early unlocking. The increase in elapsed window is therefore not accompanied by an increase in summed task duration. Within this controlled contrast, the timing decomposition supports the



(a) Bronze processing time under early and delayed dependency release in the separate controlled cloud experiment.



(b) Observed dependency-release times in the separate Bronze experiment.

Figure 4.13: Bronze unlocking mechanism. The delayed condition postpones the high-unlock root and downstream eligibility.

explanation that later dependency release reduces useful overlap and creates a longer sequence of waves; task-seconds are not a direct measure of CPU or memory consumption.

The controlled Bronze experiment provides context for the benchmark results. The tested HEFT-based rank places the high-unlock `item` task later, whereas Static and the deployed XGBoost-derived priorities place it in the opening wave. The experiment supports the value of early unlocking in this DAG, not a claim that critical-path reasoning is generally ineffective. The differing measurement settings of the strategy benchmark and the controlled Bronze study are described in Section 3.10.

The five early-unlocking windows are lower than all five delayed-unlocking windows (median 120.823 versus 149.021 s). An unadjusted exact two-sided Mann–Whitney test gives $U = 0$ and $p = 0.0079$. This exploratory result supports an association within the controlled study but does not eliminate run-order or session confounding; the observed 24% mean difference is a descriptive sample contrast.

4.5.1 Reading the Bronze Intervention as a Timing Mechanism

The Bronze intervention produces two linked differences rather than merely a change in a final average. Under the early condition, the `item` prerequisite completes on average 4.305 seconds after the first computational task starts, and the first downstream sales task begins at 39.973 seconds. Under the delayed condition, `item` completion occurs at 41.224 seconds and first downstream start at 73.943 seconds. The corresponding delays of 36.919 and 33.970 seconds provide temporal evidence for the proposed dependency-release mechanism. The fact that the downstream-start shift is slightly smaller than the `item` completion shift is compatible with other prerequisites and scheduling transitions also affecting eligibility and actual execution.

Importantly, these timing offsets are not themselves the total layer window. After a sales

task becomes eligible, it still competes for the available capacity and may execute alongside another task. Its own completion time then depends on its duration and any subsequent wave. The observed 30.009-second increase in the mean processing window thus need not equal the 33.970-second difference in first downstream start. The two measurements describe different positions in the dependency graph and are linked through the remainder of the schedule. Treating them as independent effect estimates would double-count a shared scheduling event.

The sum of task durations helps distinguish waiting from longer recorded task execution. The delayed condition has a slightly lower mean sum of task durations, 79.679 task-seconds, than the early condition's 83.412. Hence the longer layer window is not accompanied by a greater recorded total of task elapsed times. The result is consistent with waiting and reduced useful overlap arising from delayed eligibility, rather than with the delayed runs simply containing more computation. This is an interpretation of the measured offsets and durations under the two treatments; it does not require claiming that all remote submission and processing-state effects were held perfectly constant.

The Bronze benchmark and the controlled experiment serve different purposes. In the benchmark, the HEFT policy happens to delay the same high-unlock root; in the mechanism study, the root order is deliberately manipulated while the remaining logical work is held fixed. The intervention therefore strengthens the explanation for the unfavourable Bronze ordering observed in the benchmark. Its sample remains five per condition, and the estimated 24.36% increase applies to this graph, source subset and cluster configuration, not to every ingestion pipeline with a task called `item`.

4.6 Gold Opening-Pair Experiment

Table 4.10: Gold processing window by first-wave task pair.

First-wave pair	n	Mean (s)	Median (s)	SD (s)	Range (s)
LTV + Top Items	5	90.313	89.872	1.551	[88.857, 92.510]
LTV + Region	5	94.032	92.898	6.866	[86.492, 105.102]
Top Items + Region	5	107.586	95.194	22.604	[88.808, 141.193]

Note: Each condition used max_active_tasks=2; the third task executed in the second wave.

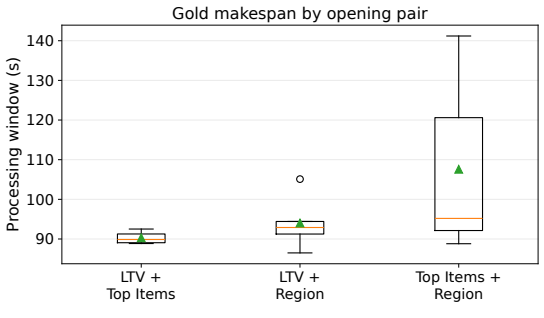
As shown in Table 4.10, the lowest mean processing window is obtained when Customer LTV and Top Items form the first wave, at 90.313 s. Customer LTV plus Sales by region averages 94.032 s. Top Items plus Sales by region is slowest at 107.586 s and has a much larger standard deviation of 22.604 s. Its median of 95.194 s is below its mean, suggesting that slow observations influence the five-run average.

Table 4.11 and Figure 4.14 show that, for each Gold task, the pooled first-wave overlapping mean exceeds the non-overlapping second-wave mean. The descriptive increases relative to the second wave are 52.57% for Customer LTV, 92.24% for Top Items, and 126.16% for Sales by

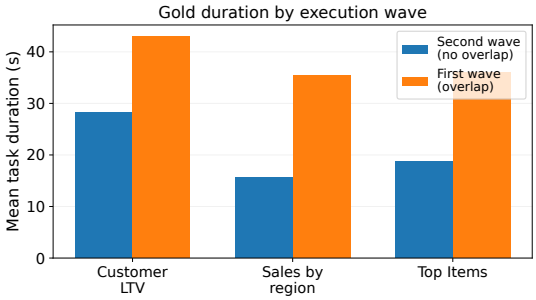
Table 4.11: Gold task durations in overlapping first-wave and non-overlapping second-wave contexts.

Task	Second-wave n	Mean (s)	First-wave n	Mean (s)	Increase (%)
Customer LTV	5	28.224	10	43.062	52.57
Sales by region	5	15.686	10	35.475	126.16
Top Items	5	18.738	10	36.023	92.24

Note: Second-wave tasks execute without task overlap after the opening pair, not as independently isolated runs. Comparisons may include wave-position and warm-state effects. Mean values pool the three controlled pairing conditions.



(a) Gold processing time for each controlled opening pair in the separate controlled experiment.



(b) Gold task durations when executed alongside another task or later in the same workflow.

Figure 4.14: Gold opening-pair experiment under two active task slots. The context panel compares first-wave overlap with non-overlapping second-wave execution, not with independent isolated runs.

region. These are comparisons across distinct execution contexts: the second-wave task runs later in the same workflow and is not an independently isolated cold-run control. The differences are consistent with contention among notebooks, but their percentages may also reflect execution position, caching, and preceding work. Top Items plus Sales by region has the highest observed pairing-study mean; this is the same opening pair used by the benchmark Static and HEFT configurations, but its 107.586-s study mean must not be substituted for either benchmark strategy mean.

Across the same five-repetition groups, median processing windows are 89.872 s (LTV and Top Items), 92.898 s (LTV and Region), and 95.194 s (Top Items and Region). The exact two-sided Mann–Whitney comparison of the lowest-mean and highest-mean pairing conditions gives $U = 6$ and an unadjusted $p = 0.2222$. The observed mean difference is sensitive to two long Top Items–Region runs; this exploratory test does not establish a stable pairing advantage.

4.6.1 Interpreting Pair-Level Effects Across Sessions

The three opening-pair conditions are meaningful because all use the same Gold notebook implementations and two available Airflow task slots. Their medians nevertheless tell a more restrained story than their means alone. The Top Items plus Region condition has a mean of 107.586 s but a median of 95.194 s, with a 22.604-s standard deviation. Its unusually high mean reflects substantial right-tail variation in five observations. Customer LTV plus Top Items has the lowest observed mean (90.313 s) and the smallest standard deviation (1.551 s) of the three pairing conditions. These quantities identify useful candidate combinations for this specific benchmark; they do not establish a universally fastest pair.

A benchmark strategy's opening pair may coincide with one of the controlled conditions, but the resulting runtime distributions should not be pooled. The benchmark Gold treatments and the separate pairing study were recorded in different sessions, and their windows include context-dependent execution and transition intervals. Their common information concerns the identity of the initial task set. The pairing results should therefore be read as additional evidence about candidate-set sensitivity rather than as an alternative estimate of a Static, HEFT or XGBoost benchmark mean.

4.6.2 Run-Level Variation in the Gold Pairing Study

The opening-pair means in Table 4.10 are accompanied by noticeably different dispersion. Table 4.12 shows the five recorded windows in each condition. The repetition labels order observations within a condition; they are not used as matched statistical blocks across the three treatments.

Customer LTV plus Top Items ranges from 88.857 to 92.510 seconds, a narrow spread over five observations. Customer LTV plus Region ranges from 86.492 to 105.102 seconds. Top Items plus Region ranges from 88.808 to 141.193 seconds and includes two long windows above

Table 4.12: Individual Gold opening-pair processing windows (seconds).

Repetition	LTV + Top Items	LTV + Region	Top Items + Region
1	92.510	91.249	120.600
2	91.266	86.492	141.193
3	89.060	94.421	95.194
4	88.857	105.102	88.808
5	89.872	92.898	92.134

120 seconds. The last condition's mean of 107.586 seconds is therefore substantially larger than its median of 95.194 seconds. This does not make the long observations invalid; it shows that a mean alone would conceal the instability relevant to a production latency objective.

The three conditions differ by which two computational notebooks occupy the opening wave under the same two-slot limit. However, the condition changes not only the identity of each concurrent companion but also which task executes later, after a first-wave completion. A comparison of overall processing windows is therefore a valid description of the complete configured opening-pair treatment. A comparison of one task's first-wave and second-wave durations, in contrast, is not a clean intervention on resource contention alone: position in the run and any resulting cache state are different. The two questions should not be conflated. The table of processing windows addresses the scheduling decision; the contextual task-duration table offers a plausible, though incomplete, explanation of its timing consequences.

The practical finding from this small paired-task catalogue is therefore conditional. Customer LTV plus Top Items has both the lowest observed mean and low dispersion in the separate Gold pairing study. The particularly variable Top Items plus Region condition merits further measurement before it is judged consistently inferior. Larger, interleaved samples and direct resource-state measurements would be needed to determine whether the right tail reflects reproducible companion-specific interference, execution position or transient cloud conditions. The 17.273-s mean difference is influenced by two long Top Items plus Region runs. The medians span only 5.322 s, and the exact comparison ($p = 0.2222$) does not support a reliable difference between pairing conditions.

4.7 Gold Concurrency-Sensitivity Experiment

Table 4.13 compares the evaluated limits of one, two and three active tasks. Three slots produced the lowest mean processing window, 56.144 s. This is an observed 54% reduction relative to one slot and a 28% reduction relative to two slots. The two-slot condition had the lowest measured coefficient of variation (1.79%), while one slot produced the highest mean window and relative variation. These are observed comparisons within the tested three-task DAG and fixed cluster, not a globally optimal concurrency recommendation.

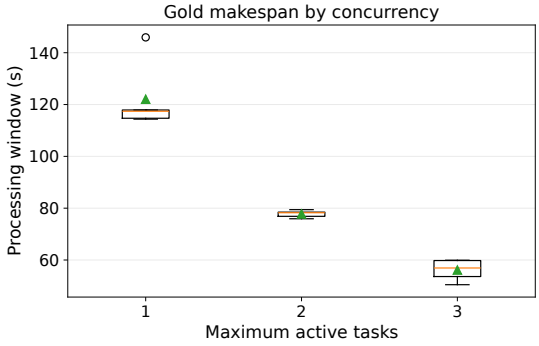
Summed task duration moves in the opposite direction from elapsed window. Mean task-

Table 4.13: Gold concurrency-sensitivity results under a fixed task-priority order.

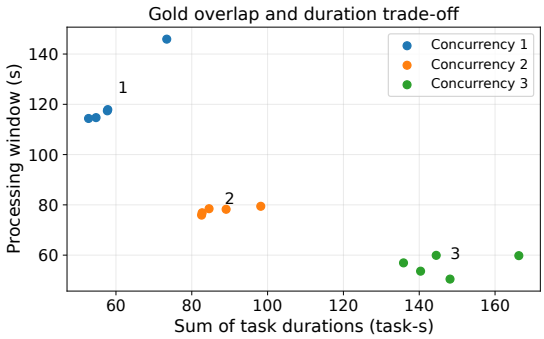
Slots	n	Mean window (s)	Median (s)	SD (s)	CV (%)	Mean task-seconds	Task-seconds/ window
1	5	122.061	117.410	13.432	11.00	59.296	0.485
2	5	77.785	78.243	1.391	1.79	87.427	1.123
3	5	56.144	56.921	4.085	7.28	147.017	2.626

Note: The fixed order was Customer LTV, Top Items, and Sales by region. Task-seconds/window summarises the average number of active task intervals over the measured window. It does not measure CPU utilisation, memory usage or energy consumption.

seconds increase from 59.296 under one slot to 87.427 under two slots and 147.017 under three slots. Relative to the single-slot condition, three slots raise summed task durations by approximately 148%. The processing window nevertheless falls because more of those duration intervals occur simultaneously. Longer task durations are compatible with execution interference, but task-seconds do not measure CPU-seconds, energy, memory use, or direct resource consumption.



(a) Gold processing time under one, two and three concurrent task slots in the controlled concurrency study.



(b) Gold concurrency study: elapsed processing time and the sum of individual task durations.

Figure 4.15: Gold concurrency sensitivity under a fixed priority order.

Figure 4.15 illustrates the elapsed-time and task-duration trade-off. The task-level timestamps also reveal transition gaps and differences in overlap. With one slot, the mean gap between the first task ending and the second starting is 33.11 s; the corresponding second-to-third gap is 29.66 s, totalling 62.77 s. With two slots, the third task starts on average 33.92 s after the earlier of the first-wave completions and 20.49 s after the later completion. These values are measured inter-task intervals; the experiment does not partition them into Airflow polling, Databricks admission, cluster state or other individual sources of delay.

The tested slot limits show different trade-offs between elapsed time and summed task durations. One slot has the smallest summed task duration, two slots the lowest observed runtime variability, and three slots the smallest mean wall-clock window. The ratio of summed task duration to elapsed window can be interpreted as mean task overlap during the measured interval,

not as resource efficiency. A production choice would require a specified objective and direct resource measurements that are not available in this experiment.

Table 4.14: Magnitude and interpretation of the controlled mechanism effects.

Mechanism	Controlled contrast	Observed effect	Interpretation
Bronze release timing	item early versus delayed	Delayed mean window +30.009 s (+24.36%)	Later root completion postpones broad downstream eligibility
Gold pair identity	Lowest- versus highest-mean pair	Mean gap 17.273 s; median gap 5.322 s	Exploratory contrast; unadjusted exact $p = 0.2222$; no reliable difference
Gold task context	Overlapping first wave vs. later non-overlap	Mean slowdown 52.57–126.16%	Contextual duration differences can include wave position
Gold concurrency	Three versus one active task	Window −54.00%; task-seconds +147.94%	Additional overlap reduces the observed elapsed window

Table 4.14 summarises the measured contrasts without treating all of them as equally strong causal evidence. The Bronze intervention deliberately changes when a prerequisite unlocks later tasks. The Gold pairing experiment changes the opening set at fixed capacity but compares task contexts with different wave positions. The concurrency experiment alters permitted overlap under a fixed priority order. Together, these cases show why a task-wise ranking must be interpreted through dependencies, available slots and realised co-execution rather than as a direct prediction of the layer window.

4.7.1 Why Task-Seconds and Elapsed Time Diverge

The observed ratio of summed task durations to the layer window also gives a useful interpretation of execution overlap. For a run, let $D = \sum_i d_i$ denote the sum of elapsed task durations and W the processing window. Then D/W is the mean number of tasks active over that window, under the recorded start/end intervals. The descriptive mean ratios in the concurrency results are 0.485, 1.123 and 2.626 at slot limits one, two and three, respectively. Values below one are possible because W also includes intervals when no notebook task is running. Values above one indicate overlapping task execution; they do not indicate processor utilisation above 100% or a calibrated quantity of consumed computing resources.

This distinction explains the apparent trade-off. Three admitted Gold tasks can take longer individually because of shared execution conditions while still finishing the layer sooner through overlapping work and fewer later waves. In contrast, a single-slot schedule eliminates task overlap but experiences the recorded submission and transition gaps. The result is evidence about latency and realised overlap on one single-node cluster. Choosing a concurrency limit that also accounts for resource consumption would require measurements absent from the present dataset.

4.7.2 Connecting Concurrency Limits to the Measured Timeline

The concurrency results can be understood without equating a task-second to a hardware resource-second. If $N(t)$ is the number of recorded notebook tasks whose timed regions are active at time t, summing all task durations is equivalent to integrating that count over the processing window:

$$\sum_i T_i = \int_{t_{\min}}^{t_{\max}} N(t)\, dt. \tag{4.1}$$

This identity follows from adding the lengths of the recorded task intervals. Dividing by the window length gives average observed task overlap, including idle intervals when no timed task is active. It does not give core occupancy or machine utilisation because several active notebooks share the same host and their respective executors may use resources differently.

The one-slot condition gives a clear example. Only one notebook task can occupy a slot at any time, so the sum of task durations cannot exceed the layer window. The reported mean task sum of 59.296 seconds is substantially smaller than the mean window of 122.061 seconds. The remaining interval comprises periods outside the individual timed task regions, including the transitions between successive notebook submissions. The observed mean inter-wave gaps of approximately 33.11 and 29.66 seconds explain why serial admission is slow even when individual tasks avoid direct overlap. These gaps are measured at the boundary of the timed regions; allocating all of them to one specific Airflow or Databricks component would require instrumentation unavailable in the experiment.

With two slots, two tasks overlap during the opening wave, followed by the remaining aggregation. Its observed 87.427 task-seconds are larger than the 77.785-second mean window, consistent with part of the recorded task time overlapping. With three slots, all three aggregations can be admitted initially; the mean sum grows to 147.017 task-seconds while the mean window falls to 56.144 seconds. The processing window can be shorter than the summed durations because tasks run at the same time.

In particular, the 59.296 s one-slot task-duration sum is close in magnitude to the 56.144 s three-slot processing window; meanwhile, the two measured gaps total 62.77 s, approximately 51% of the one-slot mean window. This shows the importance of task hand-offs and overlap, without separating their causal shares of the improvement or implying additional hardware resources. The larger sum is consistent with slower task execution in a shared environment, but no independent measurement separates CPU saturation, memory pressure, cache effects and other possible causes.

4.8 Run-Order and Session Diagnostics

The Azure benchmark runs were executed sequentially, with the strategy order rotated across rounds but without full randomisation or matching on cloud state. Its recorded processing-window variations include a particularly fast Gold XGBoost Round 1 and a slow Round 5. A host restart

near the last Gold run creates another possible difference in transient compute state. The individual-run measurements in Table 4.4 and the leave-one-round-out sensitivity in Table 4.5 show how the extremes affect the summary; neither warrants removing an otherwise successful observation. The exploratory studies retain their own session-level trend diagnostics.

4.9 Output Consistency Across Experiments

The controlled mechanism studies showed consistent aggregate output counts within their experimental conditions, although the complete table-level checks described in Section 4.4.7 were performed only for the strategy benchmark. This difference in validation coverage is considered when interpreting the controlled results.

4.10 Research-Question Results Summary

The results relevant to each research question are consolidated in Table 4.15. The table reports what the observed comparisons establish, while retaining the distinction between an effective scheduling change and a numerical difference between equivalent-schedule batches.

Table 4.15: Summary of the research-question answers.

RQ	Evidence-based answer
RQ1	HEFT was slower than Static on average in Bronze. Its lower Silver and Gold mean times occurred without a meaningful difference in the task execution order, so those differences cannot be attributed to task prioritisation alone.
RQ2	The cloud study used priorities predicted by the saved XGBoost model. XGBoost had the lowest Bronze mean, but selected the same Silver first-wave task pair and subsequent sequence as the alternatives. Its small Gold mean advantage over HEFT varied considerably between repetitions. The model was trained on designed priority scores, not demonstrated optimal schedules.
RQ3	The delayed-release condition showed a longer mean Bronze processing window in the study. Gold opening pairs had descriptive mean differences, but their median gap was only 5.322 s and an exact test ($p = 0.2222$) did not establish a reliable pairing difference. The available measurements do not isolate CPU or memory contention.
RQ4	Some scheduling patterns were similar locally and in Azure, but the measured time differences did not consistently transfer. Higher Gold concurrency reduced elapsed time while increasing the sum of task runtimes.

Chapter 5

Discussion

5.1 Interpreting the Azure Benchmark

The Azure Databricks benchmark compares Static, HEFT-based and model-derived XGBoost priorities under the same layer-specific task dependencies and concurrency limit. The relevant question is whether these priorities change which tasks begin first and whether any resulting timing differences are robust across repetitions.

Three distinctions govern interpretation: the priority rule, the effective task-start order and the resulting layer processing time. Different numerical priorities may lead to the same admission decisions. When this occurs, a difference in mean processing time cannot be attributed to a different ordering decision alone. Correctness of the materialised outputs is a separate requirement, addressed in the Results chapter.

The three layers illustrate different limitations on scheduling freedom. Bronze contains prerequisites with different downstream-release consequences. Silver has three cleaning tasks converging into a final join, with the same first-wave task pair and subsequent dependency sequence for all studied policies. Gold has three independent aggregations competing for two available slots, so the opening pair and execution context can matter. Pooling the layers into a single scheduler ranking would obscure these structural differences.

Cross-session sensitivity. The earlier cloud benchmark also favoured XGBoost in Bronze, but its lowest Silver and Gold means belonged to Static and HEFT respectively, not HEFT and XGBoost as in the repeated benchmark. These historical windows use different recording and execution conditions, so they cannot be pooled with the final sample or interpreted as a randomised replication. In particular, the much longer Gold processing windows of the repeated benchmark are not explained by the surviving records; the cloud workspace was discontinued before further instrumentation could resolve this. The observed rankings therefore remain conditional on the measurement and execution session rather than establishing stable policy performance.

5.2 RQ1: HEFT Compared with Static Scheduling

In the Azure benchmark, the adapted HEFT policy is not consistently better than the manually weighted Static baseline. HEFT has a larger mean Bronze processing window than Static (246.057 versus 226.697 s), whereas in Silver and Gold the recorded HEFT means are lower. The Silver difference cannot be assigned to a better priority-selected schedule, because the same first two cleaners, remaining cleaner, and prerequisite-constrained join are admitted under both strategies. In Gold, HEFT and Static again select the same first-wave pair, despite different numerical priorities. Their mean-duration contrast thus lacks the scheduling-order intervention that a causal "HEFT improvement" claim would require.

5.2.1 Bronze Dependency Release

HEFT's upward-rank calculation depends on estimated remaining path costs. The evaluated HEFT ordering favours `ingest_customer` and `ingest_date_dim` before `ingest_item`; the Azure Static rule prioritises `ingest_date_dim` and `ingest_item`. Because `ingest_item` enables multiple downstream sales ingestions, delay can postpone the expansion of the ready-task set. In the controlled Bronze intervention, the delayed-root condition increased mean processing time relative to the early-unlock condition. That study changes the root ordering deliberately while preserving graph dependencies, making it evidence for the proposed mechanism under its own session conditions. It does not quantify how much of the benchmark HEFT contrast was caused specifically by the root choice.

5.2.2 The Limits of Upward Rank in a Single-Cluster Admission Policy

This implementation uses HEFT's upward rank to assign Airflow priorities but does not implement the original processor-assignment stage (Topcuoglu et al., 2002). All submitted notebooks share one Databricks cluster; upstream ranks are not assignments to alternative heterogeneous processors. Thus the empirical question is whether a fixed HEFT ranking chooses a more useful next task under the tested graph and capacity, not whether the full HEFT scheduling algorithm is superior in general. Where only one feasible ready task remains, or distinct ranks induce the same top two tasks, additional rank sophistication cannot change the observed task order.

5.3 RQ2: XGBoost-Based Scheduling

The XGBoost priorities came from predictions calculated before the benchmark and were used consistently by Airflow. This is an offline scoring policy, not an adaptive scheduler: it did not update priorities in response to the ready-task queue, Spark resource use or concurrent workloads.

5.3.1 Observed Layer Differences

XGBoost has the lowest observed sample mean in Bronze: 187.639 s, compared with 226.697 s for Static and 246.057 s for HEFT. It selects the early-unlocking `item/date_dim` opening pair, although Static also does so. In all five repetitions per policy, the recorded later ordering of `ingest_web_sales` and `ingest_catalog_sales` was reversed between Static and XGBoost (Table 4.7). This is an observed admission difference after their common opening pair, but its contribution to the mean contrast remains unidentified. The Azure benchmark was not a randomised, matched-state intervention isolating each alternative schedule. The measured difference therefore describes this sample rather than an isolated causal effect of XGBoost scheduling.

In Silver, XGBoost averages 154.377 s and HEFT 152.387 s against Static at 165.323 s. All three policies admit the same computational tasks in the same broad order. The observed mean differences therefore reflect variability between runs with an effectively unchanged scheduling order; they do not demonstrate a benefit caused by XGBoost or HEFT priorities.

In Gold, the XGBoost mean is 346.976 s compared with HEFT at 353.782 s and Static at 399.317 s. The XGBoost–HEFT difference is about 1.92%; XGBoost's own five observations include a 157.748-s first round and a 456.820-s fifth round. In a sensitivity calculation excluding Round 1 *from both* policies, the XGBoost four-run mean becomes 394.283 s versus HEFT's 354.492 s, reversing their original ordering. Round 1 was a completed valid run and is retained in the main analysis. The change of ordering under omission demonstrates instability of the apparent Gold lead rather than an acceptable reason to discard an inconvenient observation.

5.3.2 What the XGBoost Model Has and Has Not Learned

The regression target was an engineered priority formula incorporating normalised rank, duration estimates, fan-out and a designed short-unlock preference. The 130-row training table consists of repeated observations of only 13 distinct task identities. A row-wise train/test partition therefore contains familiar identities on both sides; near-perfect prediction of the engineered scores is not an independent test of performance on unseen tasks or of learned makespan optimality.

As the target is a deterministic formula constructed from quantities represented among the model inputs, high within-task prediction accuracy is an anticipated property of this design, not evidence of discovering better scheduling decisions from runtime outcomes. A grouped-task holdout, DAG-level holdout or training objective based on observed scheduling rewards would support substantially different generalisation claims.

The model assigns each task a fixed score without accounting for which other task runs at the same time. It does not receive the identity of the task that will run concurrently with it, live cluster capacity or cache state. Its output therefore does not predict the completion time of a particular task pair. The separate Gold mechanism studies show that observed durations depend on execution context, while the Gold benchmark outliers show why a fixed single-task ranking

should not be described as a universally optimal co-scheduling policy. These are limitations of the training objective and feature representation, not proof that all ML scheduling methods fail (Mao et al., 2019).

5.4 RQ3: Task Dependencies and Concurrent Execution

The controlled Bronze and Gold studies deliberately vary task admission decisions using predefined priorities, independently of the XGBoost model. Their purpose is to examine dependency release, opening-pair selection and permitted concurrency under specified conditions. The resulting effects therefore concern those scheduling choices, not the predictive performance of a learned model.

5.4.1 Bronze Unlocking

The Bronze controlled study compares early versus delayed admission of a high-release prerequisite. It observes a mean processing-window increase from 123.19 to 153.20 s in the delayed condition. That within-study contrast supports the practical hypothesis that completing a short, high-fan-out root may create more useful parallelism than choosing a task solely by its own estimated duration. These processing windows should not be combined numerically with those from the strategy benchmark because they were measured in different experimental sessions.

5.4.2 Gold Pairing and Concurrency

The controlled Gold opening-pair study intentionally starts each of the three possible pairs first, while retaining the third aggregation for the second wave. The pair means range from about 90.31 to 107.59 s, but the medians differ by only 5.322 s and the exploratory exact comparison gives $p = 0.2222$. This does not demonstrate a reliable pairing advantage. The descriptive differences may involve execution-context effects; later-wave observations also differ in placement and warm-state history. Neither direct CPU nor memory interference was separately measured. The finding justifies exploring companion-aware task-cost models; it does not establish a measured hardware interference coefficient.

When Gold concurrency is varied while holding the fixed priority order constant, mean processing windows fall from approximately 122.06 s (one slot) to 77.79 s (two slots) and 56.15 s (three slots). Summed task duration rises under higher concurrency. Those outcomes need not conflict: simultaneous execution can reduce total wall-clock completion while increasing the elapsed time accumulated across concurrently executing tasks. Summed task-seconds do not directly measure CPU utilisation or energy consumption. The pairing and concurrency studies were conducted in separate sessions and are not pooled with the Azure benchmark.

5.5 RQ4: Local and Azure Results

The local workload represents analogous DAG bottlenecks and a two-task capacity limit, but uses simulated Python tasks rather than Spark notebooks. Its Static baseline retains Airflow's default weights, unlike the manually weighted Azure Static policy, and its HEFT ranks can select different tasks. Accordingly, the Bronze and Silver results can inform discussion of dependency and capacity mechanisms, but they do not constitute a matched comparison of identical priority policies. The observed within-environment rankings and absolute times are not directly transferable. The separate Gold pairing and concurrency experiments further demonstrate that changing the co-execution context changes observed completion time, but do not estimate the Azure benchmark runtime distribution.

This comparison is informative about the limits of simulation, but it does not isolate an environment effect: policy inputs and some rankings changed alongside the execution system. Local fixed-cost tasks cannot represent shared Spark resource conditions, where durations can depend on execution companions, cache state, orchestration gaps and platform activity. The design therefore supports a qualitative discussion of scheduling mechanisms, not an estimate of the effect of moving an identical policy from local execution to Azure.

5.6 Identification, Uncertainty, and Reproducibility

Alternative explanations for the principal cloud patterns

The layer results can be interpreted more carefully by separating the scheduling explanation from other explanations that produce similar observed windows. Such distinctions matter most where the effect is small, the initial task set is unchanged, or one unusually fast or slow repetition influences the mean. They do not require a new experiment to be stated clearly, but resolving them would require evidence beyond the existing run records.

Bronze: earlier release versus differences later in the schedule. The HEFT-based opening pair delays `item` relative to the Static and XGBoost pairs. The dependency graph predicts that this delay can postpone sales-task eligibility, and the separate Bronze intervention demonstrates a longer window when the same prerequisite is deliberately delayed. This is the clearest agreement between a visible scheduling change and an experiment aimed at its mechanism. Yet Bronze XGBoost is faster than Static on average even though both admit the two common prerequisites first. That additional difference cannot be explained by the initial pair alone. Later sales-task admission, variations in the duration of individual notebook executions and unmeasured session conditions are all possible contributors. The task timelines help identify later ordering choices, but the benchmark does not isolate one of those contributors as the unique cause of the observed mean difference.

Silver: an unchanged choice with different elapsed times. The three strategies reach the same first-wave pair, followed by the remaining cleaner and the join. The upstream dependencies limit what a new rank ordering can achieve once the same initial capacity is allocated. The fact that their observed Azure means differ is therefore not contradictory: completion time also depends on how long the cleaners and join execute, and on the gaps between recorded job intervals. A faster sample mean under HEFT does not show that HEFT's priority calculation selected a superior schedule in Silver. Conversely, the absence of an effective admission difference in this small graph does not demonstrate that prioritisation is irrelevant to every possible Silver workflow. A graph with more independent ready tasks or a different capacity limit might create additional choices, but that configuration is outside the present study.

Gold: a changed pair under uncertain execution conditions. XGBoost selects a different opening pair from Static and HEFT, so a scheduling explanation is plausible in principle. The controlled pairing study shows descriptive differences among Gold pairs, but its medians are close and the exact comparison is not statistically reliable ($p = 0.2222$). The observations motivate, rather than establish, the importance of that admission decision. The two studies also have markedly different absolute windows and session histories. The XGBoost–HEFT mean difference in the main benchmark is small relative to the variation among its five repetitions; removing one common round reverses their ordering. These observations prevent the controlled pair ranking from being used as proof that the benchmark's small five-run XGBoost lead was caused by its opening choice.

The controlled study tests which tasks are admitted together but does not establish a reliable ranking among pairs. The strategy benchmark describes the total outcomes of particular priority configurations under its recorded cloud conditions.

What would distinguish these explanations. An improved comparison would first ensure that each candidate policy actually changes the relevant admission choice. It would then alternate or randomise these choices across independently prepared sessions, retaining the timestamps and task identities needed to reconstruct the execution waves. For Bronze, the additional observation is the time at which each downstream task becomes eligible as opposed to when the remote notebook starts. For Silver, separately examining task execution intervals and inter-wave gaps would clarify the sources of the descriptive variation without attributing them to unchanged priorities. For Gold, the missing evidence is repeated pair-specific performance under controlled prior state, together with resource measurements capable of distinguishing contention from caching or other effects. These are proposals for strengthening attribution, not measurements performed in the thesis.

5.6.1 Internal and Construct Validity

The strategy treatments were executed in a sequential experimental session, not randomly assigned to identical cluster states. Cache warming, data materialisation, background activity, run order and a host restart near the end of Gold execution are potential confounders. Five repetitions per strategy–layer cell are inadequate for confident small-effect inference in a variable remote platform. The bootstrap intervals and leave-one-round-out checks describe how the observed sample behaves under resampling or omission; they do not correct treatment assignment bias or establish causality. The full sample, including Gold extremes, is preserved.

Two timing constructs are also relevant. Remote processing window measures provider-recorded task execution from first start to last completion. Airflow DAG wall-clock duration includes a larger orchestration interval. The Spark notebook's own timed transformation region excludes some validation queries and is narrower still. These quantities must be labelled separately. High concurrency may shorten the first measure while the second includes additional submission, state-transition or provider delays.

5.6.2 Output Validity and Lineage

The output checks described in Section 4.4.7 support consistency of the materialised results across scheduling strategies. The small Gold aggregation differences were within the stated numerical tolerance, not exactly equal at the binary level. This supports the validity of the output comparisons without providing independent ground truth for every record.

For scheduling comparisons, the measured task order and execution intervals take precedence over supplementary metrics collected through differing procedures. Sequential cloud runs also leave open the possibility that changing execution conditions contributed to the observed differences.

All policies use the same prepared inputs within each layer, which limits variation due to upstream data. The reported processing windows therefore describe separate layer-level comparisons rather than a measured end-to-end Bronze–Silver–Gold execution. Adding the three layer means would not yield an observed pipeline makespan.

5.6.3 External and Conclusion Validity

The cloud configuration uses one Databricks runtime, one single-node VM type, a fixed TPC-DS SF1 subset, three small DAGs and nominal task concurrency two. The results do not estimate general performance on other workloads, multi-node or heterogeneous clusters, larger datasets, different autoscaling regimes, or unseen task catalogues. The original HEFT formulation also includes processor assignment not used here. The XGBoost feature and target design does not include online workload adaptation. No independent resource-usage profile was obtained. These restrictions narrow the contribution to a traceable scheduling *case study*, not a production performance guarantee.

5.7 What the Observed Contrasts Can Establish

5.7.1 Policy Labels, Realised Decisions, and Counterfactuals

A comparison between strategy-labelled batches estimates an effect of *priority choice* only when the treatment changes the ready-task decision and relevant conditions are sufficiently comparable. This distinction is most visible in the benchmark Gold data. Because Static and HEFT admit the same two aggregations first under the same two-slot limit, their observed mean difference cannot estimate the effect of substituting one opening pair for another. It instead describes variability between two execution batches carrying different priority labels. Similarly, a lower observed Bronze XGBoost-derived mean than Static does not demonstrate a learned-model benefit if both configurations expose the same high-unlock roots to the first available slots.

The mechanism studies answer narrower questions. The Bronze early-versus-delayed contrast deliberately changes the release order and therefore supports an ordering explanation for that intervention. The Gold pair experiment changes the first admitted task set, so it directly compares opening-pair configurations *within that batch*. It does not, however, randomise away differences between the overlapping opening wave and the later non-overlapping task, nor does it provide a standalone isolated-task reference. The concurrency experiment intervenes on the allowed number of active notebooks, but its total elapsed-time change combines overlap, task duration, and transition gaps. Treating these contrasts according to their actual intervention avoids attaching a causal interpretation to every tabulated mean difference.

5.7.2 Uncertainty and Robustness Across Small Cloud Samples

Five Azure repetitions per cell provide useful operational observations but weak support for finely ordered strategy rankings. The dispersion, medians, leave-one-round-out summaries, and individual-run plots are therefore as important as the cell means. None of these diagnostics turns a fixed-order experiment into a randomised trial. A mean difference that persists after removing one observation is less sensitive to that observation, but the result may still reflect batch state or a common unobserved condition.

The findings are stronger when the dependency graphs, observed task starts, processing times and controlled experiments are considered together. Graph inspection establishes which root or opening-pair choices are feasible; timestamps show which choices Airflow realised; run summaries describe their elapsed-time outcomes; and controlled mechanism contrasts provide targeted checks of the proposed explanations. These sources of evidence should be interpreted together. The strongest conclusion is not that one algorithm wins on average across every configuration, but that scheduling freedom and performance sensitivity differ substantially by graph structure and execution environment.

5.7.3 A Counterfactual Standard for Attributing Scheduling Improvements

A strategy comparison is meaningful only if the policy changes a decision that can affect the outcome. In the present experiment, that decision is the admission of an eligible Airflow task under a capacity limit. It is helpful to distinguish three conceptual counterfactual questions. The first asks whether a different numerical weight vector would lead Airflow to launch different tasks, given the same ready set and available slots. The second asks whether that changed admission would alter completion time, given comparable task durations and platform state. The third asks whether the resulting improvement would persist when the workload or cluster changed. The archived priorities and observed start order inform the first question; the experiment provides evidence relevant to the second; and the current single-cluster sample offers only limited evidence for the third.

This distinction is particularly important when interpreting the XGBoost results. Model-fit statistics measure reconstruction of an engineered target. For the evaluated tasks, the saved model and engineered target generate the same within-layer ranking, although the benchmark uses rounded model predictions. Even if the model reproduced every target score exactly, the inference would be about score fidelity, not a runtime gain uniquely attributable to learning. Demonstrating an incremental ML scheduling benefit would require contrasting model-generated and appropriately designed non-ML policies that induce genuinely different task-selection choices, while controlling the experimental sessions. The reported comparisons establish the behaviour of the implemented order, not the additional benefit that a separately controlled learning-versus-rule comparison might show.

5.8 Priority, Concurrency, and the Deployment Decision

The results support a joint decision model. Let P denote a priority ranking, k the concurrency limit, and $C(S, k)$ the context-dependent completion cost of a set S admitted under capacity k. A priority-only policy assumes that tasks have stable costs and that selecting the top k tasks independently is sufficient. The Gold findings illustrate why this assumption may be unreliable: task durations vary with the set of concurrently admitted tasks.

A practical policy should evaluate the candidate set and the number of slots together. Under a latency objective, it should estimate the end time of the slowest task in the admitted set plus any remaining waves and transition costs. Under a resource-pressure objective, it should use measured CPU, memory, and I/O utilisation where available; summed task duration can only serve as an imperfect supporting indicator. Under a stability objective, it may prefer a slightly slower but lower-variance condition.

This formulation also clarifies why the three evaluated strategies are only part of the decision space. Static, HEFT, and XGBoost determine P. The concurrency experiment changes k. The observed result is generated by their interaction with the layer graph and cluster. A production scheduler should not optimise these settings independently.

5.8.1 Practical Decision Framework

Table 5.1 summarises implications as *decision diagnostics*, not unconditional recommendations. The central principle is to choose scheduling sophistication only when it has a measurable opportunity to change the admissible task set and when the workload objective and telemetry justify it. A high-release Bronze root may merit explicit priority protection. In Silver, a simpler fixed order can be sufficient when alternative strategies produce the same first-wave pair and subsequent task sequence. For independent Gold aggregations, measuring pair-specific execution context and the effect of slot count may be more informative than improving isolated task-score accuracy.

Table 5.1: Layer-aware deployment framework derived from the experiments.

Observed graph condition	Diagnostic evidence	Recommended decision logic	Reason
Broad fan-out from short roots	Downstream tasks wait for one or two dimensions	Unlock-aware Static rule or heuristic with downstream-release features	Completing the root changes the future ready set and execution waves
Convergence into one dominant task	Alternative priorities produce the same start order	Retain a simple Static order and monitor the bottleneck	Model complexity cannot improve an invariant effective schedule
Independent expensive tasks under limited slots	Task durations vary across co-execution contexts	Joint pair and concurrency prediction	Isolated duration is not a sufficient cost estimate
High orchestration delay between short tasks	Long gaps occur after slots become free	Increase safe concurrency or reduce submission overhead	Sequential efficiency can be dominated by hand-off latency
Uncertain or changing workloads	Rankings or interactions drift over time	Collect context-aware telemetry before retraining or adapting online	Sophistication is useful only when the model observes the governing mechanism

A stronger subsequent experiment would interleave or randomise independently prepared cloud conditions, repeat them across sessions, hold out task identities or DAGs when evaluating ML generalisation, and collect Spark task metrics and direct resource telemetry. A learned policy could then be evaluated on predictions of *eligible task sets* or paired completion cost rather than reconstruction of a hand-designed single-task target. Direct utilisation measurements would be required before making resource-optimisation recommendations.

5.9 Decision-Science Interpretation and Transfer

The results can be framed as a decision problem under uncertain, context-dependent processing times. The decision variables are the priority order and concurrency limit. The state contains the current ready set, graph structure, historical task information, and cluster condition. The

primary outcome is layer makespan, while supporting outcomes include variability and aggregate task-seconds. The scheduler does not observe the complete future cost with certainty before execution.

Under this framing, Static scheduling is a policy based on expert prior knowledge. HEFT is a structural model that converts duration and successors into a deterministic score. XGBoost is a statistical approximation to an engineered scoring function. The mechanism studies reveal which state variables are missing from each policy. The evaluated HEFT ranking does not explicitly count the number of tasks released by a prerequisite. The XGBoost features do not encode concurrent companion identity or live cluster conditions. The practical choice is therefore whether a more complex priority policy provides information that changes a useful scheduling decision.

The Gold concurrency results illustrate competing performance objectives, not a formally established Pareto frontier. Concurrency one has the lowest aggregate task-seconds but the highest latency. Concurrency two has the lowest relative variability. Concurrency three has the lowest latency but the greatest aggregate task duration. None dominates on all outcomes. A production owner must assign value to freshness, stability, and resource use before choosing a setting. This is preferable to deriving one universal recommendation from the fastest observed mean.

5.9.1 Choosing a Policy under Measurement Uncertainty

One practical decision rule is to preserve a manually configured Static baseline when the alternatives generate the same effective schedule. In that case, a more elaborate scoring computation does not create an identifiable priority-order benefit for the evaluated DAG and capacity. This does not mean that Static is globally optimal; it means that changing the score representation without changing the selected task set has no demonstrated ordering value under those conditions. In Silver, for example, the common join limits scheduling freedom and the three evaluated strategies share the same initial admission. Engineering effort may be better directed toward the dominant transformation or the dependency structure, though either optimisation would require separate measurement before any improvement claim.

When alternatives do change the opening set, the decision should consider both the mean and the risk of unstable completion. The Gold pairing study illustrates why. Customer LTV plus Top Items is associated with low mean and low dispersion in that particular experimental batch; Top Items plus Region has a far larger range, including a very slow observation. If a production layer has a strict freshness deadline, an uncertain tail may matter more than a small difference in mean. No service-level objective or deadline penalty was defined in this thesis, so it would be inappropriate to assign a formal weight to these tails. The contribution is to identify which extra objective and observation an operator would need before translating the measured windows into a deployment choice.

For a learned priority policy, deployment also requires a recorded training target, feature

definition, model version and corresponding task weights. The recorded model and priority checks cover that traceability for the studied tasks; evaluating new tasks or changing cloud conditions would require a separate validation design.

5.9.2 Generalisation Scenarios and Testable Propositions

The observations lead to four hypotheses that can be tested on different task graphs, data scales, and cluster configurations. They are not numerical performance guarantees for other Medallion pipelines.

Fan-out proposition: when a short ready task releases enough downstream work to increase slot utilisation, an unlock-aware order should outperform a rank based primarily on isolated duration and remaining chain length.

Convergence proposition: when alternative rankings produce the same predecessor completion order before one dominant barrier, additional priority sophistication should have negligible causal effect on makespan.

Interference proposition: when ready tasks share a bottleneck and their durations are context dependent, a task-wise ranking may be inferior to a candidate-set model when pair slowdowns are sufficiently heterogeneous.

Overlap proposition: increasing concurrency should reduce makespan only while the benefit from overlap and fewer hand-offs exceeds the additional slowdown caused by contention.

These propositions indicate how the conclusions may change under different infrastructure. A larger multi-node cluster might reduce some shared-node pressure by distributing work; the best concurrency could nevertheless change in either direction depending on shuffle, placement, and admission overhead. It could also introduce network shuffle and placement effects that create new interactions. A larger data scale may increase task duration relative to submission latency, reducing the importance of orchestration gaps. It may also cause spills and make pair compatibility more important.

A different Airflow executor or a Databricks job-cluster model could change the transition cost. If every task launches a separate cluster, startup time could dominate and concurrency policies could perform differently. If tasks run inside one multi-task Databricks job, control-plane hand-offs could be smaller. The current conclusions transfer most safely at the level of mechanisms and required measurements.

5.10 Analytical Expectations and Observed Findings

The analytical expectations in Chapter 1 concerned path-based ranking, prerequisite release, limited scheduling freedom in Silver and the possible role of Gold pairing and concurrency. In

Bronze, HEFT has the largest observed cloud mean, while the separate controlled experiment supports the relevance of early prerequisite release. Silver's smallest mean belongs to HEFT, but all policies select the same first-wave pair and subsequent task-admission pattern; this does not establish a scheduling advantage. Gold's smallest mean belongs to XGBoost, although its small lead over HEFT is sensitive to individual runs. The Gold pairing study provides descriptive, not statistically reliable, differences between opening pairs, whereas the concurrency study demonstrates substantial changes in overlap and processing windows. These comparisons assess the expectations in the evaluated workloads; they were not pre-registered hypotheses.

5.11 Engineering and Scientific Implications

The proposed layer-aware framework is conditional decision guidance, not an experimentally validated production scheduler. The table includes testable recommendations (such as companion-aware modelling and online responses to changing workloads) that were not evaluated in this thesis. Before selecting a policy, an operator should first inspect the DAG ready-set width and determine whether alternative priority scores actually change the tasks admitted under the current capacity limit. If the opening set and relevant downstream precedence choices are invariant, the simpler priority rule is more maintainable and no speedup can be ascribed to a different numerical weight alone.

For fan-out layers, the most useful additional diagnostic is *release value*: for each ready root, log which successors become eligible and whether its completion fills otherwise idle execution slots. For independent aggregation layers, the more informative unit is the *candidate set* rather than the task considered alone. Historical individual duration estimates should be accompanied by companion identity, admission gaps, cluster state, and independently measured reference durations when an interference claim is intended. Those measurements were not all available in the current experiment and should not be inferred from summed task-seconds.

A selective deployment path is consequently preferable to a general scheduler replacement. Preserve the manually specified Static order as an auditable fallback; use structural ranking when it demonstrably changes a constrained ready set; and consider an interaction-aware learned model only after collecting task-set labels under controlled execution. Model governance would then include task-identity holdouts, checks that the exported scores correspond to the model version under test, and monitoring of effective start orders rather than prediction error alone. The complexity of these steps is warranted only when the expected operational benefit exceeds model and instrumentation overhead.

5.11.1 Theoretical Contributions

Four distinctions summarise the conceptual contribution of this study.

First, it distinguishes configured priority from effective schedule. If competing strategies always admit the same eligible tasks in the same waves, their different priority scores do not

create a different ordering decision. Silver provides the empirical example.

Second, it distinguishes direct task criticality from release criticality. A short root can be critical because it creates future parallelism. Bronze provides the empirical example.

Third, it distinguishes single-task cost assumptions from context-dependent timing. Gold durations vary with concurrency and co-execution context, although the experiments do not isolate every hardware source of that variation.

Fourth, it distinguishes the sum of task durations from layer latency. Higher concurrency increases recorded task-seconds while shortening the observed processing window; the sum is not a direct measure of hardware consumption.

Together, these distinctions explain why observed execution order and concurrency are necessary for interpreting task-priority comparisons.

5.11.2 Evidence Needed for Stronger Follow-Up Studies

Follow-up experiments should establish whether the proposed scheduling mechanisms recur across independently prepared sessions. In Bronze, logging newly eligible successors and idle-slot intervals at task completion would distinguish early dependency release from faster individual execution. Interleaving policies across cloud sessions and using comparable measurement boundaries would reduce run-order and recording confounding.

For Gold, directly measuring each candidate pair under controlled execution position, cache preparation and cluster state would be necessary to identify companion-specific interference. Task duration, overlapping intervals, Spark CPU and memory metrics, shuffle or spill information, and submission gaps should be collected together. Together, these measurements would help distinguish stable pair-specific effects from transient conditions without inferring hardware causality from elapsed task-seconds alone.

A learned scheduler should be evaluated against observed layer-level rewards with task-identity or entire-DAG holdouts, depending on the intended generalisation claim. Time-separated tests would be needed for adaptive policies. Finally, direct utilisation measurements would be necessary before optimising resource consumption jointly with makespan; aggregate task-seconds are not a valid substitute.

5.11.3 Relation to Prior Work

The results are consistent with earlier ETL and DAG-scheduling research in showing that precedence and task prioritisation interact (Simitsis et al., 2005; Karagiannis et al., 2013). They also complement interference-aware cluster-scheduling literature by exposing context-dependent task timing at the Airflow admission layer, without identifying the hardware source of each slowdown (Delimitrou and Kozyrakis, 2013; Grandl et al., 2014). Finally, the distinction between fitting a proxy target and evaluating a deployed policy is material for learned scheduling research (Mao et al., 2019).

Chapter 6

Conclusion

6.1 Study Overview

Static, HEFT-based and XGBoost-based task priorities were compared across the Bronze, Silver and Gold layers of a Medallion Lakehouse ETL pipeline. The evaluation combined 90 local runs, 45 Azure Databricks runs and 40 separate controlled cloud runs. The XGBoost priorities were generated from a saved model before execution, and the cloud benchmark completed successfully.

Scheduling outcomes depended on task dependencies, execution capacity and the combination of tasks sharing that capacity. Different priority values did not always change the effective execution order, and timing differences under an unchanged order cannot be attributed to priority selection alone. The conclusions concern the measured layer processing windows in the evaluated workload and cloud environment.

6.2 Answers to the Research Questions

RQ1: HEFT Compared with Static

In the Azure benchmark, the adapted HEFT upward-rank priority policy did not dominate the manually weighted Static baseline across layers. In the Azure Bronze sample, the HEFT mean remote processing window (246.057 s) was longer than Static (226.697 s), alongside a root order that delayed a high-unlock prerequisite. In Silver, HEFT's lower mean (152.387 versus 165.323 s) occurred with the same first-wave pair and subsequent task sequence. In Gold, HEFT's lower mean (353.782 versus 399.317 s) also occurred with the same first-wave pair as Static. The latter mean differences cannot be claimed as consequences of changing the effective priority-selected schedule.

RQ2: XGBoost Compared with Static and HEFT

The Azure benchmark used integer priorities derived from the saved XGBoost model. XGBoost achieved the lowest observed Bronze mean (187.639 s), while HEFT had the lowest Silver mean

(152.387 s). In Gold, XGBoost's five-run mean (346.976 s) was slightly lower than HEFT's (353.782 s), but this result depended strongly on individual rounds and reversed in a common Round 1 exclusion sensitivity check. These patterns do not support a claim of consistent ML superiority. The model learned to reproduce a designed priority score for the tasks included in training, and its predictions were calculated before execution; it was not trained on measured optimal schedules and did not adapt during live executions.

RQ3: Task Dependencies and Concurrent Execution

The separate Bronze controlled study supports the importance of completing upstream tasks that release multiple downstream operations. Its delayed-unlock condition increased the observed mean processing window relative to early unlocking. The Gold opening-pair study found descriptive mean differences, but the pair medians were close and the exact comparison ($p = 0.2222$) did not support a reliable difference. Separately, increasing the permitted Gold concurrency from one to three slots shortened the recorded processing window. Thus dependency release and concurrency have stronger evidence in this study than a general Gold pairing advantage. The controlled studies do not isolate specific CPU, memory or caching interference mechanisms, and their measured durations should not be pooled with the Azure benchmark results.

RQ4: Local and Cloud Results and Concurrency

The local results provide qualitative context for dependency and capacity constraints, but local Static relied on Airflow defaults while Azure Static used manual priorities, and local and Azure HEFT rankings were not identical. Consequently, differences in within-environment performance cannot be attributed solely to moving from simulated tasks to Spark and Delta execution; nor can local runtimes be treated as direct predictions of Azure runtimes. In the Gold exploratory experiment, increased admitted task concurrency reduced processing-window duration while increasing summed task elapsed time. The observed reduction in elapsed time does not establish lower resource consumption. In the one-slot Gold study, inter-task gaps account for approximately half the mean window; the contributions of gap avoidance and concurrent execution to the improvement are not independently identified.

6.3 Contributions and Boundaries

The study compares configured task priorities with observed task starts and processing times in a defined Airflow–Databricks workload. The results show which priority configurations selected different eligible tasks and which produced the same execution order. The study also distinguishes model-derived priorities from the predefined priorities used to isolate particular scheduling mechanisms in the controlled experiments.

These conclusions are bounded by a single-node cluster, a selected TPC-DS SF1 subset, small layer DAGs, five remote repetitions per cell, sequential cloud execution, differing local and Azure

priority implementations, and an XGBoost model trained on engineered targets for familiar task identities. The output checks support consistency at the reported numerical precision but do not establish independent row-by-row correctness. The earlier and repeated cloud benchmarks also produced different Silver and Gold mean rankings and strongly different Gold runtimes; different recording boundaries and execution sessions preclude interpreting this discrepancy as the effect of any one strategy. Because all strategies read prepared upstream snapshots within each layer, the three observed processing windows cannot be added into a realised strategy-specific end-to-end makespan.

6.4 Further Research

Future experiments should block or randomise the cloud treatment sequence across independently prepared sessions, compare multiple workload sizes and cluster types, and directly record Spark resource consumption and task-pair slowdown. To test learned generalisation, model validation should hold out task identities or complete DAG families and train against observed performance consequences rather than only engineered single-task scores. Task-set-aware prediction and validated resource telemetry could then support a scheduler that explicitly balances latency and resource pressure.

6.5 Final Perspective

For practical deployment, the first diagnostic should be whether the proposed priority rule actually changes which ready tasks are admitted under the available slot limit. When it does not, apparent timing advantages require explanations other than the selected task order. When prerequisites unlock multiple successors, early admission can matter; when independent tasks share a small cluster, the opening pair and between-wave delays also warrant measurement. These recommendations describe the evaluated case study and are not a universal policy.

A further experimental lesson is to retain both the planned priority settings and the actual task-start records. Without that comparison, faster completion can be mistaken for a scheduling improvement even when task admission is unchanged. Across the evaluated layers, additional priority complexity did not consistently shorten processing windows. The evidence favours checking the effective schedule and its variability before replacing a transparent baseline.

Appendix A

Detailed Experimental Configuration

A.1 Observation Counts

The complete experiment inventory is reported in Table 3.1. The detailed treatment counts are:

Table A.1: Detailed repetition and task-observation counts.

Phase	Treatment unit	Runs	Task observations
Local	Each layer-by-strategy cell	10	Layer-dependent
Local	All nine cells	90	390
Azure benchmark	Each layer-by-strategy cell	5	Layer-dependent
Azure benchmark	All nine cells	45	195
Bronze unlocking	Each root-order condition	5	30
Gold pairing	Each opening-pair condition	5	15
Gold concurrency	Each slot condition	5	15
All phases	Complete retained record	175	735

A.2 Local Simulated Task-Duration Inputs

The numbers in this table are configured sleep-based simulation inputs, not profiled Spark timings. Cloud HEFT priorities used a different historical cost configuration and are listed separately below.

A.3 Azure Priority Weights

The following priorities were recorded during the Azure benchmark. The XGBoost column contains integer priorities derived from the saved model predictions rather than the engineered training targets. They are fixed within a task identity and layer across all five repetitions.

Table A.2: Configured local simulation base durations (seconds).

Layer	Computational task	Base duration (s)
Bronze	Date dimension	0.5
Bronze	Item	1.2
Bronze	Customer	1.8
Bronze	Web sales	3.2
Bronze	Store sales	4.5
Bronze	Catalog sales	5.1
Silver	Clean sales	6.0
Silver	Clean customer	2.5
Silver	Clean item	1.8
Silver	Join fact sales	9.5
Gold	Top items	5.5
Gold	Sales by region	7.0
Gold	Customer LTV	8.5

A.3.1 Bronze

Table A.3: Bronze task priorities for all five repetitions.

Task	Static	HEFT	Model-derived XGBoost
`ingest_catalog_sales`	100	797	465
`ingest_customer`	400	1886	531
`ingest_date_dim`	600	1868	671
`ingest_item`	500	1858	748
`ingest_store_sales`	200	1622	365
`ingest_web_sales`	300	813	149

A.3.2 Silver

Table A.4: Silver task priorities for all five repetitions.

Task	Static	HEFT	Model-derived XGBoost
`clean_customer`	300	6633	546
`clean_item`	200	3979	505
`clean_sales`	400	7585	749
`join_fact_sales`	100	3549	251

A.3.3 Gold

Table A.5: Gold task priorities for all five repetitions.

Task	Static	HEFT	Model-derived XGBoost
`agg_customer_ltv`	100	32974	749
`agg_sales_by_region`	200	47668	575
`agg_top_items`	300	47663	400

A.4 Mechanism Configurations

Table A.6: Controlled mechanism-study conditions.

Study	Condition	Repetitions
Bronze unlocking	date_dim + item first	5
Bronze unlocking	customer + date_dim first; item delayed	5
Gold pairing	Customer LTV + Top Items first	5
Gold pairing	Customer LTV + Sales by region first	5
Gold pairing	Top Items + Sales by region first	5
Gold concurrency	max_active_tasks = 1	5
Gold concurrency	max_active_tasks = 2	5
Gold concurrency	max_active_tasks = 3	5

A.5 Experiment Identification

Run identifiers distinguish observations from the local benchmark, the Azure Databricks benchmark and the three controlled cloud experiments. They allow individual executions to be traced to their corresponding task records and prevent observations from separate studies being pooled.

A.6 Silver Sensor Treatment

The local Silver DAG contains the cross-layer sensor `wait_for_bronze`. It is excluded from the computational benchmark because it does not execute a Silver transformation. Azure uses prepared Bronze outputs and evaluates the same four computational tasks without the sensor.

A.7 Software and Infrastructure

Table A.7: Key software and infrastructure versions.

Component	Configuration
Apache Airflow	2.9.0 in Docker
Databricks provider	apache-airflow-providers-databricks 7.0.0
Python graph library	NetworkX 3.6.1
Azure Databricks runtime	17.3.16 LTS; Spark 4.0; Scala 2.13
Azure VM	Standard_D4ds_v4, single node, 4 cores, 16 GB
Photon	Disabled
Unity Catalog	catalog `dbw_thesis_ml_validation`; schema `thesis_etl_scheduling`
Metrics table	`etl_task_run_metrics_azure`

A.8 Saved-Model Priority Verification

The table compares the saved model's predictions with the corresponding integer priorities used by Airflow in the Azure Databricks benchmark. File checksums were used to confirm the model and feature data underlying these values. *Interpretation.* The 65 XGBoost task observations

Table A.8: XGBoost model predictions and the corresponding Airflow task priorities.

Layer	Task	Predicted score	Airflow priority
Bronze	`ingest_item`	748.337	748
Bronze	`ingest_date_dim`	670.530	671
Bronze	`ingest_customer`	530.681	531
Bronze	`ingest_catalog_sales`	464.733	465
Bronze	`ingest_store_sales`	364.775	365
Bronze	`ingest_web_sales`	148.504	149
Silver	`clean_sales`	749.114	749
Silver	`clean_customer`	545.598	546
Silver	`clean_item`	505.404	505
Silver	`join_fact_sales`	250.996	251
Gold	`agg_customer_ltv`	749.276	749
Gold	`agg_sales_by_region`	574.810	575
Gold	`agg_top_items`	400.493	400

use the priorities shown above, unchanged across repetitions. The table establishes the applied scoring policy but does not demonstrate generalisation to unseen tasks, online adaptation or optimal scheduling.

A.9 Observed Opening Pairs in Every Benchmark Run

Table A.9: All 45 observed benchmark opening pairs, by remote job-start order.

Layer	Strategy	Round	First	Second
Bronze	HEFT	1	Cust	Date
Bronze	Static	1	Date	Item
Bronze	XGBoost	1	Item	Date
Bronze	HEFT	2	Date	Cust
Bronze	Static	2	Date	Item
Bronze	XGBoost	2	Item	Date
Bronze	HEFT	3	Date	Cust
Bronze	Static	3	Item	Date
Bronze	XGBoost	3	Item	Date
Bronze	HEFT	4	Date	Cust
Bronze	Static	4	Date	Item
Bronze	XGBoost	4	Item	Date
Bronze	HEFT	5	Cust	Date
Bronze	Static	5	Item	Date
Bronze	XGBoost	5	Date	Item
Gold	HEFT	1	Region	Top
Gold	Static	1	Top	Region
Gold	XGBoost	1	Region	LTV
Gold	HEFT	2	Region	Top
Gold	Static	2	Region	Top
Gold	XGBoost	2	LTV	Region
Gold	HEFT	3	Top	Region
Gold	Static	3	Region	Top
Gold	XGBoost	3	LTV	Region
Gold	HEFT	4	Region	Top
Gold	Static	4	Region	Top
Gold	XGBoost	4	LTV	Region
Gold	HEFT	5	Region	Top
Gold	Static	5	Top	Region
Gold	XGBoost	5	LTV	Region
Silver	HEFT	1	Sales	Cust
Silver	Static	1	Cust	Sales
Silver	XGBoost	1	Cust	Sales
Silver	HEFT	2	Cust	Sales
Silver	Static	2	Sales	Cust
Silver	XGBoost	2	Sales	Cust
Silver	HEFT	3	Sales	Cust
Silver	Static	3	Cust	Sales
Silver	XGBoost	3	Sales	Cust
Silver	HEFT	4	Sales	Cust
Silver	Static	4	Cust	Sales
Silver	XGBoost	4	Sales	Cust
Silver	HEFT	5	Cust	Sales
Silver	Static	5	Cust	Sales
Silver	XGBoost	5	Sales	Cust

Appendix B

Robustness and Integrity Details

B.1 Earlier and Repeated Benchmark Comparison

Table B.1: Earlier and repeated cloud benchmark means (seconds; five runs per cell).

Layer	Strategy	Earlier mean	Repeated mean
Bronze	HEFT	141.546	246.057
Bronze	Static	128.545	226.697
Bronze	XGBoost	118.035	187.639
Gold	HEFT	90.614	353.782
Gold	Static	93.902	399.317
Gold	XGBoost	96.323	346.976
Silver	HEFT	122.541	152.387
Silver	Static	117.797	165.323
Silver	XGBoost	123.276	154.377

The earlier XGBoost policy used the engineered formula directly, whereas the reported repeated benchmark used saved-model-derived priorities. Both datasets have five recorded runs per strategy–layer cell, but their measurement provenance and execution contexts differ. The earlier 45 runs are not pooled with the primary 175-run analysis; see the comparison in this appendix section. The comparison is descriptive and cannot distinguish the influence of cloud state, code differences or different measurement boundaries.

B.2 Output-Integrity Scope

The Azure benchmark produced 45 successful DAG runs and 195 successful remote task executions. All 195 corresponding Delta tables were readable, and all 105 independently specified row-count expectations were met. Among 13 groups of equivalent logical tasks, the only exact numerical differences occurred in the Gold top-item aggregation and disappeared when aggregate values were rounded to eight decimal places. The controlled mechanism studies had consistent aggregate output totals but were not subjected to the same complete table-level

checks. These findings support output consistency within the stated scope rather than exact equality against independent ground truth.

B.3 Output Verification Method

The 195 run-specific Delta tables were checked without changing their contents. The verification examined availability, row counts where independent expectations existed, schema consistency, source identifiers and numerical agreement for comparable logical tasks. These were layer-specific comparisons, not an end-to-end execution of all three layers. For the 15 Gold top-item tables, results from the two numerical variants had the same 1,000 item keys, rankings and integer measures. Differences in floating-point aggregates were below 10^{-8} and disappeared at eight-decimal rounding, although exact `EXCEPT` comparisons still identified unequal values.

A separate check reconciled all 195 benchmark task identifiers and their stored priority values. For the first nine Bronze runs, corresponding to 54 tasks, supplementary metrics were reconstructed from archived records; the remaining 141 entries used the later collection procedure. Four preliminary test records were excluded. The original Airflow priorities and Databricks job histories are the primary evidence for scheduling decisions and remote execution timing. A separate Databricks export recovered successful job records for all 195 benchmark tasks.

B.4 Cloud Robustness

The Azure processing-window robustness results are shown in Table 4.5 and Figure 4.5. The Gold XGBoost–HEFT ordering changes in a sensitivity analysis that omits Round 1 from both strategy samples. Every completed run, including extreme results, is retained in the main five-run estimates. The independent-group bootstrap intervals and leave-one-round-out ranges are descriptive diagnostics only; they do not establish treatment causality, independence of cloud state or statistical equivalence.

B.5 Run-Order Diagnostics

The Azure benchmark runs were executed sequentially with strategy order rotated across rounds, as documented by the archived run start timestamps; this did not provide a fully randomised or cloud-state-matched design. Databricks job durations fluctuated across the session. In particular, the Gold XGBoost cell includes a fast Round 1 and slow Round 5; an orchestration host restart was recorded near the end of the Gold measurements. These facts support describing cloud-state, caching and run order as potential sources of variation. The separately conducted exploratory studies retain their own run-order diagnostics and are not treated as matched blocks with the Azure benchmark.

B.6 Gold Orchestration Gaps

Under concurrency one, the first task is followed by a mean gap of approximately 33.11 s before the second task begins, and the second is followed by a mean gap of approximately 29.66 s before the third begins. The total sequential gap is therefore approximately 62.77 s. Under concurrency two, the second-wave Sales-by-region task begins approximately 33.92 s after the first opening task frees a slot and 20.49 s after both opening tasks finish. These intervals show that time between recorded task executions contributes to the longer processing window at low concurrency; the measurements do not isolate Databricks submission latency from other causes.

Bibliography

Alipourfard, O., Liu, H. H., Chen, J., Venkataraman, S., Yu, M. and Zhang, M. (2017), CherryPick: Adaptively unearthing the best cloud configurations for big data analytics, *in* 'Proceedings of the 14th USENIX Symposium on Networked Systems Design and Implementation', pp. 469–482.

Apache Software Foundation (2026*a*), 'Priority weights: Apache Airflow 2.9.0 documentation', `https://airflow.apache.org/docs/apache-airflow/2.9.0/administration-and-deployment/priority-weight.html`. Accessed: 2026-07-27.

Apache Software Foundation (2026*b*), 'Scheduler: Apache Airflow documentation', `https://airflow.apache.org/docs/apache-airflow/2.9.0/administration-and-deployment/scheduler.html`. Accessed: 2026-07-27.

Arabnejad, H. and Barbosa, J. G. (2014), 'List scheduling algorithm for heterogeneous systems by an optimistic cost table', *IEEE Transactions on Parallel and Distributed Systems* **25**(3), 682–694.

Armbrust, M., Das, T., Sun, L., Yavuz, B., Zhu, S., Murthy, M., Torres, J., van Hovell, H., Ionescu, A., Łuszczak, A., Switakowski, M., Szafranski, M., Li, X., Ueshin, T., Mokhtar, M., Boncz, P., Ghodsi, A., Paranjpye, S., Senster, P., Xin, R. and Zaharia, M. (2020), 'Delta Lake: High-performance ACID table storage over cloud object stores', *Proceedings of the VLDB Endowment* **13**(12), 3411–3424.

Armbrust, M., Ghodsi, A., Xin, R. and Zaharia, M. (2021), Lakehouse: A new generation of open platforms that unify data warehousing and advanced analytics, *in* 'Proceedings of the 11th Annual Conference on Innovative Data Systems Research'.

Armbrust, M., Xin, R. S., Lian, C., Huai, Y., Liu, D., Bradley, J. K., Meng, X., Kaftan, T., Franklin, M. J., Ghodsi, A. and Zaharia, M. (2015), Spark SQL: Relational data processing in Spark, *in* 'Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data', pp. 1383–1394.

Chen, T. and Guestrin, C. (2016), XGBoost: A scalable tree boosting system, *in* 'Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining', pp. 785–794.

Databricks (2026), 'What is the Medallion Lakehouse architecture?', `https://learn.microsoft.com/en-us/azure/databricks/lakehouse/medallion`. Accessed: 2026-10-20.

Delimitrou, C. and Kozyrakis, C. (2013), Paragon: QoS-aware scheduling for heterogeneous datacenters, *in* 'Proceedings of the 18th International Conference on Architectural Support for Programming Languages and Operating Systems', pp. 77–88.

Delimitrou, C. and Kozyrakis, C. (2014), Quasar: Resource-efficient and QoS-aware cluster management, *in* 'Proceedings of the 19th International Conference on Architectural Support for Programming Languages and Operating Systems', pp. 127–144.

Efron, B. (1979), 'Bootstrap methods: Another look at the jackknife', *The Annals of Statistics* **7**(1), 1–26.

Gao, K., Hu, Y., Nie, C. and Li, W. (2025), 'Deep q-learning-based intelligent scheduling for ETL optimization in heterogeneous data environments', arXiv preprint arXiv:2512.13060. **URL:** *https://arxiv.org/abs/2512.13060*

Ghodsi, A., Zaharia, M., Hindman, B., Konwinski, A., Shenker, S. and Stoica, I. (2011), Dominant Resource Fairness: Fair allocation of multiple resource types, *in* 'Proceedings of the 8th USENIX Symposium on Networked Systems Design and Implementation', pp. 323–336.

Gog, I., Schwarzkopf, M., Gleave, A., Watson, R. N. M. and Hand, S. (2016), Firmament: Fast, centralized cluster scheduling at scale, *in* 'Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation', pp. 99–115.

Grandl, R., Ananthanarayanan, G., Kandula, S., Rao, S. and Akella, A. (2014), Multi-resource packing for cluster schedulers, *in* 'Proceedings of the 2014 ACM Conference on SIGCOMM', pp. 455–466.

Hindman, B., Konwinski, A., Zaharia, M., Ghodsi, A., Joseph, A. D., Katz, R. H., Shenker, S. and Stoica, I. (2011), Mesos: A platform for fine-grained resource sharing in the data center, *in* 'Proceedings of the 8th USENIX Symposium on Networked Systems Design and Implementation', pp. 295–308.

Holm, S. (1979), 'A simple sequentially rejective multiple test procedure', *Scandinavian Journal of Statistics* **6**(2), 65–70.

Jain, R. (1991), *The Art of Computer Systems Performance Analysis*, Wiley.

Karagiannis, A., Vassiliadis, P. and Simitsis, A. (2013), 'Scheduling strategies for efficient ETL execution', *Information Systems* **38**(6), 927–945.

Leitner, P. and Cito, J. (2016), 'Patterns in the chaos—a study of performance variation and predictability in public IaaS clouds', *ACM Transactions on Internet Technology* **16**(3), 1–23.

Ma, L., Hu, Y.-X., Wang, Y., Zhao, Y., Ren, Y., Liao, J.-X., Wu, F. and Li, X.-Y. (2025), 'LeJOT: An intelligent job cost orchestration solution for databricks platform', arXiv preprint arXiv:2512.18266.
URL: *https://arxiv.org/abs/2512.18266*

Mann, H. B. and Whitney, D. R. (1947), 'On a test of whether one of two random variables is stochastically larger than the other', *The Annals of Mathematical Statistics* **18**(1), 50–60.

Mao, H., Schwarzkopf, M., Venkatakrishnan, S. B., Meng, Z. and Alizadeh, M. (2019), Learning scheduling algorithms for data processing clusters, *in* 'Proceedings of the ACM Special Interest Group on Data Communication', pp. 270–288.

Maricq, A., Duplyakin, D., Jimenez, I., Maltzahn, C., Stutsman, R. and Ricci, R. (2018), Taming performance variability, *in* 'Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation', pp. 409–425.

Ousterhout, K., Wendell, P., Zaharia, M. and Stoica, I. (2013), Sparrow: Distributed, low latency scheduling, *in* 'Proceedings of the 24th ACM Symposium on Operating Systems Principles', pp. 69–84.

Simitsis, A., Vassiliadis, P. and Sellis, T. K. (2005), Optimizing ETL processes in data warehouses, *in* 'Proceedings of the 21st International Conference on Data Engineering', pp. 564–575.

Topcuoglu, H., Hariri, S. and Wu, M.-Y. (2002), 'Performance-effective and low-complexity task scheduling for heterogeneous computing', *IEEE Transactions on Parallel and Distributed Systems* **13**(3), 260–274.

Transaction Processing Performance Council (2026), 'TPC-DS benchmark and current specifications', `https://www.tpc.org/tpcds/`. Accessed: 2026-07-27.

Uta, A., Custura, A., Duplyakin, D., Jimenez, I., Rellermeyer, J., Maltzahn, C., Ricci, R. and Iosup, A. (2020), Is big data performance reproducible in modern cloud networks?, *in* 'Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation', pp. 513–527.

Vargha, A. and Delaney, H. D. (2000), 'A critique and improvement of the CL common language effect size statistics of McGraw and Wong', *Journal of Educational and Behavioral Statistics* **25**(2), 101–132.

Vavilapalli, V. K., Murthy, A. C., Douglas, C., Agarwal, S., Konar, M., Evans, R., Graves, T., Lowe, J., Shah, H., Seth, S., Saha, B., Curino, C., O'Malley, O., Radia, S., Reed, B.

and Baldeschwieler, E. (2013), Apache Hadoop YARN: Yet another resource negotiator, *in* 'Proceedings of the 4th Annual Symposium on Cloud Computing'.

Venkataraman, S., Yang, Z., Franklin, M., Recht, B. and Stoica, I. (2016), Ernest: Efficient performance prediction for large-scale advanced analytics, *in* 'Proceedings of the 13th USENIX Symposium on Networked Systems Design and Implementation', pp. 363–378.

Verma, A., Pedrosa, L., Korupolu, M., Oppenheimer, D., Tune, E. and Wilkes, J. (2015), Large-scale cluster management at Google with Borg, *in* 'Proceedings of the 10th European Conference on Computer Systems'.

Zaharia, M., Chowdhury, M., Das, T., Dave, A., Ma, J., McCauley, M., Franklin, M. J., Shenker, S. and Stoica, I. (2012), Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing, *in* 'Proceedings of the 9th USENIX Symposium on Networked Systems Design and Implementation', pp. 15–28.